G Web Development Software

ni.com

G Web Development Software

Contents

1. G Web Development Software . . . 6

1.1 Getting Started with G Web Development Software . . . 7

1.1.1 Keyboard Shortcuts . . . 7

1.1.2 Tips and Tricks for Editing Diagram Code . . . 13

1.1.3 Customizing G Web Development Software . . . 15

1.1.3.1 Customizing Mouse Wheel Behavior . . . 15

1.1.3.2 Displaying Tabs in the Editor . . . 16

1.1.3.3 Aligning Objects on the Panel . . . 16

1.1.3.4 Displaying the Diagram Grid . . . 16

1.1.3.5 Resetting the Workspace . . . 17

1.1.4 Preview Features . . . 17

1.1.5 Enabling SystemDesigner Visibility . . . 18

1.2 Developing a Web Application . . . 18

1.2.1 Creating a Web Application . . . 18

1.2.1.1 Applications . . . 19

1.2.1.2 Libraries . . . 22

1.2.1.3 Creating User Interfaces . . . 27

1.2.1.4 Debugging a WebVI . . . 77

1.2.1.5 Building a Web Application as a Web Page . . . 78

1.2.1.6 Linking WebVIs in a Web Application . . . 79

1.2.1.7 Creating Localized String Dictionaries . . . 80

1.2.1.8 Enabling SystemDesigner Visibility . . . 83

1.2.1.9 Using JavaScript with a Web Application . . . 84

1.2.1.10 Hosting a Web Application on a Server . . . 98

1.2.2 Running Operations Using the Command Line Interface . . . 102

1.2.2.1 Global Arguments . . . 103

1.2.3 Predefined Command Line Operations . . . 103

1.2.3.1 build-application . . . 103

1.2.3.2 build-library . . . 105

1.2.3.3 optimize-project . . . 106

1.2.3.4 distribution . . . 107

1.2.3.5 applylicensing . . . 109

1.3 Packaging an Application or Library . . . 110

1.3.1 Considerations for Packaging a Web Application . . . 111

1.3.2 Packaging an Application . . . 111
1.3.3 Packaging a Library . . . 112
1.3.4 Types of Package Outputs . . . 114
1.4 Collaborating on Web Applications . . . 115
1.4.1 Package Dependencies . . . 116
1.4.1.1 Sharing a Project and Including Package Dependencies . . . 116
1.4.1.2 Capturing the Package Dependencies of a Project . . . 117
1.4.1.3 Resolving Missing or Mismatched Package Dependencies on a Development System . . . 117
1.4.2 Recommendations for Developing a Project Stored in a Source Code Control Repository . . . 118
1.4.2.1 Source Code Control Models . . . 121
1.4.2.2 Recommendations for Interacting with a Centralized Source Code Control System while Developing a Project . . . 121
1.4.2.3 Recommendations for Developing a Project in a Distributed Source Code Control System . . . 122
1.4.2.4 Merging Files and Resolving Conflicts . . . 123
1.4.3 Code Snippets . . . 124
1.4.3.1 Creating a Code Snippet . . . 124
1.4.3.2 Adding a Code Snippet to the Diagram . . . 125
1.5 Communicating Data with a Web Application . . . 125
1.5.1 Accessing Resource Files from a WebVI . . . 126
1.5.2 Retrieving Data From a Web Service . . . 127
1.5.3 Considerations When Accessing Data from Web Services . . . 130
1.5.3.1 Types of Cross-Origin HTTP Requests . . . 131
1.5.3.2 Enabling CORS for a Web Service . . . 132
1.5.3.3 Configuring CORS for a LabVIEW Web Service . . . 134
1.5.3.4 Sending a Credentialed Cross-Origin HTTP Request . . . 141
1.5.4 Communicating Data with Web Services Using WebVIs . . . 143
1.6 Hosting a WebVI in SystemLink . . . 145
1.7 Using Hardware with a Web Application . . . 145
1.7.1 Using a Web Application with a CompactRIO Device . . . 148
1.8 Converting LabVIEW NXG Web Module Projects to G Web Development Software Projects . . . 152
1.8.1 Differences between LabVIEW and G Web Development Software . . . 152
1.9 Programming in G . . . 155
1.9.1 Nodes: Computational Units . . . 156
1.9.2 Wires: Transferring Data between Nodes . . . 156
1.9.2.1 Troubleshooting Broken Wires . . . 157
1.9.2.2 Wiring Best Practices . . . 157

1.9.2.3 Wiring Shortcuts . . . 158
1.9.3 Constants . . . 159
1.9.4 SubVIs . . . 159
1.9.4.1 Creating a SubVI from a Section of Existing Code . . . 161
1.9.4.2 VI Reentrancy . . . 162
1.9.4.3 Choosing the Right VI Reentrancy Option for a SubVI . . . 163
1.9.4.4 Minimizing SubVI Overhead . . . 164
1.9.5 Terminals . . . 164
1.9.5.1 Data Transfer between the Panel and the Diagram . . . 165
1.9.5.2 Dataflow between the Diagram and Another VI . . . 165
1.9.5.3 Dataflow between Duplicates of the Same Terminal . . . 166
1.9.6 Opening, Processing, and Closing Files . . . 167
1.9.7 Strategies for Improving VI Execution Speed . . . 169
1.9.8 Repeating Operations . . . 171
1.9.8.1 Types of Loops . . . 172
1.9.8.2 Repeating Operations until a Condition Occurs . . . 172
1.9.8.3 Repeating Operations a Set Number of Times . . . 174
1.9.8.4 Repeating Operations Once for Every Element in an Array . . . 175
1.9.8.5 Synchronizing the Execution of Multiple Loops . . . 177
1.9.8.6 Accessing Data from the Previous Loop Iteration . . . 178
1.9.8.7 Accessing Data from Multiple Past Loop Iterations . . . 180
1.9.8.8 Executing Code Based on a Condition . . . 181
1.9.8.9 Adjusting the Execution Speed of a Loop . . . 183
1.9.8.10 Preventing Data Loss When Transferring Data Between Loops . 184
1.9.9 Adding Code Snippets to a Custom Palette . . . 188
1.9.10 Data Type Reference . . . 189
1.9.10.1 Numeric Data . . . 191
1.9.10.2 Boolean Data . . . 195
1.9.10.3 String Data . . . 196
1.9.10.4 Arrays . . . 199
1.9.10.5 Clusters: Building and Interacting with Grouped Data . . . 201
1.9.10.6 Comparison Modes for Array and Cluster Data . . . 203
1.9.10.7 Enum Data . . . 204
1.9.10.8 Type Definitions . . . 205
1.9.11 Programming Design Patterns . . . 207
1.9.11.1 Event-Driven Programming . . . 207
1.9.11.2 State Machine Design Pattern . . . 219
1.9.12 Best Practices for Creating Projects in G Web Development Software . 226
1.9.12.1 File and Project Organization in G Web Development Software 226

1.9.12.2 Icons and Connector Panes for G Web Development Software Projects . . . 227
1.9.12.3 Panel Design for G Web Development Software Projects . . . 231
1.9.12.4 Diagram Design for G Web Development Software Projects . . . 238
1.9.13 Best Practices for Designing and Developing an Application Programming Interface (API) in G Web Development Software . . . 244
1.9.13.1 File Organization and Node Naming for Distributed APIs . . . 245
1.9.13.2 Component Organization for Distributed APIs . . . 246
1.9.13.3 Icons and Connector Panes for Distributed APIs . . . 247
1.9.13.4 Data Type Selection for Distributed APIs . . . 252
1.9.13.5 Palette Taxonomy for Distributed APIs . . . 254
1.9.13.6 Documentation for Distributed APIs . . . 255
1.9.13.7 Error Message Design for Distributed APIs . . . 256
1.9.13.8 API Design for Distributed APIs . . . 256
1.10 Testing and Debugging . . . 257
1.10.1 Using Probes to Check Values on a Wire . . . 257
1.10.2 Identifying Errors That Prevent You from Running Code . . . 258
1.10.3 Improve Applications with Execution Logs . . . 259
1.10.4 Viewing the Hierarchy of VIs in Your Application . . . 259
1.10.5 Standard Error Behavior . . . 260
1.10.6 Error Codes . . . 261

G Web Development Software

Use the G Web Development Software to create browser-based user interfaces to visualize data from distributed systems. This manual contains step-by-step instructions for working with WebVIs and creating web applications.

Developing
Create, develop, and debug your web application.

Communicating Data
Send and receive data from files and web services with your web application.

Using JavaScript
Use JavaScript functions in your web application.

Hosting
Enable users to access your web application by hosting on a web server.

Sharing Code
Create add-ons and share code pieces by creating libraries.

- *Developing*—Create a web application, design responsive UIs and controls, and debug, build, and package your WebVIs.
- *Communicating Data*—Access WebVI resource files, web service data, and use web services like SystemLink Tag and Message in your web application.
- *Using JavaScript*—Add JavaScript functions to your web application using a JavaScript Library Interface (JSLI) document and create custom UI elements.
- *Hosting*—Learn about your hosting options during development and how to host your web application to share with users.
- *Sharing Code*—Build a library and add custom palettes to share code and create add-ons.

TIP
To access examples, on the **Learning** tab, click **Examples**»**Programming WebVIs**.

Manuals for Related Products

- *NI Web Server*
- *SystemLink*
- *SystemLink Cloud*

Getting Started with G Web Development Software

G Web Development Software is a science and engineering-focused programming environment for designing and testing systems within a visual code diagram. The **Projects** tab appears when you launch G Web Development Software. Select either the **Projects** or **Learning** tab to get started.

From the **Projects** tab you can:

- Create a new blank web virtual instrument (webVI) project.
- Create a *new project webVI* based on a template.
- Open a recent project.
- Access help documentation from the **right panel** to learn more about G Web Development Software.

The **Projects** tab will close once you make your selection. You can access it again by clicking **Home** in the upper-left corner of the document.

Keyboard Shortcuts

The following tables list keyboard shortcuts in the environment.

File Operations

Shortcut	Action
Ctrl-N	Open a new document and add it to the existing project.
Ctrl-Shift-N	Open a new project.
Ctrl-O	Open an existing project.
Ctrl-W	Close the current document.
Ctrl-S	Save the current file.
Ctrl-Shift-S	Save all open files.
Ctrl-P	Print the current document.
Alt-F4	Quit.

Basic Editing

Shortcut	Action
Ctrl-X Shift-Delete	Cut.
Ctrl-C Ctrl-Insert	Copy.
Ctrl-V Shift-Insert	Paste.
Ctrl-Z Alt-Backspace	Undo.
Ctrl-Y Alt-Shift-Backspace	Redo.
Shift-F10 Application Key	Open shortcut menu for selected item.
Delete	Delete.
Ctrl-Delete	Delete and rewire.

Selecting and Moving Objects

Shortcut	Action
Shift-Click	Select multiple objects. Add object to the current selection.
Ctrl-A	Select all objects.
Arrow keys	Move selected objects in grid-sized increments.
Shift-Arrow keys	Move selected objects four grid units.
Ctrl-Drag	Copy and drag selected object.
Ctrl-Shift-Drag	Copy selected object and move it along one axis.
Shift-Resize	Resize selected object while maintaining aspect ratio.

Shortcut	Action
Ctrl-Resize	Resize selected object while maintaining center point.
Shift-Ctrl-Resize	Resize selected object while maintaining both aspect ratio and center point.
Ctrl-Drag open area	Create additional blank space along the axis you drag the mouse.
Double-click open area	Add free label or comment to panel or diagram.
Spacebar-Drag	Pan across panel or diagram.

Navigating the Environment

Shortcut	Action
Ctrl-F	Find and replace text or objects within a document.
Ctrl-Shift-F	Find and replace text or objects within a project.
Enter F3 Ctrl-G	Search document for next instance of text or an object. This command is only available when in Find mode.
Shift-Enter Shift-F3 Shift-Ctrl-G	Search document for previous instance of text or an object. This command is only available when in Find mode.
Ctrl-Tab	Cycle through document tabs in the order in which they appear onscreen.
Ctrl-Shift-Tab	Cycle through document tabs in the opposite order in which they appear onscreen.
Ctrl-Shift-Spacebar	Shift focus to the application-wide search bar.
Ctrl-\	Hide and show all tabs. Hide and show all panes.

Navigating the Panel and Diagram

Editor Command	Shortcut	Action
Panel and diagram selectors	Ctrl-E	Toggle between the panel and diagram view. If the icon view is active, switch to the panel view.
Search	Ctrl-Spacebar (Chinese keyboards) Ctrl-Alt-Spacebar	Shift focus to the palette search bar.
—	Ctrl-'	Toggle the diagram grid on/off.
—	Shift-Ctrl-;	Toggle **Smart Guides** on/off. **Smart Guides** help you align objects on the panel.
Vertical scroll bar	Mouse wheel	Scroll the document vertically.
Horizontal scroll bar	Shift-Mouse wheel	Scroll the document horizontally.
—	Tab	Shift focus from one control to another in tabbing order while the code is running.
—	Shift-Tab	Shift focus from one control to another in reverse tabbing order while the code is running.
—	Ctrl-Alt-I	Open and close the icon editor.

Help Commands

Shortcut	Action
Ctrl-H	Display the **Context Help**.
F1	Access additional information on ni.com.

Running Code

Editor Command	Shortcut	Action
Run	Ctrl-R	Execute this code.
Abort	Ctrl-.	Stop the code immediately, before it finishes executing.
	Ctrl-**Run** button	Recompile the code in the current document.

Editor Command	Shortcut	Action
	Ctrl-Shift-**Run** button	Recompile the code in all documents in memory.

Wiring

Editor Command	Shortcut	Action
Remove Broken Wires	Ctrl-B	Delete all broken wires from the diagram.
—	Esc Right-click Ctrl-Z	Delete a wire you are in the process of creating.
—	Single-click wire	Select one wire segment.
—	Double-click wire	Select a wire branch.
—	Triple-click wire	Select the entire wire.
—	Ctrl-click wire	Create a new wire branch from an existing wire.
—	Single-click while wiring	Tack down the wire segment and start a new wire segment.
—	Double-click while wiring	End the wire without connecting it to a node.
—	Tap spacebar while wiring	Switch the direction of a wire between horizontal and vertical.
Clean Up Diagram Clean Up Selection	Ctrl-U	Organize the diagram or the selected code to make it easier to understand.
Clean Up Wire	Select **Clean Up Wire** from the shortcut menu	Route a selected wire to decrease the number of bends in the wire and avoid crossing objects on the diagram.

Editing Text

Shortcut	Action
Double-click text	Select a single word in a string.
Triple-click text	Select the entire string.

Shortcut	Action
Ctrl-Right arrow Ctrl-Left arrow	Move the cursor within a string by one word in the direction of the arrow.
Home	Move the cursor to the beginning of the current line.
End	Move the cursor to the end of the current line.
Ctrl-Home	Move the cursor to the beginning of the string.
Ctrl-End	Move the cursor to the end of the string.
Esc	Cancel text entry.
Ctrl-Enter	Submit text entry.

Navigating the Project Files and Captured Data Tabs

Shortcut	Action
* (Numeric keypad)	Expand everything under the selected folder.
+ (Numeric keypad)	Expand the selected folder.
- (Numeric keypad)	Collapse the selected folder.
Right arrow	Expand the selected folder if it is closed. Otherwise, this keyboard shortcut selects the first child.
Left arrow	Collapse the selected folder if it is open. Otherwise, this keyboard shortcut selects the parent.
Ctrl-up arrow Ctrl-down arrow	Scroll through the pane without changing the current selection.
Any printable key(s)	Select the item beginning with the entered letter(s).
Enter	Open the selected document.
F2	Rename the selected item.
Page Up Home	Move the selection to the first item in the tree.
Page Down End	Moves the selection to the last item in the tree.

Zooming

Shortcut	Action
Ctrl-Mouse wheel	Zoom.
Ctrl-+	Zoom in.
Ctrl--	Zoom out.
Ctrl-0	Zoom to fit.
Ctrl-9	Zoom to fit the selection.

Tips and Tricks for Editing Diagram Code

The following time-saving tools can help you edit diagram code more efficiently.

Creating Diagram Code

Task	Tips and Tricks
Create constants, controls, and indicators using the shortcut menu.	You can save time when adding constants, controls, and indicators by using the shortcut menu to add the object instead of navigating the palette. Right-click any input or output of a node and select one of the Create options from the shortcut menu. For example, to create a constant, select the `Create Constant` icon.
Create a subVI from existing code.	You can use the **Create SubVI from Selection** option on the document toolbar to automatically generate a new subVI from a section of existing code. Refer to *Creating a SubVI from a Section of Existing Code* for more information.

Editing Diagram Code

Task	Tips and Tricks
Replace a diagram object with a similar object without using the palette.	You can replace an object on the diagram, such as a node, terminal, or constant, with a different type of the same object without navigating the palette. For example, you can replace an Add node with a `Subtract` node, a `Numeric Terminal` with a `Boolean Terminal`, or a `True Constant` with a `False Constant`. Right-click the object, select **Replace** from the shortcut menu, and select the desired replacement object.
Remove a structure without deleting objects in the structure.	You can remove a loop, `Case Structure`, or `Flat Sequence` from the diagram without deleting the code it contains. Right-click the border of the structure and select **Remove *`Name of Structure`*** from the shortcut menu.

Organizing Diagram Code

Task	Tips and Tricks
Align or distribute diagram objects to improve the organization of the diagram.	On the document toolbar, click **Order**, **Align**, or **Distribute** to organize a set of selected objects.
Delete all broken wires at one time.	On the document toolbar, click **Remove Broken Wires** to delete all broken wires on the diagram.
Clean up wires.	You can automatically route a selected wire to decrease the number of bends in the wire and avoid crossing objects on the diagram. Right-click a wire and select **Clean Up Wire** from the shortcut menu.
Move a terminal label to improve diagram readability.	To change the position of a terminal label, select the terminal and update the **Label Placement** option in the **Item** tab.
Show a node label to improve diagram readability.	To show a node label, select the node and enable the **Show Label** option in the **Item** tab.

Customizing G Web Development Software

Customize the G Web Development Software environment by changing the mouse wheel behavior, displaying tabs in the editor, and adding guide lines to the panel and diagram.

Procedure

- *Customizing Mouse Wheel Behavior*—Change the action the mouse wheel performs within the workspace.
- *Displaying Tabs in the Editor*—Access information about your project and complete project-related tasks.
- *Aligning Objects on the Panel*—Display guide lines to help you align and organize objects on the panel.
- *Displaying the Diagram Grid*—Display guide lines to help you align and organize objects on the diagram.

What to do next

You can *reset the workspace* to the default settings.

RELATED INFORMATION

Customizing Mouse Wheel Behavior

You can change the action the mouse wheel performs within the workspace.

Procedure

1. Click **File**»**Preferences** to display the **Preferences** dialog box.
2. On the **Editor** tab, select the desired setting for Zoom Control.

Setting	Description
Zoom with Mouse Wheel	Scroll with the mouse wheel to zoom in and out within the workspace. To scroll vertically within the workspace, hold down the <Ctrl> key while scrolling with the mouse wheel.
Zoom with Ctrl + Mouse Wheel (Default)	Hold down the <Ctrl> key while scrolling with the mouse wheel to zoom in and out within the workspace. To scroll vertically within the workspace, scroll with the mouse wheel.

3. Click **OK**.

Displaying Tabs in the Editor

Display tabs in the editor to access different types of information about your project as well as to complete project-related tasks, such as troubleshooting or resource management.

Procedure

Click **View** and select the tab(s) you want to display from the pull-down menu.

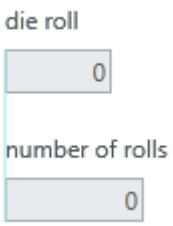
Aligning Objects on the Panel

When Smart Guides are enabled, the editor displays guide lines to help you align and organize objects as you add them to the panel.

Procedure

1. Click **File**»**Preferences** to display the **Preferences** dialog box.
2. On the **VI** tab, in the **Panel Editing Defaults** section, place a checkmark in the **Smart Guides** checkbox.

   As you add objects to the panel, the editor displays guide lines to assist with alignment, as shown in the following image.

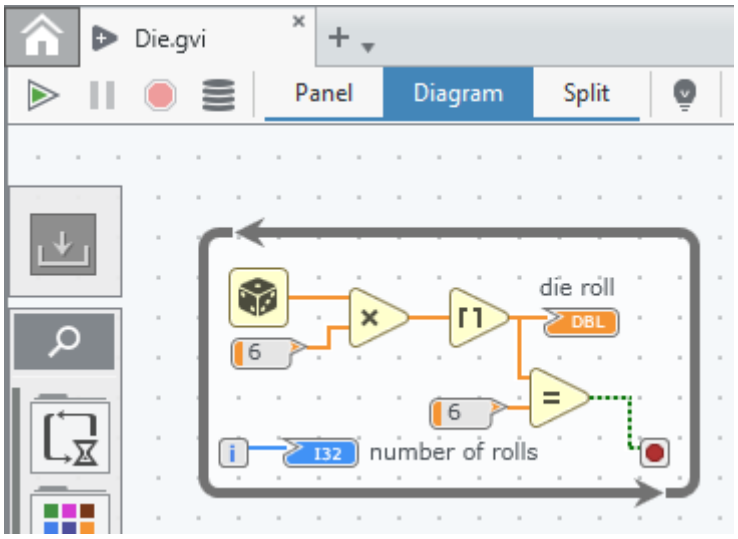


Displaying the Diagram Grid

As you place objects on the diagram, they automatically align to a grid. Making this grid visible can help you keep the diagram organized as you add and arrange objects.

Procedure

1. Click **File**»**Preferences** to display the **Preferences** dialog box.
2. On the **VI** tab, in the **Diagram Editing Defaults** section, place a checkmark in the **Always Show Grid** checkbox.



Resetting the Workspace

You can quickly restore the workspace to the default settings after you make adjustments.

Procedure

Click **View**»**Reset Workspace**.

- The Navigation Pane and Configuration Pane are expanded.
- The **Errors and Warnings** tab is collapsed on the bottom edge of the workspace.

Preview Features

Preview features are new features in G Web Development Software that support specific workflows. They are release-quality features, but they have functionality gaps and are only supported for specific uses.

Preview features in G Web Development Software:

- do not change compilation or execution of existing code.
- do not change the persistence of existing files.
- have supporting documentation. You can also request support at *ni.com/support*.

Procedure

Go to **File**»**Preferences**»**Preview features** and select the features to enable from the listed items.

Restart software for the changes to take effect.

RELATED INFORMATION

Enabling SystemDesigner Visibility

Update your preferences to see a visual representation of web targets in SystemDesigner.

Procedure

1. Click **File**»**Preferences**.
2. Select **Web Server** and enable **Show SystemDesigner**.
3. Reopen the project to apply the update.

RELATED INFORMATION

Developing a Web Application

Learn about the different tasks you complete to develop a web application using G Web Development Software.

- *Create a web application*—Write a program to run in a web browser.
- *Design a Responsive User Interface*—Create responsive interfaces and customize controls for your web application.
- *Debug WebVIs*—Learn about strategies to debug your WebVIs.
- *Build a web application*—Build and view your application in a web browser
- *Link WebVIs in a web application*—Link multiple WebVIs to create multiple web pages.
- *Package a web application*—Learn how to configure a package or package installer for your web application.

Creating a Web Application

Use the web application project template to create an application that can run in a web browser.

Procedure

1. In the **Projects** tab, click **Web Application Project**, name your project, and click **Create**.

   A web application project includes the following parts:

   Web application document (.gcomp) —The container for all the files in your application.

WebVI (.gviweb) —A specialized VI that generates HTML, JavaScript, and CSS files after you build a web application. Each WebVI marked as top-level in your application generates one HTML file, which corresponds to one web page after you build your web application.

Other files —(Optional) Any other files you want to include in your web application, such as CSS, JavaScript, and image files.

Web Server —Hosts the web application.

2. Open or create a WebVI within the application document and customize the WebVI for your unique programming goals.
3. (Optional) If you want to customize the appearance of your WebVI or add interactive functionality, create CSS and JavaScript files and include them in your application.
4. Test and debug your code.

   Select **Run**»**Run in browser** to test your WebVI in a browser.

   NOTE
   Unlike desktop VIs, WebVIs do not currently support debugging tools, such as execution highlighting, probes, and breakpoints.

5. Repeat steps 2–4 to create additional WebVIs.

What to do next

Once you finish creating and testing your application, build it into a web application that can run in a web browser.

RELATED INFORMATION

Building a Web Application as a Web Page on page 78

Debugging a WebVI on page 77

Unlike desktop VIs, WebVIs do not currently support debugging tools, such as execution highlighting, probes, and breakpoints. However, you can still debug a WebVI using the following strategies.

Applications

An *application* is a program designed to perform a group of coordinated functions or tasks. A web application is an application that is capable of running in web browsers.

Use an *Application* document (`.gcomp`) to create an application from scratch or from existing code. The Application document contains your source files, such as VIs, text files, and other types of files.

You can build your application into HTML, JavaScript, and CSS that you share on web servers and run in web browsers outside the development environment.

Creating a WebVI

Create a webVI (`.gviweb`) in which you can write a program.

Procedure

1. In the Navigation pane, click **New**»**WebVI**.

   NOTE
   Top-level VIs must be WebVIs.

2. Complete any of the following tasks to accomplish your programming goals:
   - On the diagram, write code that performs your desired task when the program executes.
   - On the panel, ***build a user interface***.
   - On the icon editor, add inputs and outputs to ***configure a WebVI or a VI as a SubVI***. After you save the SubVI, you can add it to the diagram of other VIs.

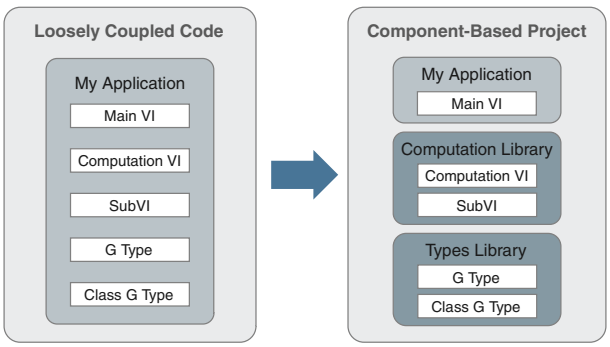
What to do next

After you create a WebVI, run the WebVI in the development environment or ***build the WebVI into a web page*** to execute the code.

Componentizing Applications

Componentizing an application helps you organize code into logical groups, save time from creating duplicate code, and enhance software stability. Use Application and Library documents to create component-based applications, in which you compose loosely coupled code into modular and cohesive projects.

The following figure demonstrates how to convert a project with loosely coupled code into a component-based project.



When you build the application, each top-level VI is built into a separate HTML page with required support files. The following figure shows an example of the build result.

Build Queue

Status	Name	Type	Build
Complete	My Application.exe	Executable	My Application
Complete	Computation Library.gll	Library	Computation Library
Complete	Types Library.gll	Library	Types Library

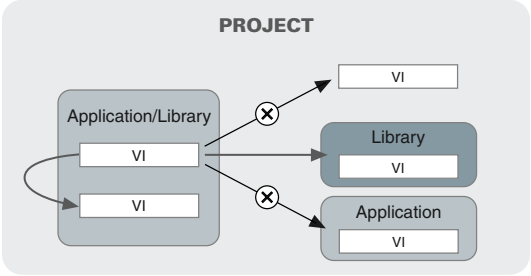
Best Practices for Componentizing Applications

Consider the following guidelines when componentizing an application:

- Keep application components small and self-contained.
- Store code that you might reuse in a separate Library document.
- *Create namespaces* to organize files into logical groups.
- Avoid creating circular dependencies between libraries, which cause problems such as build failures.
- Analyze your application composition frequently for opportunities to refactor code. Refer to Code Dependency Guidelines when refactoring code.

Code Dependency Guidelines

A VI in an Application or Library document can call only VIs within the same component or in a dependency library.



Organizing Code to Avoid Name Collisions

A *namespace* is a container used to organize files into logical groups. Items in the same namespace must have a unique name. Namespaces help to prevent name collisions.

About this task

Procedure

1. Open an Application or Library document.
2. On the document toolbar, click **New**»**Add namespace**.
3. Rename the namespace.

   TIP
   Give your namespace a meaningful name to indicate that the namespace contains files of the same logical group.

4. Drag files from the project to the namespace.
5. Optional: Rename the files as necessary.

   Different namespaces can contain files that have the same name. For example, the `Main VI` namespace and the `SubVI` namespace in the same Application or Library document can each contain a file named `Function.gvi`.

6. Click **File**»**Save all**.

Results

After you build an application, each namespace corresponds to a folder in the output directory.

RELATED INFORMATION

Libraries

A *library* is a collection of source files, such as VIs, type definitions, classes, and other support files. You can use a library to organize source files into a single hierarchy of items.

A library allows you to reuse code across multiple applications without having to implement the same functionality in each application. Your code must be part of a library before you can reuse it across applications.

Use a *Library* document (`.gcomp`) to create a library. The Library document allows you to perform the following actions:

- ***Organize files using namespaces*** to avoid name collisions.
- Customize the organization of files on the palette.
- Enable source files to be called from other applications and libraries.

Creating a Library

To create a library, create a Library document (`.gcomp`) and add source files to the document.

About this task

Procedure

1. Create a new Library document.
   a) Click **File**»**New**»**Library**.
   b) In the **Create Library** dialog box, enter values in **Library name** and **Namespace**.
   c) Click **OK**.
   d) Save the Library document.
2. Add source files to the Library document in one of the following ways:
   - Create files under the project and drag the files from the **Project Files** tab to the Library document.
   - In the Library document, click **New** on the document toolbar and select the type of file to create.
   - In the Library document, click **New**»**Import files** on the document toolbar and select files on disk to copy to the Library document.
3. For each item in the Library document, configure the build options as needed.
   - Enable **Export** if you want to allow a source file to be called from other applications or libraries. Disable **Export** for internal source files.
   - Enable **Always include** if you want to include a support file, such as a text, image, or audio file or source files, such as .gvis or .gtypes, when you share the library.
4. On the **Document** tab, enter information about your library in the **Details** section.
5. Click **File**»**Save all**.

What to do next

Now that you have created a library, consider *optimizing your library* before you *package it*.

RELATED INFORMATION

Creating a Library from Existing Files

Use a Library document (`.gcomp`) to create a library from existing files.

Before you begin

Before you begin, create a project that contains the files you want to include in your library.

Procedure

1. On the **Project Files** tab, right-click the VI(s) you want to include in your library and select **Create Application/Library from item**.
2. In the **Create Application or Library** dialog box, set **Type** to **Library**.
3. Enter values in **Library name** and **Namespace**.
4. Determine whether to enable the **Include dependencies not in an Application or Library** checkbox. If you enable the **Include dependencies not in an Application or Library** checkbox, the Application or Library document you create includes any dependencies one or more of your VIs require that are not already in an Application or Library document.
5. Click **OK**.

Results

The Library document appears in the project and contains the files you selected to include in the library.

RELATED INFORMATION

Creating a Custom Palette

The palette provides an organized hierarchy of all the objects you can add to the diagram or panel. Create a palette file (`.gpal`), or an add-on library, to customize a palette category with the objects in your library. By default, custom categories are added to the `Add-ons` palette category.

Procedure

1. On the **Document** tab of a Library document, click **Create palette file**.
2. In the **Palette Options** dialog box, select the type of palette you want to create.
   - **G diagram palette**—Create a palette containing objects you can drop on the diagram.
   - **G panel palette**—Create a palette containing objects you can drop on the panel.
3. Click **OK**.
4. In the palette file, use the options on the **Item** tab to customize each palette item.

What to do next

Now that you have created your custom palette, consider *creating localized string dictionaries* for and *optimizing your add-on library* before *creating a web application* to demonstrate the use of your add-on library.

Adding Code Snippets to a Custom Palette

Add items to a palette file (`.gpal`) that include chunks of code from your diagram.

Before you begin

Before you begin, complete the following prerequisite tasks:

- Enable `.gpal` option to place diagram contents when dropped.
- Enable *preview features*.
- Relaunch G Web Development Software.

Procedure

1. *Create a VI within your project.*
2. *Create a library.*
3. Add the VI to the library.
4. On the **Document** tab of a Library document, click **Create palette file**.
5. Select **G diagram palette** from the palette type drop-down menu.
6. Open your palette file (`.gpal`). Select your VI.
7. In the **Item** tab, under **Behavior**, select **Place diagram contents**.

   The block of diagram code from the VI will appear on the diagram in projects that include your custom `.gpal`.

   NOTE
   You cannot include a VI in an application if any of the exported VIs in the library are broken. If a VI in your library containing a snippet is broken, disable the VI's **Export** box before building the package.

Building Shareable Libraries

Use your web application library to share reusable code.

About this task

You can create an add-on for your web application and distribute it in a package using the following steps.

Procedure

1. *Create a library*—Add all of the source files for your web application to the library.

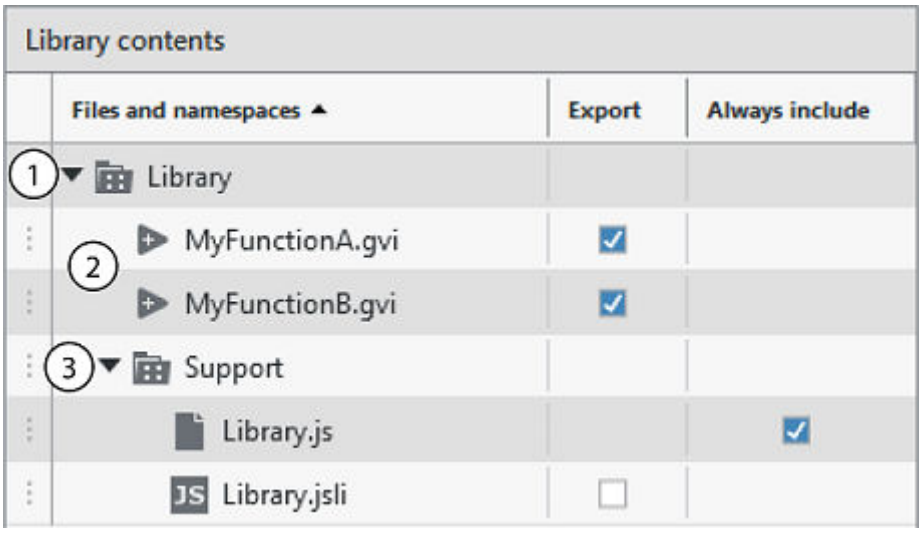
   NOTE
   Ensure you create the library on the web server target.

2. *Add a palette to the library*—Add a HTML panel palette and all supporting functions to the web application library.

3. *Add custom UI elements to the library*—Add a palette with your custom UI elements to the web application library.
4. *Package the library*—Build the web application library into a package or package installer.

Recommendations for Building a Web Application Library

Consider the following recommendations when building a reusable library for your web application.



1. Use VIs (.gvi) instead of WebVIs (.gviweb)—Use a WebVI as the top-level VI in your web application and VIs with the .gvi file extension for all other VIs and libraries. Using VIs with the .gvi file extension enables you to use your libraries across targets.
2. Wrap JavaScript Library Interface (JSLI) nodes in VI (.gvi) files and uncheck **Export** for JSLI documents—Rather than exporting nodes defined in the JSLI document, wrap the nodes in VI (.gvi) files and ensure they are not accessible outside of the library. The wrapper VI isolates the JSLI nodes from the library user, giving you the flexibility to add functionality, such as supporting additional data types or having multiple outputs.
3. Put JavaScript resources and JSLI in a support namespace—Use a separate namespace for JavaScript resources, such as CSS, and JSLI documents to organize web application specific resources.

If you want to add custom visual elements to your web application library, refer to *Adding a Custom UI Palette to Your Web Application Library*.

RELATED INFORMATION

Adding a Custom UI Palette to Your Web Application Library

Add custom UI elements you created with the Placeholder HTML Container to a palette in your web application library.

Before you begin

Before you create a palette for your custom UI elements, complete the following tasks:

- Enable the preview feature—Navigate to **File**»**Preferences**»**Preview Features** and in the Web Module section, select **Enable Placeholder HTML Containers in WebVI library palettes** to enable this feature. Refer to *Preview Features* to learn more about LabVIEW NXG preview features.
- *Create UI elements*—Use the Placeholder HTML Container and `Obtain JavaScript Reference` node to create custom UI elements for your web application.

About this task

Use a control definition file to make the custom UI elements you create available in the palette when someone installs your web application add-on.

Procedure

1. Create or open a library in your web application project.

   TIP
   Make sure this library has an extension of `.gcomp` and has a Web Server target.

2. Create a control definition file and add it to the library.
3. Add your Placeholder HTML Container to the control definition file and configure how you want it to appear when placed on the front panel.

   NOTE
   If you use the Image URL option, you cannot use an image file in the library at deployment. You must use an image URL from the public internet or a self-contained data URL. If you try to reference an image file in the library component, it may not resolve in the correct location once deployed.

4. In the library with the control definition file selected, click **Create palette file** from the configuration pane.
5. Select **G HTML panel palette** for the palette type.
6. (Optional) Use the configuration pane to configure your palette.

Creating User Interfaces

Create a user interface (UI) to allow a user to interact with a program.

About this task

Use the **Panel** to design a UI for your program.

Build UIs with controls and indicators. A *control* is an object that allows a user to input information into a program. An *indicator* is an object that gives the user information about the program.

Procedure

1. Create a program in the **Diagram**.
2. Switch to the **Panel**.
3. From the **Palette**, add controls and indicators to the **Panel**.

   NOTE
   Controls and indicators have different data types, including numeric, Boolean, and string. Right-click the control or indicator to replace it with another control or indicator of the same data type.

4. Switch to the **Diagram**.
5. In the **Palette**, click **Unplaced Items** to add terminals that correspond to your controls and indicators onto the diagram.
6. Wire the new terminals into your program.
7. Click **Run**.

What to do next

You can use the **Configuration pane** to change the appearance of your control or indicator.

RELATED INFORMATION

Centering a Cursor on a Graph or Chart

If you cannot locate a cursor that you added to your graph or chart, you can reset the cursor position to the center of your graph or chart.

Use the cursor legend to center a cursor.

NOTE
Cursors are not available for intensity graphs.

Procedure

1. If you do not see the cursor legend, select **Cursor Legend** in the **Parts** section on the **Item** tab for the graph or chart.
2. On the **Cursor Legend**, navigate to the cursor you want to center.
3. Ensure the **Visible?** button is selected.

4. Click the **Center Cursor** button to center the cursor.

RELATED INFORMATION

Clearing Indicator Display Data in a Chart, Graph, or Array

Procedure

Right-click the chart, graph, or array and select **Clear Data** to remove all data from the indicator displays.

RELATED INFORMATION

Controlling Panel Objects Programmatically

Control a panel object, such as a control, indicator, or the panel itself, by creating a control reference to the object. Control references are either strictly typed or general. Use control references with the `Panel Manipulation` nodes to control panel objects programmatically.

NOTE

The `Chart` class does not support the **Value** property whether strictly typed or general.

RELATED INFORMATION

Strictly Typed Control References

Strictly typed control references include data type information and accept only references of the same data type. For example, if a subVI accepts a strictly typed control reference that contains a 32-bit integer data type, you can wire only a reference that also contains a 32-bit integer data type to the subVI.

About this task

Complete the following steps to create a strictly typed control reference:

Procedure

1. Place a control or indicator on the panel.
2. Right-click a panel object and select **Create reference**.
3. On the diagram, place the `Static Control Reference` of the panel object from the `Unplaced Items` tray and right-click the reference to create a control, indicator, or constant.

General Control References

General control references are flexible in the type of data they accept. For example, if a subVI accepts a general slide reference, you can wire a reference to a 32-bit integer slide, single-precision slide, or a cluster of 32-bit integer slides to the slide reference terminal of the subVI.

About this task

The **Behavior** section on the **Item** tab displays the following statement if a control reference is general:

```
Reference does not include value data type
```

NOTE
The **Value** property is not available on general control references.

Complete the following steps to create a general control reference:

Procedure

1. Place a `Control Reference` on the panel.
2. On the diagram, place the `Control Reference` from the `Unplaced Items` tray and click the **Select reference type** button next to the **Reference type** option on the **Item** tab to select a type.

Results

Writing Multiple Plots to a Graph or Chart

Before you begin

Before you write multiple plots to a graph or chart, you must generate all the data sets you want to plot. Make sure each set of data has the same data type.

Procedure

1. Wire each set of data you want to plot to a `Build Array` node.
2. Wire the **appended array** output from `Build Array` to a graph or chart indicator.

   If your data consists of numeric, complex, or cluster values, **appended array** is a 2D array. Each row of the array is a separate plot. If your data consists of waveforms, **appended array** is a 1D array. Each waveform is a separate plot.

RELATED INFORMATION

Designing a Responsive User Interface

You can design the user interface of a WebVI to respond to different screen sizes.

About this task

Suppose you want your WebVI to run on both a computer monitor and a tablet device; or you want the WebVI to run on a mobile phone that users expect to scroll only vertically. Instead of creating a user interface for each use case, you can design a single user interface that adapts the panel and its contents to the screen size.

Procedure

1. Create a WebVI within your application document.
2. On the **Panel** tab, set **Panel layout** to **Flexible**.
3. Create containers by placing objects on the panel.
   - On an empty panel, place a control or indicator anywhere on the panel to create a container.
   - Place a control or indicator outside an existing container to create a new container. You can create multiple containers in a column or in a row.

   A container appears as a border when you hover over objects on the panel.
4. Select a container and configure the layout options in the **Layout** section of the **Item** tab to determine how objects in the container arrange and align when the screen size changes.
5. Select an object and configure the layout options in the **Layout** section of the **Item** tab to set the object size and whether the object resizes when the screen size changes.
6. Click **File**»**Save all** to save all files.
7. Select **Run**»**Run in browser** to view your application in a web browser.

   Resize the web browser and ensure the controls and indicators resize appropriately.

What to do next

To make your web application available to other users, host your build output on a web server that is accessible to other users.

RELATED INFORMATION

Aligning and Arranging Objects in a Responsive User Interface on page 32

A *container* groups together controls and indicators and determines their arrangement on the panel. If the screen size on which you view your application changes, the objects in a container rearrange according to the layout settings of the container. Containers exist only on panels whose layout type is flexible.

Building a Web Application as a Web Page on page 78

Hosting a Web Application on a Server on page 98

To make your web application available to other users, host your build output on a web server that is accessible to other users.

Aligning and Arranging Objects in a Responsive User Interface on page 32

A *container* groups together controls and indicators and determines their arrangement on the panel. If the screen size on which you view your application changes, the objects in a container rearrange according to the layout settings of the container. Containers exist only on panels whose layout type is flexible.

Building a Web Application as a Web Page on page 78

Hosting a Web Application on a Server on page 98

To make your web application available to other users, host your build output on a web server that is accessible to other users.

Aligning and Arranging Objects in a Responsive User Interface

A *container* groups together controls and indicators and determines their arrangement on the panel. If the screen size on which you view your application changes, the objects in a container rearrange according to the layout settings of the container. Containers exist only on panels whose layout type is flexible.

Before you begin

Before you begin, create a WebVI whose layout type is **Flexible**.

Procedure

Select a container and configure the layout options in the **Layout** section of the **Item** tab to determine how objects in the container arrange and align when the screen size changes.

1. Set the **Direction** to one of the following options:

Option	Description
Row	Arranges objects in a single row as the screen width allows. As the screen width decreases, objects rearrange into multiple rows.

Column	Arranges objects in a single column.

2. Set the **Horizontal alignment** to one of the following options:

Option	Description
Left	Aligns objects to the left of the container.
Center	Aligns objects to the center of the container.
Right	Aligns objects to the right of the container.
Space between	Arranges objects with space between the left and right of objects.
Space around	Arranges objects with space before, between, and after objects.

3. Set the **Vertical alignment** to one of the following options:

Option	Description
Top	Aligns objects to the top of the container.
Center	Aligns objects to the center of the container.
Bottom	Aligns objects to the bottom of the container.
Space between	Arranges objects with space between the top and bottom of objects.
Space around	Arranges objects with space above, between, and, below objects.

Select an object and configure the layout options in the **Layout** section of the **Item** tab to set the object size and whether the object resizes when the screen size changes.

4. Set the **Width** and **Height** of the object.

5. Set **Flexible resize** to one of the following options:

Option	Description
Do not resize	Does not resize the object.
Resize height and width	Resizes the height and width of the object while maintaining aspect ratio.
Resize width	Resizes only the width of the object.

NOTE
The flexible resize options are not available for all objects, as some objects have fixed width and/or height. For example, text objects do not support flexible resize options. You must enable the **Auto size** property for text objects to resize to the container or disable the **Auto size** property to prevent text objects from resizing.

6. Optional: Set **Minimum height** and **Minimum width** of the objects. This prevents the objects from becoming too small to read or use when the screen size decreases.

7. Click **File**»**Save all** to save all files.

8. Select **Run**»**Run in browser** to view your application in a web browser.

   Resize the web browser and ensure the controls and indicators resize appropriately.

RELATED INFORMATION

Designing a Responsive User Interface on page 31

You can design the user interface of a WebVI to respond to different screen sizes.

Designing a Responsive User Interface on page 31

You can design the user interface of a WebVI to respond to different screen sizes.

Customizing the Appearance of Controls in a WebVI

Use *Cascading Style Sheets* (CSS) to customize the appearance of controls in a WebVI. You can modify properties such as the font, color, shape, or layout of a control.

Before you begin

Before you begin, complete the following:

- Familiarize yourself with CSS on the *Mozilla Developer Network*
- Open or create a web application project

About this task

NOTE

If you modify the CSS to customize the appearance of controls, these modifications may not persist from release to release. If you modify the CSS from NI defaults, plan to test your code each time you upgrade to a new version of the G Web Development Software and make manual updates to your CSS styles if necessary.

Procedure

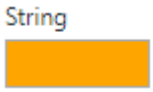
1. Open the panel of a WebVI and click **Edit HTML Source** (</>).

2. In the `<head>↙head>` tags, add the following lines of code after the `<style ni-autogenerated-style-id="">↙style>` tags.

```
<style>
    selector {
        property 1: value 1;
        property 2: value 2;
        ...
        property n: value n;
    }
</style>
```

CSS Item	Description
selector	The element you want to customize. You can select an element by its tag name, ID, class, or attribute. Example: `ni-string-control`
property: *value*	The property of the element that you want to customize and the value you want to change it to. Example: `--ni-control-background-color: orange;`

For example, you can use the following code to change the background color of a string control from the default white to orange.




```
<style>
    ni-string-control {
        --ni-control-background-color: orange;
    }
</style>
```

CSS Item	Description
`ni-string-control`	This selector targets every string control in a WebVI.
`--ni-control-background-color: orange;`	This sets the CSS Custom Property named `--ni-control-background-color` to the color orange. The `--ni-control-background-color` CSS Custom Property targets the background color of string controls.

3. Click **OK** and verify your changes.

Example

Search within the programming environment to access the following installed example:

Customize with CSS

RELATED INFORMATION

Best Practices for Customizing the Appearance of Controls in a WebVI on page 36

Best Practices for Customizing the Appearance of Controls in a WebVI on page 36

Best Practices for Customizing the Appearance of Controls in a WebVI

Use a combination of configuration options in G Web Development Software, NI-defined custom properties, and browser-defined Cascading Style Sheet (CSS) properties to customize the controls in your WebVI.

By following these best practices, you can create, edit, and maintain the appearance of your controls more easily.

NOTE
If you modify the CSS to customize the appearance of controls, these modifications may not persist from release to release. If you modify the CSS from NI defaults, plan to test your code each time you upgrade to a new version of the G Web Development Software and make manual updates to your CSS styles if necessary.

When to Use CSS Properties

Always use configuration options within G Web Development Software instead of NI-defined custom properties or browser-defined CSS properties when configuration options are available. Refer to the following examples of control configurations with G Web Development Software:

- If the control supports the Configuration Pane UI to accomplish the same effect.
- If the control supports Property Node configuration to accomplish the same effect.

When Web Module defines custom CSS properties, such as `--ni-fill-background`, you should use these over other browser-defined CSS properties whenever possible. You can use browser-defined CSS properties, such as `font` or `margin`, to format your controls if G Web Development Software configuration options are not available.

Styling Guidelines

Use the following guidelines to style your WebVI controls.

Goal	How to implement
Style the entire page.	Select `body`
Style the panel canvas.	Select `ni-front-panel`

Goal	How to implement

Style a specific control in my WebVI.	Set an HTML class attribute in the IDE and use that class selector in the CSS:
	`.your_class_selector {`
	`--ni-true-background: yellow;`
	`--ni-true-foreground-color: #067bc2;`
	`}`
	NOTE
	To use an HTML class attribute as a CSS class selector, you must prefix the class attribute with a period in the CSS. In the example above, the HTML class attribute referenced is `your_class_selector`. See W3schools *CSS Selectors* for more information.

Style a group of controls in my WebVI.	Set multiple HTML class attributes in the IDE and apply the styles to those classes in the CSS:
	`.activation-progress {`
	`font-weight: bold;`
	`}`
	`.activation-progress.warning {`
	`--ni-fill-background: yellow;`
	`}`
	`.activation-progress.danger {`
	`--ni-fill-background: red;`
	`}`
	NOTE
	When specifying multiple HTML class attributes in the IDE, the list of classes must be space-delimited. In the example above, the HTML class attribute for the control is `activation-progress`. To make the control bold and yellow, you would use the HTML class attribute of `activation-progress warning`. Notice the class is not prefixed with a period and the class names are separated with a space.

For recommendations on how to style specific controls, such as setting border styles on a button or configuring font styles in text boxes, see the *How do I customize a specific control on my WebVI Panel?* section of ***CSS Frequently Asked Questions***.

RELATED INFORMATION

Customizing the Appearance of Controls in a WebVI on page 34

Use *Cascading Style Sheets* (CSS) to customize the appearance of controls in a WebVI. You can modify properties such as the font, color, shape, or layout of a control.

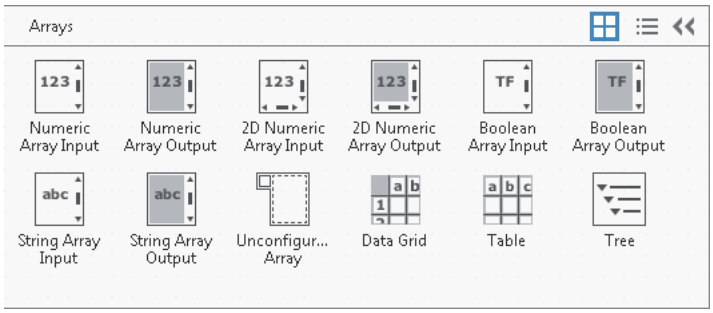
Customizing the Appearance of Controls in a WebVI on page 34

Use *Cascading Style Sheets* (CSS) to customize the appearance of controls in a WebVI. You can modify properties such as the font, color, shape, or layout of a control.

Control Reference

Array Controls

Group data elements of the same type.



Examples

An example of a simple array is a 1D text array that lists the eight planets of our solar system in order.

Array elements are ordered. An array uses an index so you can readily access any particular element. The index is zero-based, which means it is in the range 0 to n - 1, where n is the number of elements in the array. For example, n = 8 for the eight planets, so the index ranges from 0 to 7. Earth is the third planet, so it has an index of 2.

What Is an Array?

An array consists of elements and dimensions. Elements are the data that make up the array. A dimension is the length, height, or depth of an array. An array can have one to 32 dimensions and as many as (2^{31}) - 1 elements per dimension, memory permitting. You can build arrays of numeric, Boolean, path, string, waveform, and cluster data types. Input

arrays are classified as controls by default and allow users to input data into the control. Output arrays are classified as indicators by default and display data on the panel.

When Should I Use an Array?

Consider using arrays when you work with a collection of similar data and when you perform repetitive computations. Arrays are ideal for storing data you collect from waveforms or data generated in loops, where each iteration of a loop produces one element of the array.

How Do I Find an Array Element?

To locate a particular element in an array, you must use indexes. Each dimension in the array has an index. Indexes let you navigate through an array and retrieve elements, rows, columns, and pages from an array on the diagram.

NOTE
Array indexes are zero-based. The index of the first element in the array, regardless of its dimension, is zero.

How Do I Create a Multidimensional Array?

Specify **Dimensions** on the **Item** tab to define the dimension or size of the array.

How Do I Manipulate Array Data?

On the diagram, use the array nodes to manipulate data. Some of the tasks you can complete with the array nodes include:

- Extracting individual data elements from an array.
- Inserting, deleting, or replacing data elements in an array.
- Splitting arrays.

You can use the `Build Array` node or a loop to build an array programmatically.

Data Grid

Group elements of various data types as rows on a table.

The data grid control is a 1D array of clusters. Each row of the table is one cluster, and each element in a cluster represents one column.

You can use the data grid control to configure hardware or to display data in a table.

Data Grid

	A	B	C
0			
1			
2			
3			
4			
5			

Tree

Group string data as a hierarchical list of items in multiple columns.

The tree control is a 1D array of clusters. Each row of the tree is one cluster. The first element in the row represents the hierarchical location of the item in the tree, and subsequent elements represent data to display in each column of the tree.

Use `tree` properties to write the column header or read and write selected items of the tree control.

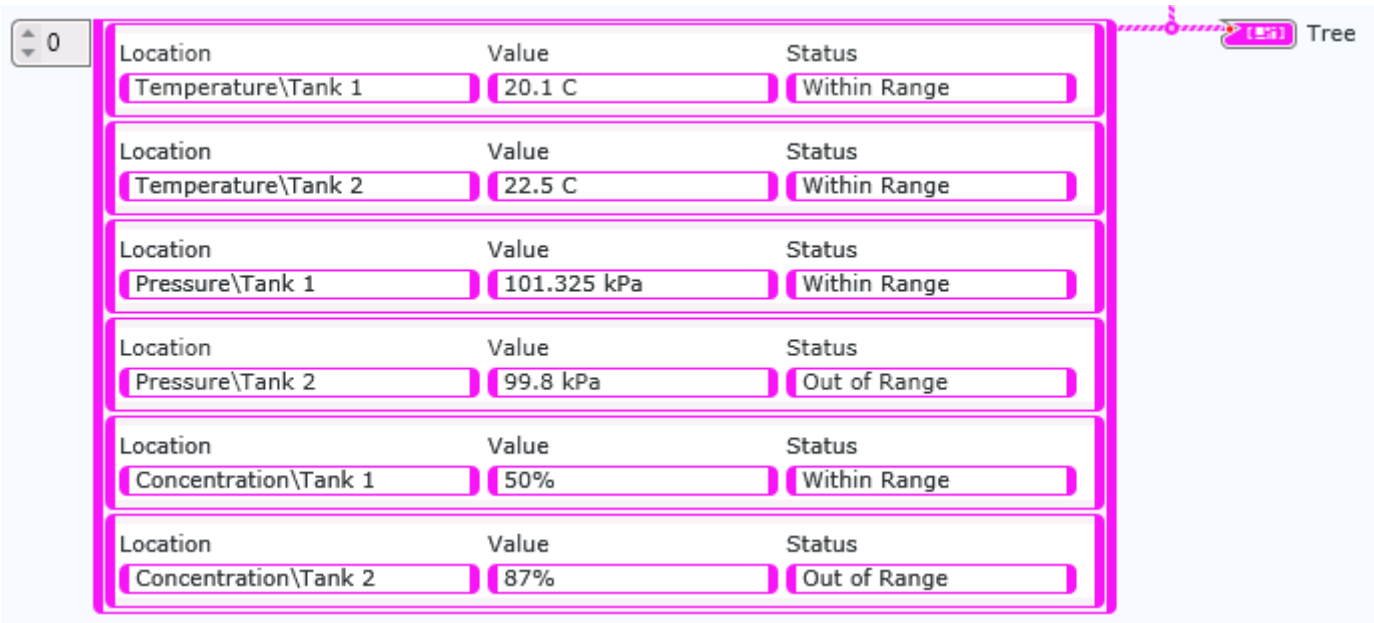
Location	Value	Status
▾ Temperature		
Tank 1	20.1 C	Within Range
Tank 2	22.5 C	Within Range
▾ Pressure		
Tank 1	101.325 kPa	Within Range
Tank 2	99.8 kPa	Out of Range
▾ Concentration		
Tank 1	50%	Within Range
Tank 2	87%	Out of Range

How Do I Select Items in a Tree?

You can allow your users to select one or multiple items in the tree by placing a checkmark in the **Allow selection** checkbox on the **Item** tab. Use the **Selection** property to return information about the selected item(s).

How Do I Define the Hierarchy of a Tree?

You populate items in a tree using an array of clusters, as shown in the following figure.

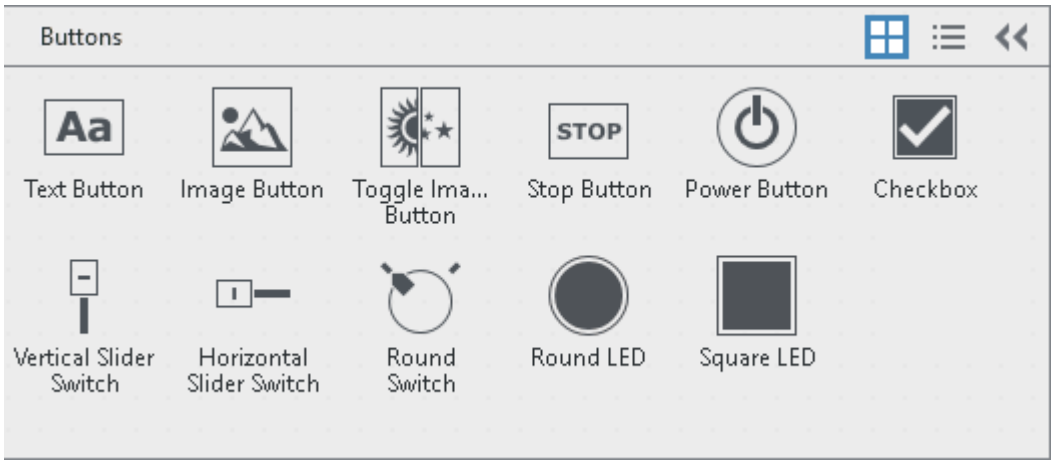


The first element of each cluster is the hierarchical location of the item. The hierarchical location is a backslash-delimited string, where each segment of the delimited string denotes a level of hierarchy in the tree. Use the following general syntax to define a hierarchical location:

Syntax	Definition
<empty string> or \	An empty string or backslash represents the root of the tree.
`\parent\child` or `parent\child`	A leading backslash is equivalent to a location in the tree without a leading backslash.
`parent\child\` or `parent\child`	A trailing backslash is equivalent to a location in the tree without a trailing backslash.
Escape sequence or double backslash	Escape sequences for character representations, such as \s to represent a space character, are not permitted.

Button Controls

Enter or display true and false data.



Which Button Should I Use?

Button Type	Use	Default Mechanical Action
Buttons	Enter true or false input.	Latch when released
Checkbox	Select an item or items from a list.	Switch when released only
Switches	Enter true or false input in an application.	Latch when released
LEDs	Indicate true or false data. You must select **Show values** on the **Item** tab for content to appear on the LED.	Switch when released

Which Mechanical Action Should I Use?

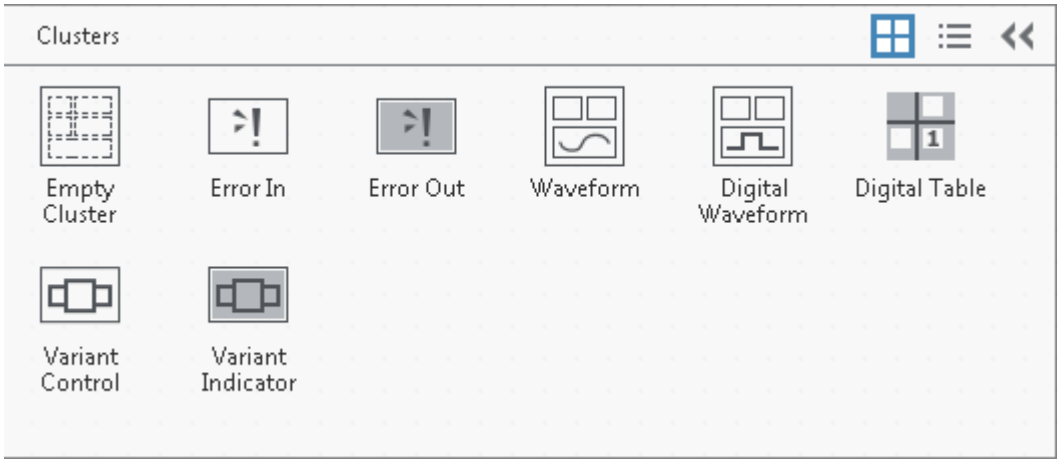
The mechanical action of the control determines how the control behaves. You classify mechanical actions as either switch actions or latch actions:

I want my control to behave like a...	Mechanical Action	What This Mechanical Action Does	Caveats
Power button	Switch when pressed	Changes as soon as you click down on the control without waiting for you to release the mouse.	
On/off switch	Switch when released	Changes the control value only after you release the mouse button within the graphical boundary of the control.	The mechanical action for radio buttons and checkboxes is not configurable; it is always switch when released.

I want my control to behave like a...	Mechanical Action	What This Mechanical Action Does	Caveats
Door buzzer	Switch until released	Changes the control value when you press it and retains the new value until you release the mouse button.	The frequency with which the program reads the control does not affect this behavior. You cannot select this behavior for a radio button control or a checkbox control.
Circuit breaker	Latch when pressed	Changes the control value when you press it and retains the new value until the program reads it once.	This behavior is useful for getting the program to perform an action only once each time you set the control. You cannot select this behavior for a radio button control or a checkbox control.
Dialog box button	Latch when released	Changes the control value only after you press and then release the mouse button within the graphical boundary of the control and retains the new value until the program reads it once.	This behavior is useful for stopping a `While Loop`. When the program reads the control once, the control reverts to its default value. You cannot select this behavior for a radio button control or a checkbox control.

Cluster Controls

Group data elements of a mixed type that logically belong together.



What Is a Cluster?

Clusters group data elements of mixed types. A cluster is similar to a record or a struct in text-based programming languages.

Which Cluster Should I Use?

You can choose from the following cluster types, and clusters can be nested:

Use	Type
Groups a checkbox, numeric control, and string control together to input error information.	Error In
Groups a checkbox, numeric indicator, and string control together to display error information.	Error Out
Groups a timestamp, numeric array, and numeric control together to display waveform data.	Waveform
Create a custom group by dragging and dropping controls, such as buttons, graphs, arrays, and so on, into the cluster shell.	Empty Cluster

How Do I Reorder Cluster Elements?

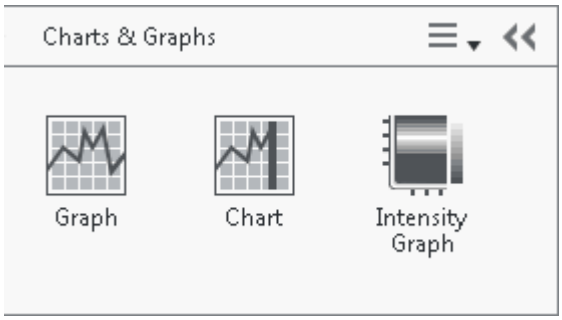
You can reorder cluster elements by using the **Data items** section on the **Item** tab or by moving controls inside or outside the cluster. The order of the controls within the cluster is determined by the order in which you add them to the cluster.

How Do I Tab Through Cluster Elements?

Press Ctrl-Down Arrow or Ctrl-Up Arrow to tab through cluster elements, even nested clusters.

Charts and Graphs

Chart and plot data.



What Is the Difference between Graphs and Charts?

You can collect data, usually in an array, and then plot the data to a graph. This process is similar to a spreadsheet that first stores the data and then generates a plot of it.

A chart appends new data points to those points already in the display to create a history. On a chart, you can see the current reading or measurement in context with data previously acquired.

Deciding Which Graph or Chart to Use

Knowing what type of data you need to display determines the type of graph or chart you should use.

Control	Accepted Data Types	Display Examples	When to Use
Graph	Numeric, complex, X/Y, or point sample data contained in arrays and analog waveforms	Analog waveform data acquired at a constant rate	Use a graph for fast processes that acquire data continuously.
Chart	Numeric data contained in arrays and analog waveforms	Analog waveform data acquired at a constant rate with a history, or buffer, of the data from previous updates	Use a chart for processes that require a cumulative history of incremental updates to be maintained.
Intensity Graph	3D data on a 2D plot by using color to display the values of the third dimension	Patterned data, such as frequency of sound intensity	Use an intensity graph for 3D data.

Autoscaling

You can enable autoscaling for graphs and charts, which means they adjust their horizontal or vertical scales to fit the data you wire to them. Graphs and charts support the following types of autoscaling:

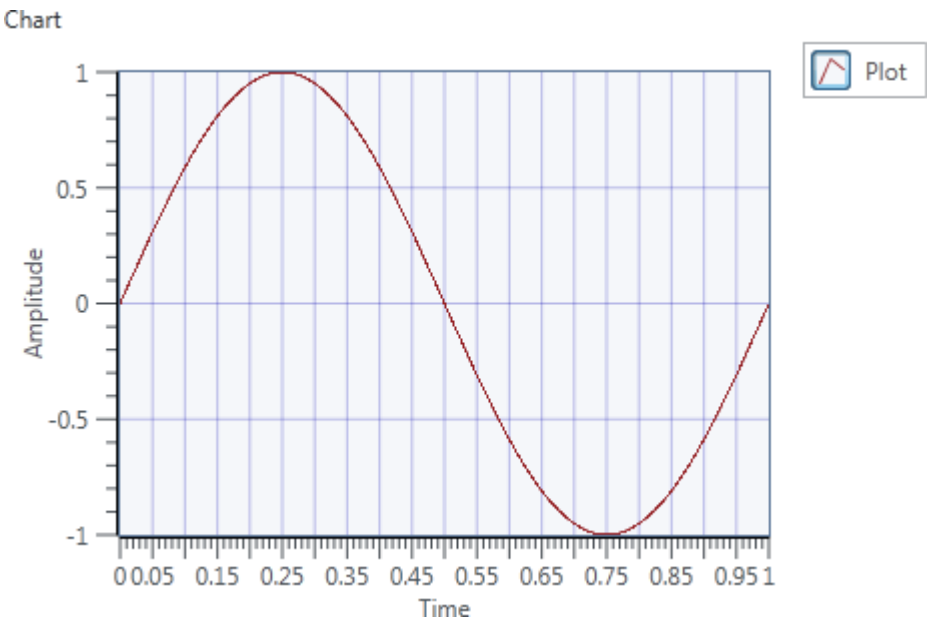
- **Fit exactly**—Fits the end markers to the exact minimum and maximum values of the data.
- **Fit loosely**—Rounds the end markers to a multiple of the increment used for the scale.
- **Grow only**—Fits the end markers to the minimum and maximum values of the data at run time. This type of autoscaling is available only for the vertical axis of graph, chart, and intensity graph.

RELATED INFORMATION

Clearing Indicator Display Data in a Chart, Graph or Array

Chart Control

Chart numeric data by displaying a history of values.



How is Charting Different Than Plotting?

Graphs and charts differ in the way they display and update data. VIs with a graph usually collect the data in an array and then plot the data to the graph. This process is similar to a spreadsheet that first stores the data then generates a plot of it. When the data is plotted, the graph discards the previously plotted data and displays only the new data.

In contrast, a chart appends new data points to those points already in the display to create a history or a buffer. On a chart, you can see the current reading or measurement in context with data previously acquired. When more data points are added than can be displayed on the chart, the chart scrolls so that new points are added to the right side of the chart while old points disappear to the left.

What Types of Data Can I Chart?

You can chart numeric data contained in arrays and analog waveforms. By default, the chart data type is an array of doubles. You can change the data type on the diagram.

The waveform data type carries the start time (**t0**), delta t (**dt**), and y-values (**Y**) of a waveform. When you wire waveform data to a chart, the chart automatically plots a waveform based on the start time, delta t, and y-values of the waveform. A waveform that specifies **t0** and a single-element **Y** array is useful for plotting data that is not evenly sampled because each data point has its own time stamp.

For data contained in arrays, the chart displays a general-purpose Cartesian graph that charts any set of points, evenly sampled or not.

The following table describes how different types of array data are charted.

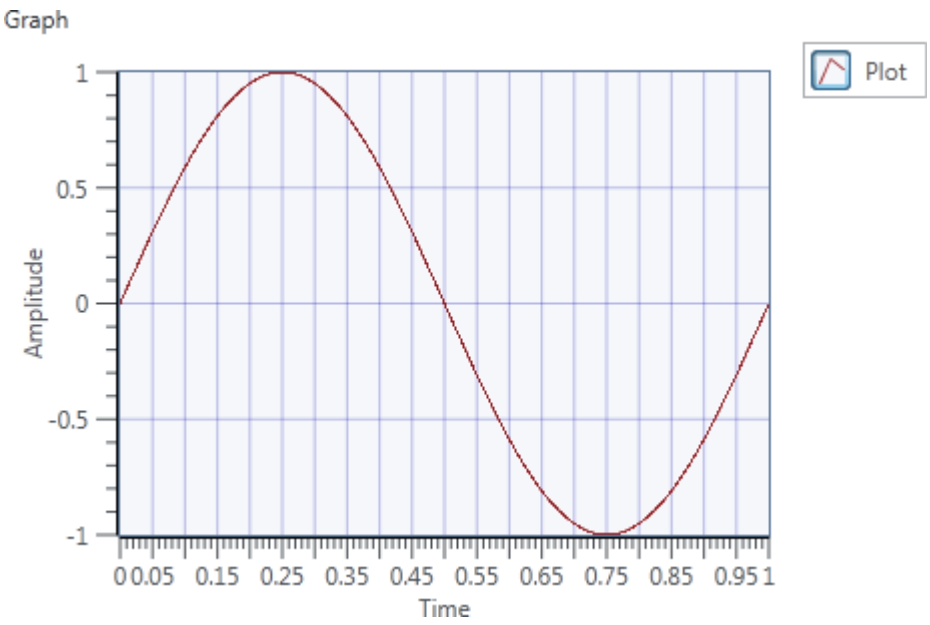
Data Type	Plot Behavior
Scalar numeric value	The chart displays a single plot using the scaler numeric value as the most recent value and previous values up to the history length.
1D array of numeric values	The array is a single plot. X-value: Array index Y-value: Array element
Cluster of numeric values	Each element of the cluster is a single plot which uses the numeric value as the most recent value and previous values up to the history length.
1D array of clusters of numeric values	Each element of the cluster is a single plot which uses the numeric value as the most recent value and previous values up to the history length. The array represents an aggregated set of samples.
1D array of waveforms	Each waveform is a separate plot. This data type is not supported.
2D array of numeric values	Each column of the array is a separate plot.

How Do I Define the Chart History Length?

On the **Item** tab, in the **Behavior** section, specify **History length** to configure the size of the buffer. For waveforms, the history length is the number of individual waveforms the buffer can hold. The waveforms can be of various sample sizes. For numeric data, the history length is the number of samples the buffer can hold.

Graph Control

Plot numeric, complex, X/Y, or point sample data contained in arrays and analog waveforms.



What Types of Data Can I Plot?

You can plot numeric, complex, or point sample data contained in arrays and analog waveforms. By default, the graph data type is an array of doubles. You can change the data type on the diagram.

The waveform data type carries the start time (**t0**), delta t (**dt**), and y-values (**Y**) of a waveform. When you wire waveform data to a graph, the graph automatically plots a waveform based on the start time, delta t, and y-values of the waveform.

For data contained in arrays, the graph displays a general-purpose Cartesian graph that plots any set of points, evenly sampled or not.

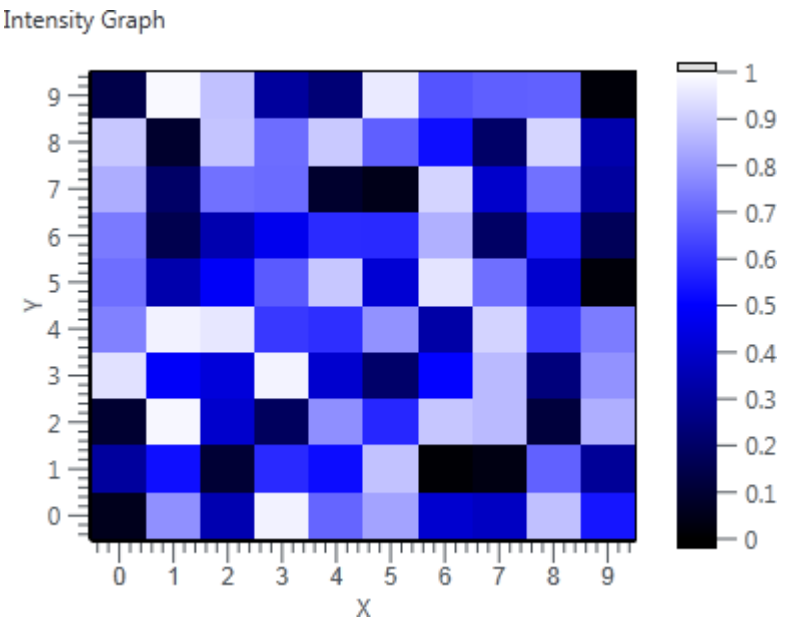
The following table describes how graphs plot different types of array data.

Data Type	Plot Behavior
1D array of numeric values	The array is a single plot. X-value: Array index Y-value: Array element
1D array of complex or cluster values	The array is a single plot. The cluster must be a two-element cluster. X-value: First element of the complex number or cluster Y-value: Second element of the complex number or cluster

Data Type	Plot Behavior
Cluster of two 1D arrays of numeric values	The cluster is a single plot. X-value: Elements of the first array Y-value: Elements of the second array
1D array of waveforms	Each waveform is a separate plot.
2D array of numeric values	Each row of the array is a separate plot.
2D array of complex or cluster values	Each row of the array is a separate plot. The cluster must be a two-element cluster. X-value: First element of the complex number or cluster Y-value: Second element of the complex number or cluster
1D array of clusters of 1D arrays	Each cluster of arrays is a separate plot. Arrays within a cluster must be the same size, but arrays in different clusters can be different sizes.
Cluster of two numeric values and a 1D array	The cluster is a single plot. X-value: An initial x value incremented by a delta x. The first numeric value represents the initial x value, and the second numeric value represents the delta x value Y-value: Elements of the array This data type is not supported.

Intensity Graph Control

Plot 3D data on a 2D plot by using color to display the values of the third dimension. For example, you can use an intensity graph to display patterned data, such as terrain, where the magnitude represents altitude.



What Types of Data Can I Plot?

Intensity graphs accept 2D arrays of numeric values. Each number in the array represents a specific color. The array indexes correspond to the center of the color block. The row index of the array is the x-value, and the column index of the array is the y-value. The color block has a unit area, which is the area between the two points, as defined by the array indexes.

Rows of the array appear as columns on the graph and vice-versa. If you want rows to appear as rows on the graph, use `Transpose 2D Array` before wiring the array to the graph.

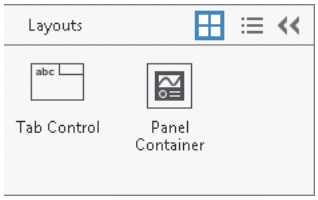
Configuring the Color Scale

When you set the color mapping for an intensity graph, you configure the color scale of the graph. The color scale consists of at least two markers, each with a numeric value and a corresponding display color. The colors displayed on an intensity graph correspond to the numeric values associated with the specified colors. Color mapping is useful for visually indicating data ranges, such as when plot data exceeds a threshold value.

Click **Color scale** on the **Item** tab to set the color mapping. Lower and upper out-of-range colors are specified as **High color** and **Low color** and are displayed on the graph.

Layouts

Organize the appearance of controls and indicators or display the panel of other VIs.



When Should I Use a Tab Control?

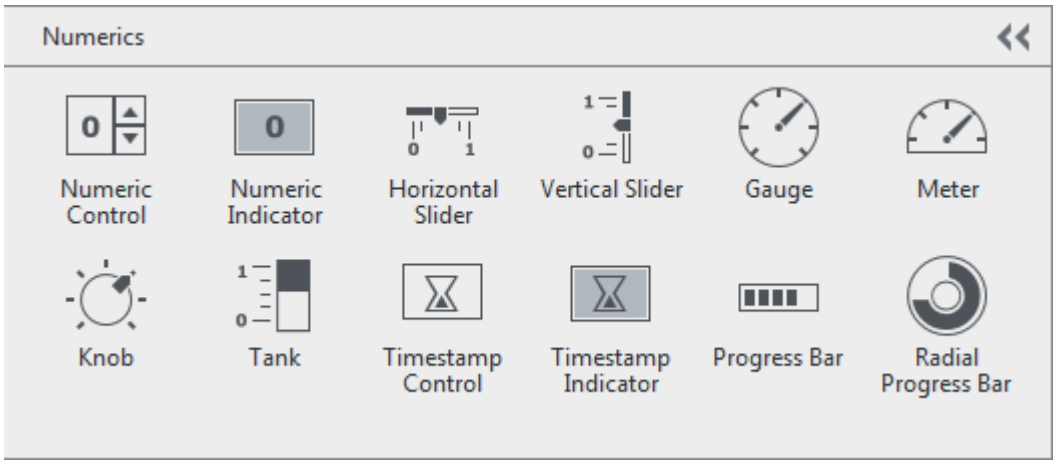
Tab controls are useful when you have several panel objects that are used together or during a specific phase of operation. For example, you might have a VI that requires the user to first configure several settings before a test can start, then allows the user to modify aspects of the test as it progresses, and finally allows the user to display and store only pertinent data.

How Do I Use a Panel Container?

Panel containers display the panel of a running VI in the panel of the current VI. To access the panel container on the diagram, you must create a panel container reference. To create a reference, right-click the panel container and select **Create reference**. Place the panel container reference on the diagram and use the reference with the `Panel Container` nodes.

Numeric Controls

Enter and display numeric data.



Which Numeric Should I Use?

Control	Use
Numeric control and indicator	Enter or display numeric data.
Slider	Display numeric data in a vertical or horizontal slide with a customizable scale and a pointer that helps you see the exact value.
Gauge, meter, and knob	Enter or display numeric data in a rotary scale.
Tank	Display numeric data in a vertical slide that resembles a real tank or thermometer instrument.

Control	Use
Timestamp	Enter or display a time and date value.
Progress bars	Show progress in a vertical bar or circle.

What Is the Difference between Significant Digits and Digits of Precision?

Significant digits specifies the number of significant digits to display in the numeric control.

Digits of precision specifies how many digits to display after the decimal mark.

How Do I Use, View, and Store Absolute Time?

Use the timestamp control to use, view, and store absolute time with high precision. This data type can accurately store 18 digits of precision in whole seconds and 19 digits of precision in fractions of a second. The timestamp control displays values in local time. However, the timestamp wire data type stores values in UTC.

Although you can use a numeric control to display timestamp values, the numeric control holds a relative quantity. The timestamp control holds an absolute quantity.

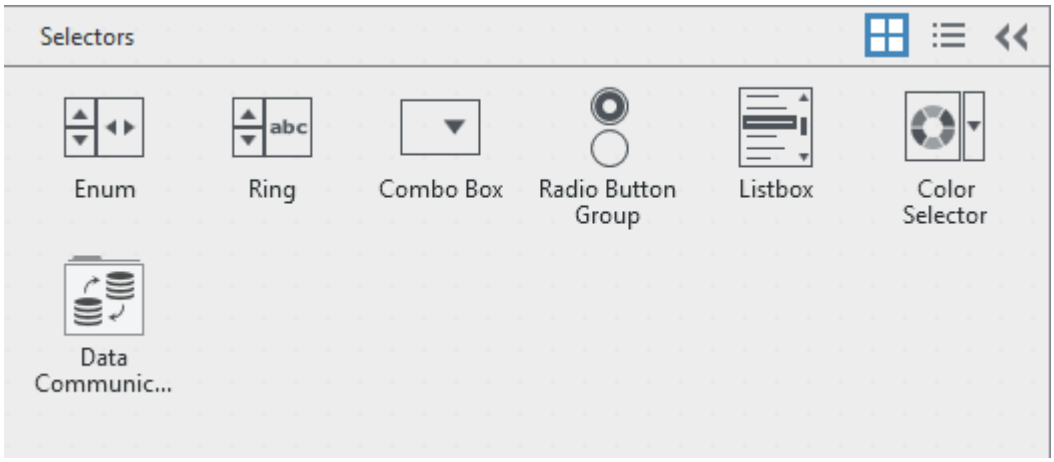
What Characters Do Numeric Controls Accept?

Character(s)	Description
Hexadecimal digits	0 through F
Octal digits	0 through 7
Binary digits	0 and 1
Decimal digits	1, 1.0, 2, 3.5, and so on
.	Decimal point
,	Decimal comma
+	Positive symbol
-	Negative symbol
E or e	For scientific or engineering notation format
Infinity	Infinity
NaN	Not a number
/	For use in absolute time format

Character(s)	Description
:	For use in absolute time format
AM, am, PM, pm	For use in absolute time format
SI prefixes	
y	yocto (10^{-24})
z	zepto (10^{-21})
a	atto (10^{-18})
f	femto (10^{-15})
p	pico (10^{-12})
n	nano (10^{-9})
u	micro (10^{-6})
m	milli (10^{-3})
c	centi (10^{-2})
d	deci (10^{-1})
da	deka (10^{1})
h	hecto (10^{2})
k	kilo (10^{3})
M	mega (10^{6})
G	giga (10^{9})
T	tera (10^{12})
P	peta (10^{15})
E	exa (10^{18})
Z	zetta (10^{21})
Y	yotta (10^{24})

Selector Controls

Create a list that a user can cycle through to make selections.

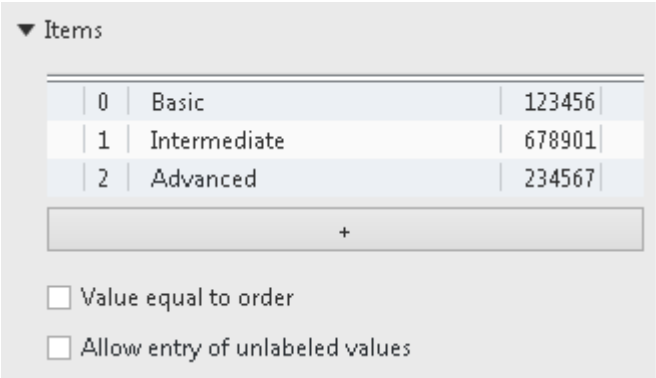


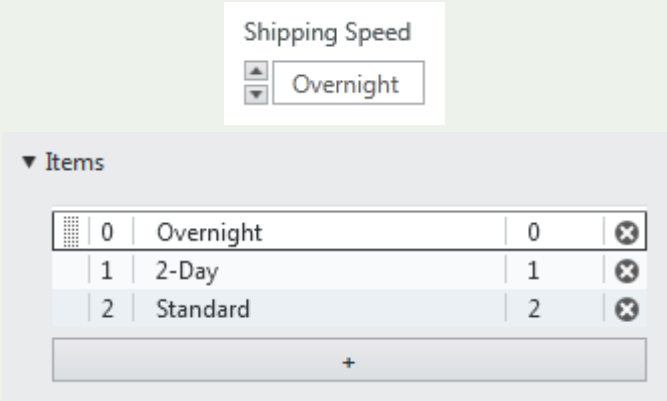
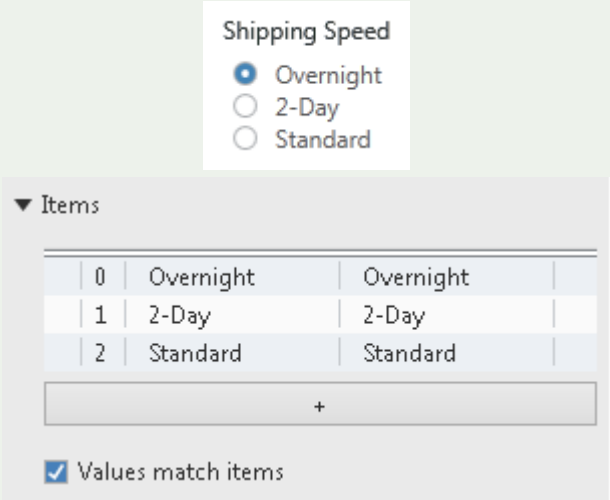
Types of Selector Controls

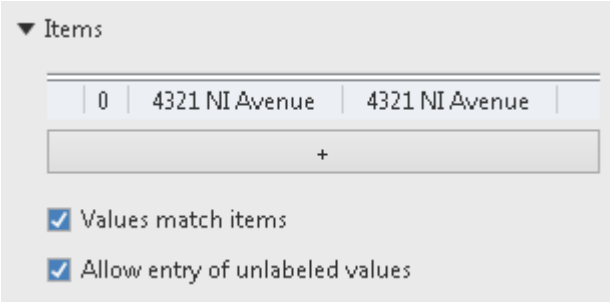
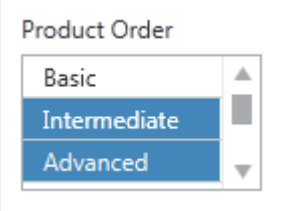
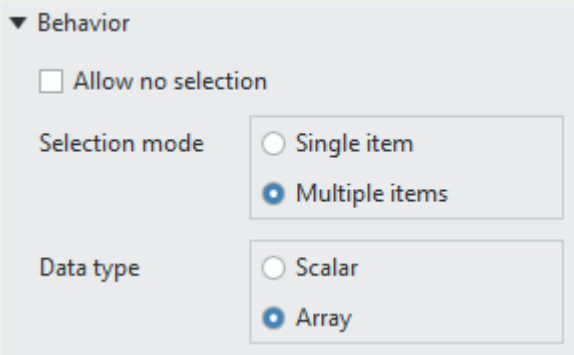
Control	Description
Ring	A ring control is a numeric input that presents a set of choices associated with numeric values.
Enumerated type, or Enum	The enum control is an object that presents a set of choices associated with an enumerated data type. The enumerated data type consists of a set of named constants.
Radio Button Group	The radio button group control is an object that presents a set of mutually exclusive choices associated with an enumerated data type. The enumerated data type consists of a set of named constants.
Combo Box	The combo box control is an object that presents a list of names associated with string values. Users may select from items in a predefined list or enter new values directly, at edit time or run time.
Listbox	The listbox control is an object that presents a set of items. Users may select single or multiple items at run time.
Color Selector	The color selector control is an object that presents a color selection. Each color corresponds to RGBA color values or a hexadecimal value.

Which Selector Control Should I Use?

Selector controls all contain a list of items from which the user can make a selection.

Control	When to Use	Examples: E-Commerce Application
Ring	Use a ring when you want to provide a list of choices that represent numeric values. The ring control is a numeric input that allows the user to choose from a list of meaningful, human-readable values instead of numeric values. If the logic of your program prefers numeric values to string values, use a ring.	You want to present users with a list of product names, each of which corresponds to a unique product number. You can specify the products' numbers as their respective numeric values. You can then cross-reference those numeric values to find the prices of the items. 

Control	When to Use	Examples: E-Commerce Application
Enum and Radio Button Group	Use an enum or a radio button group when you want to provide a list of choices associated with an enumerated data type. If you want your program to deal with a fixed set of constants instead of arbitrary numeric data, use an enum or radio button group. In general, use a radio button group when you have less than five options for the user to select from. A radio button group can help the user quickly scan the options.	You want to present users with a list of shipping speed options for the item they want to order. The numeric values assigned to each of the options are automatically defined. You cannot edit the numeric values. You can use the enumerated data type in the enum control to drive the logic of your program. For example, because the names of the choices are part of the data type, you can use them to populate the cases of a Case Structure. This ensures that only valid states are used in your program. **Figure 1 :** Enum  **Figure 2 :** Radio Button Group 

Control	When to Use	Examples: E-Commerce Application
Combo Box	Use a combo box when you want to provide a list of items for users to cycle through while also providing the option to type new values into the menu at edit time or run time. If the logic of your program requires string values rather than numeric values, use a combo box.	You want to present users with a list of previous shipping addresses, while also allowing them to enter a new shipping address for this purchase. 
Listbox	Use a listbox when you want to provide a list of items in which users can select items at run time.	You want to present users with a list of products in which they can select one or more items. You can also allow users to select zero items by checking the **Allow no selection** checkbox. For example, you can allow users to deselect all items.  

Assigning Specific Values to Selector Controls

Unlike the strings listed in rings, enums, and radio button groups, the strings listed in a combo box and listbox have no numeric representation. Additionally, enums and radio button groups require these numeric values to be sequential, unsigned integer values, while rings allow for much more freedom in assigning a numeric value.

1. Select the control to which you want to assign specific values.
2. On the **Item** tab, in the **Items** group, assign and arrange specific values as desired.

Enabling a Combo Box to Accept New Values from the User at Edit Time or Run Time

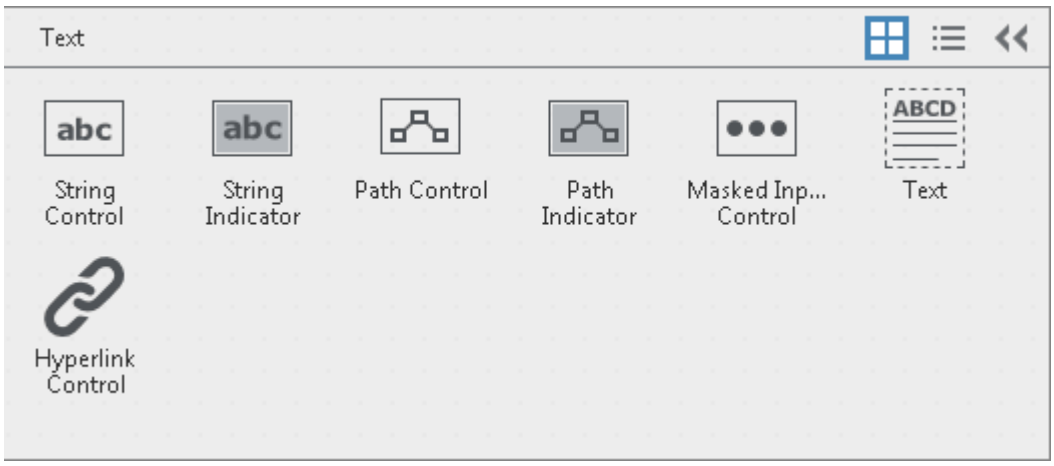
1. Select the combo box control.
2. On the **Item** tab, in the **Items** group, select **Allow entry of unlabeled values**.

RELATED INFORMATION

Enum Data

Text Controls

Create text entry boxes and labels or enter or return the location of a file or directory.



Which Text Control Should I Use?

Use	Control
Enter string data.	String Control
Display string data.	String Indicator
Enter the location of a file or directory.	Path Control

Use	Control
Display the location of a file or directory.	Path Indicator
Create free floating text that you can use as a label or annotation.	Text
Enter or display a URL hyperlink.	Hyperlink Control
Enter string data in masked display.	Masked Input Control **NOTE** This control only masks the text visually. When you read the string data from the diagram, you read the actual data the user entered. Any mechanism that can examine the memory of the program can also read the actual data.

When Does My Entered Text Pass to the Diagram?

By default, new or changed text does not pass to the diagram until you terminate the edit session by clicking elsewhere on the panel or by pressing Ctrl-Enter.

You can change the behavior of the Enter key for a **String** control by clicking the control on the panel and selecting a new **Enter key behavior** in the **Item** tab.

Option	Behavior
Commits text	Pressing Enter terminates the edit session. You can also terminate the edit session by clicking elsewhere on the panel. Carriage returns are not allowed.
Enters new line (default)	Pressing Enter inserts a carriage return. You can terminate the edit session by clicking elsewhere on the panel or pressing Ctrl-Enter.

Should I Use an Absolute or Relative Path?

If possible, use a relative path. A relative path describes the location of a file or directory relative to an arbitrary location in the file system. By using a relative path, you avoid having to rework the paths if you build an application or run the code on a different computer.

Escape Sequences

The following table lists allowable escape sequences for character representations. Escaping characters using hex codes is not supported.

Escape Sequence	Character Represented
\b	Backspace

Escape Sequence	Character Represented
\f	Form feed
\n	Newline or line feed
\r	Carriage return
\s	Space
\t	Tab
\\	Backslash

Use escape sequences to substitute for reserved characters in string terminals and string constants. On the **Item** tab, toggle the **Show escape sequences** checkbox to display escape sequences or character representations.

RELATED INFORMATION

Buttons Controls

Enter or display true and false data.

Gauge

Enter or display numeric data in a rotary scale.

Horizontal Slider

Enter or display numeric data in a horizontal slider.

Knob

Enter or display numeric data in a rotary scale.

Meter

Enter or display numeric data in a rotary scale.

Numeric Control

Enter numeric data.

Numeric Indicator

Display numeric data.

Radial Progress Bar

Display progress in a circle.

Progress Bar

Display progress in a horizontal bar.

Timestamp Indicator

Display a time and date value.

Timestamp Control

Enter a time and date value.

Tank

Display numeric data in a vertical slide that resembles a real tank or thermometer instrument.

Vertical Slider

Enter or display numeric data in a vertical slide.

Masked Input Control

Enter string data in masked display.

Path Indicator

Display the location of a file or directory.

Path Control

Enter the location of a file or directory.

String Indicator

Display string data.

String Control

Enter string data.

Text

Create free-floating text that you can use as a label or annotation.

Text

Create free-floating text that you can use as a label or annotation.

Hyperlink Control

Enter or display a URL hyperlink.

Table

Group data elements of the string type or numeric type in a 2D array.

Reference Terminal

Transfers data between the diagram and other VIs.

Use this terminal to add tag reference inputs or outputs to a subVI.

Ring

Create a list that a user can cycle through to make selections. The internal output is an integer or a floating-point number.

Combo Box

Create a drop-down listbox that a user can cycle through to make selections. Users can also type values into the list at edit time or run time.

Radio Button Group

Create a list of mutually exclusive options.

Listbox

Create a list of items in which a user can select single or multiple items.

Color Selector

Display a color selector whose color corresponds to RGBA color values or a hexadecimal value.

Enum

Create a list that a user can cycle through to make selections. Enum values are pre-defined and do not change at run time.

Variant Indicator

Represent the variant data type.

Variant Control

Enter variant data.

Digital Table

Represents data in up to six digital data states.

Value	Digital Data State	Description
0	0 (Drive Low)	Forces logic low. Drives to the low voltage level (V_{OL}).
1	1 (Drive High)	Forces logic high. Drives to the high voltage level (V_{OH}).
2	Z (Force Off)	Forces logic high impedance. Turns the driver off.
3	L (Compare Low)	Compares logic low (edge). Compares a voltage level lower than the low voltage threshold.
4	H (Compare High)	Compares logic high (edge). Compares a voltage level higher than the high voltage threshold.
5	X (Compare Unknown)	Compares logic unknown. Does not compare.

Digital Waveform

Specify start time, delta t, input data, and attributes of a digital waveform.

Waveform

Group a timestamp, numeric array, and numeric control together to display waveform data.

Error Out

Group a checkbox, numeric indicator, and textbox to display error information.

Error In

Group a checkbox, numeric indicator, and textbox to input error information.

Empty Cluster

Create an empty group you can customize by dragging and dropping controls, such as buttons, graphs, arrays, and so on, on the cluster shell.

Drawing Controls

Decorate the user interface with lines or shapes.

Image

Decorate the user interface with bitmap or vector graphics.

Picture Display

Create a list of pictures that you can choose from to display on the user interface.

Pencil

Draw a freeform line.

Pen

Draw a line one point at a time.

Line

Draw a straight line.

Line with Arrows

Draw a straight line with an arrow on one end.

Arc

Draw a curved line.

Rectangle

Create a customizable rectangle.

Rounded Rectangle

Create a customizable rectangle with rounded corners.

Ellipse

Create a customizable ellipse.

Circle

Create a customizable circle.

Arrow

Create a customizable arrow.

Triangle

Create a customizable triangle.

5 Point Star

Create a customizable star.

Objects Palette

Add images related to objects to the icon.

Administrator

Add an administrator symbol to the icon.

Advanced

Add an advanced symbol to the icon.

Certificate

Add a certificate symbol to the icon.

Check Mark

Add a check mark to the icon.

Clock

Add a clock symbol to the icon.

Coefficient

Add a coefficient symbol to the icon.

Filter

Add a filter symbol to the icon.

FPGA (large)

Add a large FPGA symbol to the icon.

FPGA (small)

Add a small FPGA symbol to the icon.

Handshake

Add a handshake symbol to the icon.

Network

Add a network symbol to the icon.

User

Add a user symbol to the icon.

Actions Palette

Add images related to actions to the icon.

Abort

Add an abort symbol to the icon.

Add

Add an addition symbol to the icon.

Ask

Add an ask symbol to the icon.

Calculate

Add a calculate symbol to the icon.

Calibrate

Add a calibrate symbol to the icon.

Close

Add a close symbol to the icon.

Configure

Add a configure symbol to the icon.

Create

Add a create symbol to the icon.

Delete

Add a delete symbol to the icon.

Flatten

Add a flatten symbol to the icon.

Generate

Add a generate symbol to the icon.

Lock

Add a lock symbol to the icon.

Measure

Add a measure symbol to the icon.

Pause

Add a pause symbol to the icon.

Read

Add a read symbol to the icon.

Request

Add a request symbol to the icon.

Reset

Add a reset symbol to the icon.

Save

Add a save symbol to the icon.

Start

Add a start symbol to the icon.

Stop

Add a stop symbol to the icon.

Unflatten

Add an unflatten symbol to the icon.

Unlock

Add an unlock symbol to the icon.

Wait

Add a wait symbol to the icon.

Write

Add a write symbol to the icon.

Data Types Palette

Add images related to data types to the icon.

String

Add a string data type symbol to the icon.

Cluster

Add a cluster data type symbol to the icon.

Array

Add an array data type symbol to the icon.

TimeStamp

Add a time stamp data type symbol to the icon.

Analog Waveform

Add an analog waveform data type symbol to the icon.

Digital Waveform

Add a digital waveform data type symbol to the icon.

Path

A line or shape. Select a line or shape on the panel and use the tool that appears to edit the drawing. Click on anchor points to add points, remove points, or convert between line segments and Bezier. This tool is not available for lines and shapes in the icon view.

Variant

Add a variant data type symbol to the icon.

Error

Add an error data type symbol to the icon.

Text Palette

Add text to the icon.

Lines Palette

Add lines to the icon.

Shapes Palette

Add shapes to the icon.

Callout

Create a customizable callout.

Hexagon

Create a customizable hexagon.

Moon

Create a customizable moon.

Pentagon

Create a customizable pentagon.

Plus

Create a customizable plus.

Control Reference and VI Reference

Represent a reference whose behavior and properties you can control and pass to other VIs.

Control Reference

Represent a reference to a control, indicator, or panel item whose behavior and properties you can control and pass to other VIs.

G Object Control

A control or indicator of the common parent of all classes in the G Dataflow inheritance hierarchy.

VI Reference

Represent a reference to a VI whose behavior and properties you can control and pass to other VIs.

Content Font Family

Select the font face.

Content Font Size

Set the font size.

Font Size

Set the font size.

Font Family

Select the font face.

Panel Container

Display the panel of a running VI in the panel of the current VI.

Tab Control

Organize controls and indicators by overlapping them in a smaller area.

Shape

Set array size and dimension.

Shape ribbon item (scalar vs array)

Configure the resource as an array or scalar.

Shape

Configure the FIFO as an array or scalar. This option changes the configuration of the FIFO container on the diagram but does not change the configuration of any FIFO resource in SystemDesigner. If you change this option, you must specify a FIFO resource that meets the new configuration to avoid a type mismatch error.

Mechanical Action

Configure button behavior.

Data items

Edit the name and order of the items on the control.

Arrange

Changes the dimensions of the cluster control.

Content

Toggle the content within the control on or off.

Control button

Change the selected indicator to a control.

Enter key behavior

Choose the behavior of the Enter key. If you select **Commits text**, carriage returns are not allowed and pressing Enter terminates the edit session. If you select **Enters new line**, pressing Enter inserts a carriage return and pressing Ctrl-Enter terminates the session. You can always terminate the edit session by clicking elsewhere on the panel.

Indicator

Change the selected control to an indicator.

Label Placement

Set the label location for selected items on the panel.

Label

Toggle the control label on or off.

Lock Label

Keep the label with the control.

On Connector Pane button duplicated

Add the terminal to or remove the terminal from the connector pane.

Size button

Choose a preset display size for selected items on the panel.

Axes Ribbon Item

Configure the axis or add a new axis.

Autoscale

Adjust the scale automatically to reflect the data.

Clockwise

Increase angles in a clockwise direction.

Color Scale

Configure the colors that represent the third dimension of an intensity graph.

Cursor Legend

Display a marker at a defined point coordinate. You can display multiple cursors on a graph.

Cursors

Add or configure a cursor.

Graph Tools

Move the cursor and zoom and pan the graph or chart while a VI runs.

History

Configure the size of the chart history buffer.

Inverted

Reverse the positions of the minimum and maximum values on the scale.

Parts Manager

Toggle the plot legend, cursor legend, graph tools, and label on or off.

Anti-aliased

Make line plots appear smoother. Anti-aliased drawing may not be visible over remote desktop.

Plot Legend

Define the color and style of plots.

Plots

Configure an existing plot or add a new plot.

Scale Legend

Define labels for scales and configure scale properties.

Stretch

Fit one column to width, distribute column widths, or size column widths individually.

Use Waveform Start Time

Use the waveform start time as the beginning of the x-scale.

Allow no selection

Allow users to select zero items at run time.

Allow no selection

Allow users to select zero items at run time.

Allow selection

Allow users to select a single row or multiple rows at run time.

Name

Set a unique name to identify the control on the diagram.

Label text linked to control name

Toggle whether the label and name of the control are set independent of each other.

Edit Items

Add, delete, and configure the items within a selector control list.

Coerce to Interval.

Configure a control to snap to the defined interval.

No Coercion

Allow the control interval to move freely.

Default button

Set the value when the VI loads or when you reinitialize the data of the control to default.

Display Digits

Set the precision of a numeric control or indicator.

Display Type

Set the numeric format of the control or indicator.

Fractional Digits

Set the number of digits to the right of the decimal point; sets the precision of the control display.

Grow Time

Configure the rate at which the bubbles grow.

Indeterminate Progress

Displays unknown progress

Integer Digits

Set the number of whole number digits to display.

Interval

Set the amount by which the increment and decrement buttons increase or decrease the value of the numeric control.

Inverted

Reverse the positions of the minimum and maximum values on the scale.

Max

Set the maximum value for the data range.

Min

Set the minimum value for the data range.

Pitch

Increase to nose up, decrease to nose down.

Rise Time

Configure the speed of the bubbles as they rise.

Roll

Increase to roll right; decrease to roll left.

Scales

Select the tick style of a numeric control or indicator.

Sweep

Set the angle of the scale.

Filename

Creates standard text to represent a filename.

Path Tool For Drawing Controls

A line or shape.

When you select this line or shape on the panel, a tool appears that allows you to edit the drawing. Use the tool to add points, remove points, or convert between line segments and Bezier by clicking on an anchor point. This tool is not available for lines and shapes in the icon view.

Placeholder Tunnel

A placeholder tunnel that replaces a tunnel type not currently supported in this version of the software.

Tag Reference

Create references to tag resources.

You can only use this control as part of a subVI.

NOTE
When you run a subVI that uses this control, the control does not return a valid reference unless a valid reference is passed from a calling VI.

Path

Combines two or more shape controls.

Dialog Type

Set whether you can select a file or a directory with the path control.

File Type

Set the file type(s) you can select with the path control.

Tab Strip Placement

Specify where the tabs appear on the selected tab control.

Edit Tabs

Edit the name and order of the tabs on the selected tab control.

Show escape sequences

Display escape sequences or character representations.

Show escape sequences

Display escape sequences or character representations.

Class Object Control

A control or indicator of a user-defined class type.

Disable mapping changes

Prevent users from changing the scale mapping mode at run-time.

Control reference type

Reference is generic and does not include value data type.

Debugging a WebVI

Unlike desktop VIs, WebVIs do not currently support debugging tools, such as execution highlighting, probes, and breakpoints. However, you can still debug a WebVI using the following strategies.

Procedure

Choose between the following debugging options.

Use Case	Instructions
You want to see the error output of diagram objects.	WebVIs support an alternative to automatic error management that logs unhandled errors to the **Output** tab or browser development console. Because this does not provide the full capabilities of automatic error management, make sure you follow the guidelines for programmatic error management. **NOTE** To view your error output, create an error indicator and place it on the panel.
Use this debugging strategy in either of the following cases. • You need to debug a subVI in your web application. • You want to log error information to the console of your web browser.	To write log error information to the console of your web browser or G Web Development Software output window, go to the diagram palette and select **Interoperability**»**Operating System**»**Write to System Log**. To view the information output by **Write to System Log** within the editor, on the **Navigation pane**, open the **Output** tab by clicking **Tool Launcher**»**All**»**Output**. To view the information output by **Write to System Log** in a web browser, use the console of the web browser development tools.

You need to debug your top-level WebVI.	Wire extra indicators at any point in your WebVI code to imitate probes and display the data that flows through a wire. **NOTE** WebVIs do not currently support G types as controls or indicators on the panel. If you create an indicator from a G type on the diagram, no corresponding indicator is available on the panel.

RELATED INFORMATION

Building a Web Application as a Web Page on page 78

Error Management

Building a Web Application as a Web Page

To view your completed web application in a web browser, build your application to generate HTML, CSS, JavaScript, and other files that are included in your application.

Before you begin

Before you begin, open or create a web application project and *enable SystemDesigner visibility*.

Procedure

1. On the **Project Files** tab, double-click the application document to open it.
2. In the application document, enable the **Top-level VI** checkbox for each WebVI you want to output as a separate web page.

   Each top-level WebVI in an application document generates one HTML file, which corresponds to one web page.
3. On the **Document** tab, click **Build**.

   Monitor the status of your build in the **Build Queue** tab.
4. In SystemDesigner, right-click your application document and select **Run** to view your application in a web browser.

What to do next

To make your web application available to other users, host your build output on a web server that is accessible to other users.

RELATED INFORMATION

Creating a Web Application on page 18

Hosting a Web Application on a Server on page 98

To make your web application available to other users, host your build output on a web server that is accessible to other users.

Linking WebVIs in a Web Application

Before you begin

Before you begin, open or create a web application project with multiple top-level WebVIs. Each WebVI that you mark as top-level outputs as a separate web page.

Procedure

1. On the **Project Files** tab, double-click the web application document to open it.
2. In the web application document, select the top-level WebVI you want to link to. On the **Item** tab, copy the **Relative URL**.
3. Open the top-level WebVI you want to link from.
4. Switch to the panel and add a Hyperlink Control.
5. Select the Hyperlink Control. On the **Item** tab, in the **URL** field, paste the relative URL of the WebVI that you want to link to.

   You may need to modify the relative URL depending on how your files are organized. Consider the following file structure.

```
Application
|
|---One.gviweb
|
|---Two.gviweb
|
|---Namespace_A
|    |
|    |---Three.gviweb
|
|---Namespace_B
     |
     |---Four.gviweb
     |
     |---Namespace_C
          |
          |---Five.gviweb
```

Link from	Link to	Relative URL
One.gviweb	Two.gviweb	Two.html
One.gviweb	Three.gviweb	Namespace_A/Three.html
Three.gviweb	One.gviweb	../One.html
Three.gviweb	Four.gviweb	../Namespace_B/Four.html
Five.gviweb	One.gviweb	../../One.html
Five.gviweb	Three.gviweb	../../Namespace_A/Three.html

6. Build your application and open the HTML output in a web browser to test the link you created.

Example

Search within the programming environment to access the following installed example:

Multiple Top-Level WebVIs

RELATED INFORMATION

Creating Localized String Dictionaries

To localize strings for objects that display at run time, such as dialog boxes, add custom entries to a string dictionary file and use the `Get Localized String` node to retrieve the strings programmatically.

Before you begin

Before you can localize objects that display at run time, generate a string dictionary file using **Export strings**, create a copy of the string dictionary file, and save it in a new folder that indicates the language of the string dictionary file. You must save the new folder within the project folder in the same location as the original dictionary folder and name the new folder using standard Microsoft locale names, such as en-US or zh-CN. **Export strings** overwrites any existing dictionary in the folder for the default language of the editor. If you want to preserve an existing dictionary, save a copy of the existing dictionary file in a separate location. You can also create your own string dictionary file which contains entries for each string you want to localize, but this file must follow the same syntax and file extension as the dictionaries that **Export strings** generates.

About this task

NOTE
You do not need to use custom dictionary entries to localize most objects in an application, such as object labels and descriptions. To localize these types of objects, refer to *Exporting Strings for Localization*.

Complete the following steps to localize strings that display at run time.

Procedure

1. Open the string dictionary file you want to edit with a text editor.
2. Add a key to the string dictionary file. You can specify any name for the key, but any new entries you add to the dictionary file must use the same syntax as the entries automatically generated by **Export strings**.
3. Add the localized text you want the object to display within the `<loc>` element. If you want to localize the text "Good Morning!" to German, for example, make your new entry look similar to the following example:

   ```
   <entry key="New_Key_Name"> <loc>[de-DE] Guten Morgen!↙
   loc> ↙entry>
   ```

   Repeat this process for each string you want to localize.
4. Save the dictionary file.
5. Add the `Get Localized String` node to the diagram.
6. Right-click the **key** input and select **Create constant**. Enter the key name you added to the dictionary file. For the above example, enter `New_Key_Name`.
7. Right-click the **fallback** input and select **Create constant**. Enter text in this constant that you want to display if the `Get Localized String` node cannot locate the key you specify in **key**.
8. Right-click the **base path** input and enter the file path to the application you want to localize.
9. Wire the **localized string** output to the object you want to localize. For example, if you want to localize the string in a dialog box, wire this output to the **message** input of a dialog box node.
10. Run the application.

    The application now displays the localized string(s).

    NOTE
    The application determines which dictionary to retrieve keys from based on the language you define in the editor preferences. You must have the editor set to the desired language, or the application does not display the localized strings.

Results

You have now created a localized string dictionary for your code.

What to do next

If you created a library, consider *optimizing its performance* before *packaging your library*.

Exporting Strings for Localization

Localize applications into additional languages by exporting the strings into a string dictionary file, translating the file, and saving your changes.

About this task

The string dictionary file contains information to localize the following strings:

- Application title and description
- Window titles and descriptions
- Object labels and descriptions
- Private data (listbox item names, graph plot names, graph cursor names, and graph annotation names)
- Free labels (panel and diagram)

> **NOTE**
> The string dictionary file does not contain information to localize string data in diagram constants, arrays, or clusters.

To localize most objects in an application, such as object labels and descriptions, complete the following steps. If you need to localize strings that display at run time, such as the strings in dialog boxes and indicators, refer to *Creating Custom String Dictionaries for Localization*.

> **NOTE**
> If you localize an application to a language the editor or run-time engine does not support, some information that displays at run time, such as error messages, remains in the default language.

Procedure

1. Open the application you want to localize.
2. Save the application.
3. On the **Document** tab, click **Export strings** in the Localization section.
4. Navigate to the string dictionary file using the path in the file dialog box and click **OK** to close the dialog box.
5. Create a copy of the string dictionary file and save it in a new folder that indicates the language of the string dictionary file. You must save the new folder within the project folder in the same location as the original dictionary folder and name the new folder using standard Microsoft locale names, such as en-US or zh-CN.

If you do not follow these guidelines, your application may not be able to locate the localized string dictionary file.

6. Translate the new string dictionary file and save the changes.
7. Save and close the project.
8. Reload the project and reopen the application.
9. On the **Document** tab of the panel, select the language from the **Preview** drop-down menu.

   The localized strings now display in the application.

NOTE
If you modify an application after you export and translate the string dictionary, the string dictionary may contain incorrect or missing keys, and the application may no longer display the correct localized content. If you need to export strings again and want to preserve an existing dictionary, make sure to save a copy of the existing dictionary in a separate location. **Export strings** overwrites any existing dictionary in the folder for the default language of the editor.

What to do next

To localize an application in multiple languages, create copies of the dictionary file and repeat this process to create dictionaries for all desired languages.

RELATED INFORMATION

Enabling SystemDesigner Visibility

Update your preferences to see a visual representation of web targets in SystemDesigner.

Procedure

1. Click **File**»**Preferences**.
2. Select **Web Server** and enable **Show SystemDesigner**.
3. Reopen the project to apply the update.

RELATED INFORMATION

Using JavaScript with a Web Application

Call JavaScript functions in your web application and use JavaScript to create UI elements.

- *Calling JavaScript functions in your web application*—Use a JavaScript Library Interface (JSLI) to create entry points to JavaScript functions in your web application.
- *Creating UI Elements with JavaScript*—Create UI elements to place on the panel of your web application using the Placeholder HTML Container and JavaScript Library Interface document (JSLI).
- *JavaScript Resources*—Research JavaScript concepts using NI-recommended resources.

RELATED INFORMATION

Calling JavaScript Functions in a Web Application

Use a JavaScript Library Interface (JSLI) to call globally accessible JavaScript functions in your web application.

About this task

A JSLI is a document in which you create *entry points*, or defined calls, to JavaScript functions.

When you create a new entry point, a visual representation of the entry point appears on the software palette of the diagram. You can place and wire entry points like nodes.

As the web application executes, the entry point calls the JavaScript function. Input data passes from the diagram to the JavaScript function, and output data returns from the JavaScript function to the diagram.

Complete the following tasks to call JavaScript functions in your web application:

Procedure

1. *Prepare your code*(Optional)—Create wrapper code for the JavaScript file you want to use.
2. *Create a JSLI*—Define the JavaScript functions you want to call in your web application.
3. *Integrate the JavaScript file into your web application*—Connect the JavaScript file to your web application and create calls to the JavaScript functions you defined in the JSLI.
4. *Call an asynchronous JavaScript function*(Optional)—Add code to your JavaScript file to use asynchronous JavaScript calls in your WebVI.

Examples

Search within the programming environment to access the following installed example:

- *Call JavaScript From a WebVI*

What to do next

After you create a JSLI and add entry points to your web application, if necessary, ***troubleshoot common issues with a JSLI***.

Preparing Your Code For Use With a JavaScript Library Interface

Create wrapper code, if necessary, to make your JavaScript functions compatible with the JavaScript Library Interface (JSLI).

Before you begin

Before you begin, create a web application and acquire a JavaScript library or find a global, built-in browser function you want to use. For more information on JavaScript concepts, refer to ***JavaScript Resources***.

About this task

Procedure

1. Determine if you need to create wrapper code.

   When calling JavaScript functions in your web application, you may need to create wrapper code to make your JavaScript code compatible with the JSLI. The following table contains common examples of when to create wrapper code and descriptions of the wrapper code to create.

Situation	Solution
Create wrapper functions that convert between unsupported JavaScript types and types supported by the JSLI.	
Complete the following steps: 1. Create a wrapper function that invokes the `new` operator and stores a reference to the new object instance. 2. Create a wrapper function that looks up the reference to that object and invokes a method.	

2. Create wrapper code for your JavaScript library. NI recommends you create one JavaScript file per JSLI document that contains all the wrapper functions you create.

Model the following code as a best practice for creating wrapper code. Customize the following code for your unique programming goals.

```
                /*1*/
(function () {
   'use strict'; /*2*/

    var counter = 1; /*3*/

    /*4*/
    var logWithCount = function (message){
      console.log(counter + ' > ' + message); /*5*/
      return counter++; /*6*/
    };

    window.logWithCount = logWithCount; /*7*/
}());
```

①	Wrap the contents of the entire wrapper JavaScript file in an immediately invoked function expression (IIFE). The IIFE creates a new lexical scope which prevents you from unintentionally adding objects to the global scope.
②	Configure the IIFE to use JavaScript strict mode. Strict mode improves error handling and allows browsers to make certain optimizations.
③	Create a private variable by declaring it within the IIFE. In this example, the private variable is named `counter`. Because this variable is private, it cannot be unintentionally modified by other code. This variable is not accessible to the JSLI because you did not add it to the global scope.
④	Create a new function and add the desired parameters to the function. In this example, we create a function named `logWithCount` that passes a parameter to a built-in browser function named `console.log`.
⑤	Call the function you are wrapping, in this example, the `console.log` function.
⑥	Return a value from your wrapper function. If you configure the return parameter in the JSLI document to be of a type other than void, you must pass a return value of the type you configured in the JSLI document to the return statement.

⑦	Add the function you want to call in your web application to the global scope to allow the JSLI document to access the function. In this example, we use the `logWithCount` function. In a web browser, the name of the global object is `window`, so the code `window.logWithCount = logWithCount;` places `logWithCount` on the global scope.

RELATED INFORMATION

Defining Calls to JavaScript Functions using a JavaScript Library Interface on page 87

To call JavaScript functions in your web application, define calls to the JavaScript functions using a JavaScript Library Interface (JSLI) document.

Defining Calls to JavaScript Functions using a JavaScript Library Interface

To call JavaScript functions in your web application, define calls to the JavaScript functions using a JavaScript Library Interface (JSLI) document.

Before you begin

Before you begin, create or open a web application project and *prepare your JavaScript code for use with the JSLI*. For more information on JavaScript concepts, refer to *JavaScript Resources*.

NOTE
For the JSLI to access a JavaScript function, the function must be accessible from the global scope.

About this task

To create and configure a JSLI document, complete the following steps:

Procedure

1. On the **Project Files** tab, add a JSLI to a web application component or library component in your project.

   A JSLI document appears in your web application.

2. Define the prototype for the JavaScript function you want to call. A defined call to a JavaScript function in a JSLI is called an *entry point*.

   a) In the JavaScript global form field, enter the name of the JavaScript function you want to call. The name you enter must exactly match the function name in the JavaScript code. You must specify all functions relative to the global scope. Use dot notation to refer to nested objects. For example, if you want to call the absolute function abs in the object Math, you must specify `Math.abs` in the JavaScript global form field.

   NOTE
   The JavaScript global form field is case sensitive.

b) Click **Add function** to create an entry point. The default entry point name is created from the JavaScript global.

c) Click **Add parameter**.

d) On the **Item** tab, specify the parameter name, data type, and JavaScript representation.

Consider the following information when you add parameters to your prototype:

- You can choose any name for the parameters you add to a JSLI document, but you must add them in the same order as they appear in the JavaScript function.
- The parameter and return data types you specify in the JSLI document must match the parameter and return data types in the JavaScript function.
- The parameter in the top position in the JSLI document is always the return type.

3. Repeat step 2 to create as many entry points as you need for your project.

RELATED INFORMATION

Waiting for Asynchronous JavaScript Operations to Complete in a Web Application on page 89

Use asynchronous JavaScript code to wait on tasks or requests while the rest of the code in your WebVI continues to execute.

Debugging JavaScript Library Interfaces on page 92

Solve common problems that may occur when configuring and using JavaScript Library Interfaces (JSLIs).

Integrating a JavaScript File Into a Web Application

Connect a JavaScript file to your web application and call functions you defined in a JavaScript Library Interface (JSLI) in your web application.

Before you begin

Before you begin, *create and configure a JLI*.

Procedure

1. Import the JavaScript file(s) you want to use into your web application.

a) Save your project.

b) On the **Project Files** tab, right-click a web application document or library component document (`.gcomp`) and select **Import files**.

c) Navigate to the JavaScript file(s) you want to add and click **Open**. Click **Copy** to close the **Copy existing file(s)** dialog box.

2. Integrate the JavaScript file into your web application.

   a) Open the JSLI document associated with the JavaScript file and enter the relative path to the JavaScript file in the HTML script and link dependencies form field.

      Example:

      ```
      <script src="library.js"></script>
      ```

      NOTE
      The JavaScript file must be in the same component as the JSLI document and must be marked as **Always include** in the component document.

   b) Click **Apply Changes**.

3. Add the entry points you defined in the JSLI to any web application in your project.

   a) Open the WebVI in which you want to call the JavaScript function.

   b) On the diagram palette, click **Project Items**»**Software**»**<YourWebAppName>**. Click the folder with the name of your JSLI document to show each entry point you defined in that JSLI.

   c) Drop the entry points you want to use on the diagram.

   d) Wire the entry points and complete the diagram.

   e) Run the WebVI. Input data passes from the diagram to the JavaScript code, and output data returns from the JavaScript code to the diagram.

Waiting for Asynchronous JavaScript Operations to Complete in a Web Application

Use asynchronous JavaScript code to wait on tasks or requests while the rest of the code in your WebVI continues to execute.

About this task

Synchronous code means that your code executes in order. Asynchronous code means that a portion of your code waits for a task to complete while the rest of your code continues to execute. For more information on JavaScript concepts, refer to *JavaScript Resources*.

What to Use

- External JavaScript file
- JavaScript Library Interface (JSLI) document

What to Do

1. Create the following code in a JavaScript (.js) file to use an asynchronous JavaScript function in your web application to run two loops in parallel. Customize the following code for your unique programming goals.

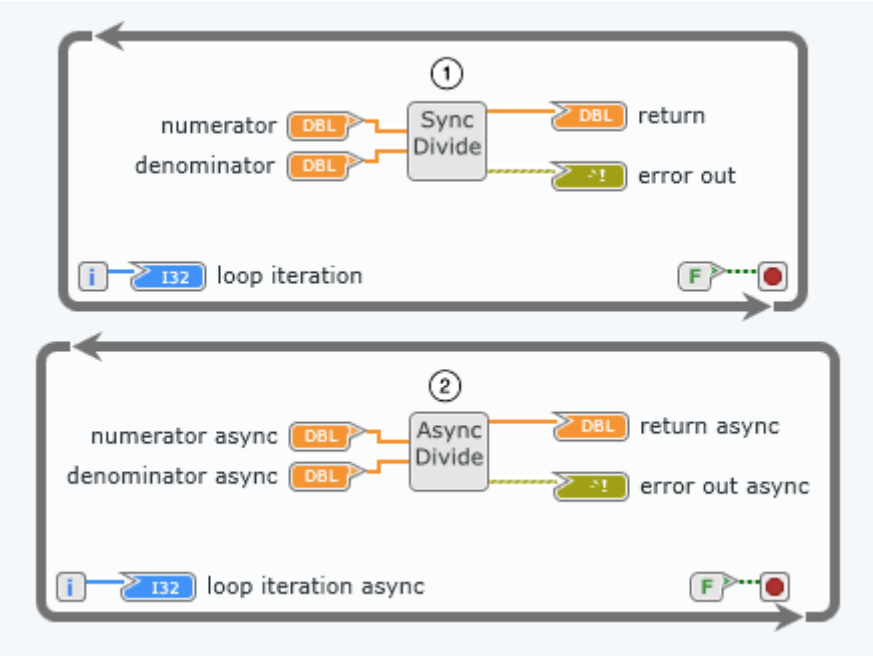
```
(function () {
    'use strict';

    /*1*/
    const synchronousDivide = function (numerator,
denominator) {
        if (denominator === 0) {
            throw new Error('Cannot divide by zero');
        } else {
            return numerator / denominator;
        }
    };

    const sleep = function (time) {
        return new Promise(function (resolve) {
            setTimeout(resolve, time);
        });
    };

    /*2*/
    const asynchronousDivide = async function
(numerator, denominator) {
        await sleep(1000); /*3*/

        /*4*/
        if (denominator === 0) {
            throw new Error('Cannot divide by zero');
        } else {
            return numerator / denominator;
        }
    };

    /*5*/
    window.synchronousDivide = synchronousDivide;
    window.asynchronousDivide = asynchronousDivide;
}());
```

① Create a JavaScript function that completes synchronously named `synchronousDivide`.

> **NOTE**
> To notify G Web Development Software that a JavaScript error has occurred in a synchronous function, you must throw an error in your JavaScript code. In your diagram code, the **error out** output on the JSLI node returns an error.

②	Create an async JavaScript function named `asynchronousDivide`. An async JavaScript function results in a JavaScript Promise that notifies G Web Development Software that the function will run asynchronously.
③	Add code that runs asynchronously. This example uses the sleep function which asynchronously pauses execution for 1000 milliseconds.
④	Return a value or throw an error from the async function. When using an async JavaScript function for asynchronous programming, returning a value or throwing an error uses syntax similar to synchronous programming.
⑤	Add the functions you want to call in your web application to the global scope to make the functions accessible to the JSLI document. In this example, we place both the `synchronousDivide` and `asynchronousDivide` functions on the global scope.

2. *Create and configure a JSLI document*.
3. *Integrate the JavaScript file into your web application*.
4. Create the following diagram to use the asynchronous JavaScript function to execute two loops in parallel in your WebVI.



①	Run the WebVI. The `Sync Divide` JSLI node executes continuously without waiting.

②	The `Async Divide` JSLI node waits 1000 milliseconds before continuing execution. Because the `asynchronousDivide` function executes asynchronously in your JavaScript code, the second loop doesn't block execution of the first loop. The `Async Divide` JSLI node behaves synchronously in your diagram code, so you do not need to wait on the asynchronously executing JavaScript code manually in your diagram code.

RELATED INFORMATION

Defining Calls to JavaScript Functions using a JavaScript Library Interface on page 87

To call JavaScript functions in your web application, define calls to the JavaScript functions using a JavaScript Library Interface (JSLI) document.

Integrating a JavaScript File Into a Web Application on page 88

Connect a JavaScript file to your web application and call functions you defined in a JavaScript Library Interface (JSLI) in your web application.

Debugging JavaScript Library Interfaces on page 92

Solve common problems that may occur when configuring and using JavaScript Library Interfaces (JSLIs).

Debugging JavaScript Library Interfaces

Solve common problems that may occur when configuring and using JavaScript Library Interfaces (JSLIs).

About this task

NOTE

Most errors occur due to mismatches between the JSLI configuration and the functions in the JavaScript library. Mismatch errors may not occur at the exact time the JavaScript function call executes on the diagram.

Procedure

Use the following table to resolve common errors.

Issue	Solution
You receive a **`Function not found`** error.	Verify the JSLI uses the correct spelling and case sensitivity for the entry points you defined.

Issue	Solution
The JSLI node does not appear in the software palette on the diagram.	Check both of the following items. • On the **Project Files** tab, the JSLI document is in a web application document. • On the **Project Files** tab, double-click the web application document. Ensure the **Select target** drop-down box is set to Web Server.
The changes you make to the **HTML** source panel do not take affect.	Verify your HTML is valid.
You do not get the results you expected from your entry point.	Verify that the parameters are in the correct order in the JSLI document.
Your function does not appear to be called.	Add temporary calls to `console.log('my custom message')` to your JavaScript code to print a custom message to the console when it is called. **NOTE** To view messages printed to the console, see *Debugging a WebVI*.

RELATED INFORMATION

Creating UI Elements with JavaScript

Create UI elements to place on the panel of your web application using the Placeholder HTML Container and JavaScript Library Interface document (JSLI).

You can add custom UI elements, such as controls, to your web application by creating a placeholder on the panel. Using the placeholder, you can set the dimensions and other properties of the element that work for both flexible and absolute layouts. You then create a control reference and wire the reference into a JSLI, directly exposing the underlying HTML for the control to your JavaScript code.

Before you begin, create or open a web application.

What to Use

- *Placeholder HTML Container*
- *Obtain JavaScript Reference*

What to Do

1. Open or create a JSLI and add the JavaScript function you want to call from your web application. Add the parameters with one input having a data type of **JS Reference**.

   NOTE
   In order to directly access your HTML element, your function must have an input with a matching data type of JS Reference. You can also include an array of JS references.

   In the following example, the JavaScript function is **Reflect.set**, a function available in most browsers, with the name **Property Set**. In addition to the JavaScript Reference input, there are two additional inputs for the function.

NAME	JAVASCRIPT GLOBAL	DATA TYPE	DIRECTION	
Property Set	Reflect.set			Add parameter
return		Void	Output	
reference		JavaScript Reference	Input	
property name		String	Input	
property value		String	Input	

   NOTE
   You can rename the function and properties, however the JavaScript Global function name must be the name of the function you are referencing.

2. Open or create a WebVI (`.gviweb`) and on the panel, select **Decorations**»**Placeholder HTML Container** and place it on the panel.
3. Use the Configuration pane on the right to configure the container.

   NOTE
   Use the Configuration pane or property nodes to configure the HTML container instead of injecting HTML to configure the placement and sizing.

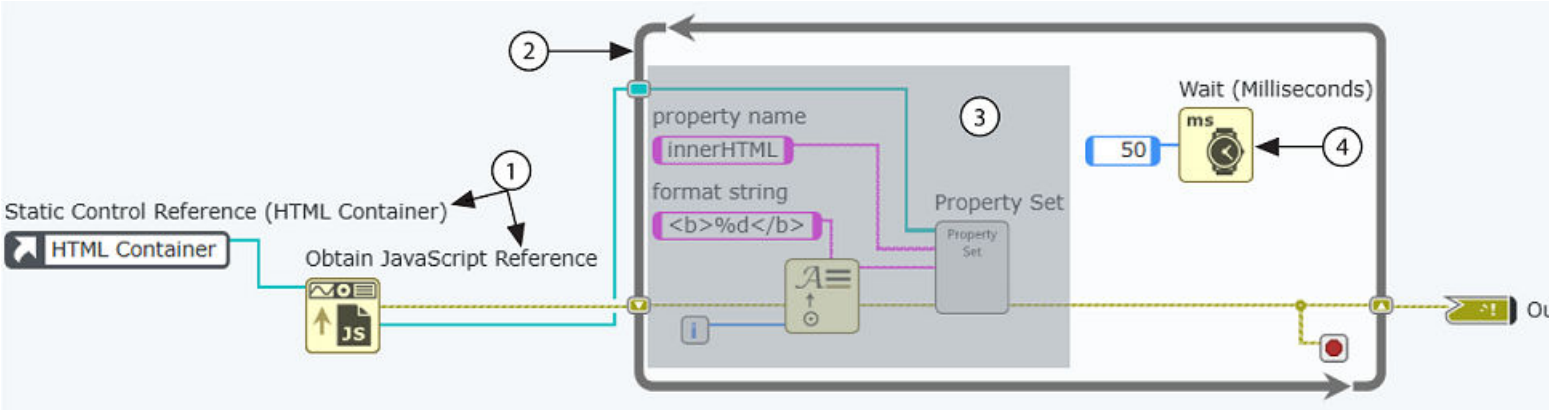
   The container is empty by default and you can populate it with a control at runtime. If you use the Configuration pane to add a placeholder image, the placeholder image is visible in the container at edit time. Select **Image visible at runtime** to make the placeholder image also visible at runtime until the contents of the container are available.

4. With the **HTML Container** selected, click **Create reference** in the Configuration pane.

   The **HTML Container** does not have a terminal, so you must create a reference to interact with it on the diagram.

5. Create the following diagram to populate the HTML container on the panel with the JavaScript function you defined in the JSLI.

   Customize the gray sections for your unique programming goals.



The `Static Control Reference` represents the **HTML Container** you placed on the panel.

TIP
You can wire the HTML container to a property node to configure the size and position of your control.

`Obtain JavaScript Reference` creates a JavaScript reference with a value of the HTML element within the container.

NOTE
The references do not have controls and are only visible on the diagram. You can wire references to terminals on a connector pane.

If you want your web application to run continuously in a web browser until it encounters an error, place your code in a `While Loop` with the conditional terminal wired to the **error out** parameter of your JavaScript function.

In this example, the Property Set JavaScript function is wired to the conditional terminal. If Property Set returns an error, the conditional terminal will stop the loop from executing.

The JavaScript function uses the JavaScript reference from Obtain JavaScript reference. You can configure the function using the properties you defined in the JSLI document along with any other supporting code.

In this example, the `Property Set` function outputs the number of loop iterations by converting the iteration counter output into a string using the HTML <b> tag.

To reduce loading time and improve performance when you run your built web application on a web browser, add a `Wait` node to any WebVI that uses an infinite `While Loop`.

Troubleshooting

Ensure you properly define the JavaScript calls you make in the JSLI. Refer to the following resources for more information.

- *Defining Calls to JavaScript Functions using a JavaScript Library Interface*

- *JavaScript Resources*

RELATED INFORMATION

Considerations When Creating UI Elements with JavaScript on page 96

JavaScript UI elements may require special sizing considerations to ensure they appear correctly after you deploy your web application or share the UI element in a library.

JavaScript Reference Functionality on page 96

JavaScript references behave similarly to references for other data types, but have some unique functionality.

Considerations When Creating UI Elements with JavaScript

JavaScript UI elements may require special sizing considerations to ensure they appear correctly after you deploy your web application or share the UI element in a library.

The size of the UI element you create can change if your web application has a flexible layout or if your user configures the element using property nodes. Consider using the following strategies to ensure the UI element retains the appropriate size to fit in the Placeholder HTML Container.

- Configure with CSS—Use a style sheet to set the size of the UI element.
- Configure with the ResizeObserver API—Use the browser-based ResizeObserver API to make adjustments programmatically.

NOTE

The ResizeObserver API is supported in most browsers. Ensure your browser supports this API before implementing it in your code. Refer to the *Mozilla Developer Network documentation* for more information about the ResizeObserver API.

RELATED INFORMATION

Creating UI Elements with JavaScript on page 93

Create UI elements to place on the panel of your web application using the Placeholder HTML Container and JavaScript Library Interface document (JSLI).

Creating UI Elements with JavaScript on page 93

Create UI elements to place on the panel of your web application using the Placeholder HTML Container and JavaScript Library Interface document (JSLI).

JavaScript Reference Functionality

JavaScript references behave similarly to references for other data types, but have some unique functionality.

When using JavaScript references in your web application, you need to consider the following:

- Primitive values **null** and **undefined**—JavaScript references treat primitive values **null** and **undefined** the same as other primitives, such as strings, booleans, numbers, etc. If a value of null or undefined is returned, it generates a new JavaScript reference that you need to clean up to prevent memory leaks.
- `Not a Number/Path/Refnum?` behavior—The node tells you if the value is valid, but it does not tell you if the JavaScript reference value is null or undefined. There is not an existing node to check for values of null or undefined.
- JavaScript values—JavaScript references can hold any JavaScript value, including primitives and objects, such as JavaScript functions, HTML elements, and class instances.
- Type definitions containing a JavaScript reference—JavaScript references do not have front panel representations, so you will need to use the following process to create a type definition containing a JavaScript reference:
  1. Create a VI (.gvi) with a web server target in your web application project.
  2. Add your JavaScript reference to the diagram.
  3. In the Configuration pane, click **Change to Type Definition**.

RELATED INFORMATION

Creating UI Elements with JavaScript on page 93

Create UI elements to place on the panel of your web application using the Placeholder HTML Container and JavaScript Library Interface document (JSLI).

Creating UI Elements with JavaScript on page 93

Create UI elements to place on the panel of your web application using the Placeholder HTML Container and JavaScript Library Interface document (JSLI).

JavaScript Resources

Research JavaScript concepts using NI-recommended resources.

The items are some concepts you may want to research when using JavaScript Library Interfaces (JSLIs) and calling JavaScript functions in your web application. Each item links to the appropriate Mozilla Developer Network (MDN) documentation resource. If the JavaScript concept you want to learn about is not listed, NI recommends you search the *MDN JavaScript documentation*.

- *Asynchronous JavaScript*
- *Built-in Objects*
- *Callback Function*
- *Global Scope*
- *Immediately Invoked Function Expression (IIFE)*
- *Objects*
- *Strict Mode*

- *TypedArray Object*

RELATED INFORMATION

Hosting a Web Application on a Server

To make your web application available to other users, host your build output on a web server that is accessible to other users.

About this task

You can host your web application on any server you choose. This includes local servers and servers in the cloud. Use the following table to choose and set up a server for hosting your web application.

Options for Hosting a Web Application

Required Products	Hosting Option	Use Case	Hosting Instructions
G Web Development Software or NI SystemLink	NI Web Server	You want to manage your own server from within your network, either through G Web Development Software or NI SystemLink.	NI SystemLink and the G Web Development Software both use the NI Web Server. Follow the instructions for *Hosting a Web Application on the NI Web Server*.
SystemLink Cloud	SystemLink Cloud Server	Choose this server in either of the following cases: • You want to access your web application from the SystemLink Cloud website and on mobile devices without sacrificing security. • You want to avoid placing the server burden on your measurement hardware, such as when using real-time hardware.	Follow the instructions for *Hosting a Web Application on SystemLink Cloud*.

Required Products	Hosting Option	Use Case	Hosting Instructions
LabVIEW 2013 or later	LabVIEW Application Web Server	Choose this server in either of the following cases: • You want to host a web application directly on a LabVIEW real-time target, such as a CompactRIO. • You want to use the security features provided by LabVIEW Web services.	Follow the instructions for *Hosting a Web Application on the G Web Development Software Web Server*.
3rd party software	3rd party server	You need functionality that is not included in the NI server options.	Before you begin, you must *build a web application*. Copy your entire web application build output to the server directory. To navigate to your web application output on your machine, click **Locate directory in Windows Explorer** on the **Document** tab of your web application component document. **NOTE** You may need to configure a 3rd party server to support the application/wasm MIME type for files with the `.wasm` file extension.

RELATED INFORMATION

Building a Web Application as a Web Page on page 78

Hosting a Web Application on the NI Web Server

To make your application available to other users, host your build output on a web server that is accessible to other users.

Before you begin

Before you upload your application or WebVI on NI Web Server, you must complete the following tasks:

- Remove the URL, username, and password from the panel and the diagram to maximize the security of your web application. Refer to the *Hosting Authentication Credentials Securely* section of ***Security in NI Web Technology*** for more information.

About this task

NI SystemLink and G Web Development Software both use NI Web Server. Complete the following steps to host your web application on NI Web Server.

Procedure

1. Open your web application project (.gwebproject), navigate to **SystemDesigner**, and open **Design** view.
2. Select **Web Server** on the diagram.

   In the configuration pane under **Item**»**NI Web Server**, you will find the NI Web Server root directory path. The root directory path populates SystemDesigner based on your configuration settings in the NI Web Server Configuration Utility.

   Here is an example of a root directory path: `C:\Program Files\National Instruments\Shared\Web Server\htdocs\`.
3. Copy your entire web application output directory into the web server directory.

   To navigate to your web application output on your machine, click **Locate item in Windows Explorer** on the **Document** tab of your web application component document.
4. Open a web browser and navigate to `http://localhost/WebApp_Web%20Server/Main.html`, `localhost` is the IP address of the server and `Main.html` is the file name of the top-level WebVI in your web application.

RELATED INFORMATION

Choosing Remote Settings

Choosing Remote Settings

Hosting a Web Application on the NI Application Web Server

To make your web application available to other users, host your build output on a web server that is accessible to other users.

Before you begin

Before you begin, you must complete the following tasks.

- Install a version of LabVIEW from 2013 or later.
- Build your web application.

Procedure

To host your web application on the Application Web Server, choose from the following options.

Option	Description
Host your web application on the Application Web Server through LabVIEW Web Services.	1. In your LabVIEW Web Service, create a public content folder. For more information, refer to *integrating static content into a web service*. 2. Copy your entire web application output directory into the public content folder you created. 3. *Publish your LabVIEW Web Service*. **NOTE** Download the latest LabVIEW Help for the most up-to-date content.
Manually host your web application files on the Application Web Server.	1. Navigate to the Application web server document root. Example if the 32-bit Application Web Server is enabled: `C:\Program Files (x86)\National Instruments\Shared\NI WebServer\www` Example if hosting on a CompactRIO: `/var/local/natinst/www` 2. Copy your entire web application output directory into the Application web server document root. To navigate to your web application output on your machine, click **Locate directory in Windows Explorer** on the **Document** tab of your web application component document. 3. Open a web browser and navigate to `http://localhost:8080/WebApp_Web%20Server/Main.html`, where `Main.html` is the file name of the top-level WebVI in your web application. **NOTE** You can update the port in the URL, if necessary, to match the Application Web Server HTTP port.

RELATED INFORMATION

Hosting a Web Application During Development

Host WebVIs running in the G Web Development Software editor with NI Web Server or the G Web Development Software embedded web server.

NI recommends hosting your WebVI with NI Web Server. NI Web Server helps you avoid additional configurations when you use other APIs NI Web Server hosts, such as SystemLink Tags and Messages. Refer to the *NI Web Server Manual* for more information.

Use the G Web Development Software embedded web server if you are unable to configure NI Web Server on your development PC. If you choose the embedded web server, you cannot connect to APIs hosted on the NI Web Server, such as the SystemLink APIs, unless you configure NI Web Server to support the cross-origin resource sharing (CORS) mechanism. Refer to *Choosing Remote Settings* to learn more about enabling CORS for NI Web Server.

NOTE

The G Web Development Software embedded web server has an origin of http://localhost:`<port>`. Your operating system assigns the port based on availability and varies between instances of the editor. Any HTTP request made by a WebVI hosted in the editor that does not target a user resource is considered a cross-origin request.

To set your hosting preference in G Web Development Software, select **File**»**Preferences**»**Web Server**.

RELATED INFORMATION

Considerations When Accessing Data from Web Services on page 130

If you want to create a web application that sends requests to and receives responses from a web service, you need to know the origin of the web service that hosts the web application as well as the origin of the target web service.

Running Operations Using the Command Line Interface

Run a predefined set of operations in G Web Development Software using the command line interface (CLI). For example, use the CLI to automate the build process of applications you develop.

Procedure

1. Open a command prompt.
2. Change directories to the location of the CLI.

   NOTE

   The default location is `C:\Program Files\National Instruments\G Web Development Software <version>`.

3. Execute a command using the following syntax:

```
gwebcli.exe <operation name> -<arguments>.
```

TIP

To access the built-in help for the CLI, execute the `gwebcli.exehelp` command. To learn more about a specific operation, execute the `gwebcli.exe <operation name> --help` command.

Global Arguments

Refer to the table below for a list of arguments you can use when executing an operation with the command line interface.

NOTE

All arguments are case-sensitive.

Argument	Description	Example
• `-v` • `--verbose`	Increases the detail of the output log.	`gwebcli.exe build-application -p "C:\G Web Projects\Project.gwebproject" -n "Application.gcomp" -v`

RELATED INFORMATION

Predefined Command Line Operations

The command line interface supports the following operations:

- *build-application*—Builds the specified application into a binary.
- *build-library*—Builds the specified library into a binary.
- *optimize-project*—Improves the load time and performance of your projects by loading, compiling, and saving project files during the build process.

RELATED INFORMATION

build-application

Builds the specified application into a binary.

To build an application from the CLI, use the following syntax:

```
gwebcli.exe build-application -p=<relative or absolute location of the project> -n=<name of the application> -t=<Web Server target containing the application> -s
```

For example:

```
gwebcli.exe build-application -p="C:\G Web Projects\Project.gwebproject" -n="Application.gcomp" -t="Default Web Server" -s
```

The following table lists the available arguments for this operation.

Argument	Required	Description	Example
• -p • --path=VALUE	Yes	Specifies the relative or absolute path of the project containing the application to build.	• -p="C:\G Web Projects\Project.gwebpr • --path="C:\G Web Projects\Project.gwebpr
• -n • --name=VALUE	Yes	Specifies the name of the application to build.	• -n="Application.gco • --name="Application.g
• -t • --target=VALUE	This argument is required if the project contains SystemDesigner targets.	The target that contains the application to build. This option is only required if the project contains SystemDesigner targets.	• -t="Default Web Serv • --target="Default Web S

Argument	Required	Description	Example
• `-s` • `--save`	No	Saves the application and project file after building. Use this argument when you select **Auto-increment version on build** in the application.	—

RELATED INFORMATION

build-library

Builds the specified library into a binary.

To build a library from the CLI, use the following syntax:

```
gwebcli.exe build-library -p=<relative or absolute location of the project> -n=<name of the library> -t
```

For example:

```
gwebcli.exe build-library -p="C:\G Web Projects\Project.gwebproject" -n="Library.gcomp" -t
```

NOTE

To use the `build-library` operation, the build machine must have the G Web Development Features for TestStand licenses.

The following table lists the available arguments for this operation.

Argument	Required	Description	Example
• `-p` • `--path=VALUE`	Yes	Specifies the relative or absolute path of the project containing the library to build.	• `-p="C:\G Web Projects\Project.gwebpr` • `--path="C:\G Web Projects\Project.gwebpr`
• `-n` • `--name=VALUE`	Yes	Specifies the name of the library to build.	• `-n="Library.gcomp` • `--name="Library.gco`
• `-t` • `--target=VALUE`	This argument is required if the project contains SystemDesigner Web Server targets.	Specifies the SystemDesigner Web Server target that contains the library to build. You can view Web Server targets you added to SystemDesigner by opening the library in the editor and clicking the **Select target** drop down.	• `-t="Default Web Serv` • `--target="Default Web S`
• `-s` • `--save`	No	Saves the library and project file after building. Use this argument when you select **Auto-increment version on build** in the library.	—

RELATED INFORMATION

optimize-project

Improves the load time and performance of your projects by loading, compiling, and saving project files during the build process.

Similar to *mass compiling in LabVIEW*, the `optimize-project` command:

- Verifies the existence of subVIs and links them to the main VI.
- Updates files to the current G Web Development Software version.
- Reports corrupt files which can prevent your project from loading correctly.

To optimize an example or tutorial from the CLI, use the following syntax:

```
gwebcli.exe optimize-project -p=<relative or absolute
location of the project> -t=<time in seconds> -s=<name of
target to exclude>
```

For example:

```
gwebcli.exe optimize-project -p="C:\G Web Projects
\Project.gwebproject" -t=60 -s="Default Web Server"
```

Argument	Required	Description	
• `-p` • `--path=VALUE`	Yes	Specifies the relative or absolute path of the project to optimize. **NOTE** You can reference multiple paths in a comma-separated list.	• `-p="C` • `--path=".\Add\Ac`
• `-s` • `--skipTargets=VALUE`	No	Specifies which targets to exclude in the optimization process.	• • -
• `-t` • `--timeout=VALUE`	No	Specifies the amount of time in seconds before the operation exits.	• •

RELATED INFORMATION

distribution

Builds a distribution(.lvdist) file.

To build a distribution from the CLI, use the following syntax:

```
gwebcli.exe distribution -p=<relative or
absolute path of the project> --dn=<name of the .lvdist
```

```
file>
```

For example:

```
gwebcli.exe distribution -p="C:\G Web Projects
\Project.gwebproject" --dn="Application.lvdist"
```

The following table lists the available arguments for this operation.

Argument	Required	Description	
• -p • --ppath=VALUE	Yes	Specifies the source path. This is a path to the .gwebproject containing the .lvdist to build.	• • --p
• --dn • --distributionDocumentName=VALUE	Yes	Specifies the name of the distribution document to build.	• • --d
• -d • --outputDirectory=VALUE	No	Specifies a custom output directory for the distribution.	
--version=VALUE	No	Specifies the version for the distribution. The distribution build version does not automatically increment when you use this parameter.	
--addonDirectory=VALUE	No	Specifies the custom addon installation location. By default, it is set to the "Addons" directory in the .exe directory.	

Argument	Required	Description	
`--autoAddDependencies`	No	Specifies whether to automatically calculate and add dependencies.	
`--retainIntermediaryOutputDirectory=VALUE`	No	Specifies whether to retain intermediary output directory upon building a package.	
`--save`	No	Specifies whether to save the distribution when finished. The default value is False.	
• `--do` • `--distributionOutputType=VALUE`	No	Specifies the output type of the distribution to build. The default is the output type set in the distribution. Supported output types: nipackage and nipackageinstaller.	

applylicensing

Applies licensing to a component.

To apply licensing to a component from the CLI, use the following syntax:

```
gwebcli.exe applylicensing -c=<relative path to
the component file> --ln=<license feature name>
--lv=<version>
```

For example:

```
gwebcli.exe applylicensing -c="C:\Desktop\Web
Application Project\WebApp.gcomp\WebApp.gcomp"
```

```
--ln="License" --lv="2"
```

The following table lists the available arguments for this operation.

Argument	Required	Description	Examp
• -c • --cpath • --compPath	Yes	This path must point directly to the component to be licensed.	• -c="C:\Desktop Project\WebApp.g • --cpath="C: Appli Project\WebApp.g • --compPath="(Appli Project\WebApp.g
• --ln • --licenseFeatureName=VALUE	Yes	Specifies the component license in the corresponding license file (.lc).	• --ln="L • --licenseFeatu
• --lv • --licenseVersion=VALUE	Yes	Specifies the version of the license in the license file (.lc) that corresponds to the component.	• --lv • --license
• --dp • --dontProtectFiles	No	Specifies that we do not want to protect the source files under a component when we license it. By default, all the source files are protected when licensed.	—

Packaging an Application or Library

Package your application or library for distribution.

Before you begin

Before you begin, create an ***application*** or a ***library*** before completing the following tasks:

Procedure

- *Package an Application*—Use a Package document (`.lvdist`) to build an application into a package.
- *Package a Library*—Use a Package document (`.lvdist`) to build a library into a package or package installer.

Considerations for Packaging a Web Application

Web applications have unique deployment factors and dependencies you need to know before configuring a package or package installer.

You can package web applications using the distribution document in software. However, you must deploy web applications to a web server. After you add your web application to the package, G Web Development Software automatically adds NI Web Server to the dependencies list. The NI Web Server location is the default root directory for your web application.

NOTE

You can view this file path by *enabling and opening* ***SystemDesigner*** in your web application project and selecting **Web Server** on the diagram. The file path appears in the configuration pane on the right under **NI Web Server.**

G Web Development Software automatically adds `/<web application name>` to the end of the file path when you add a component to the package document in one of the following ways.

- Right-click the component in the project tree and choose **Create Package/Installer with item**.
- Open a distribution document and choose **Add Files...**.

For web applications that use SystemLink APIs, such as Tags and Messages, add those dependencies to the package manually.

RELATED INFORMATION

Packaging an Application on page 111

Packaging a Library on page 112

Packaging an Application on page 111

Packaging a Library on page 112

Packaging an Application

Use a Package document (`.lvdist`) to build an application into a package. The Package document contains the build configuration and metadata for your package, such as version, maintainer, and display name.

Procedure

1. Click **File**»**New**»**Package/Installer**.

   The Package document opens in the project.

2. Drag the files you want to distribute from the **Project Files** tab to the **Files** section.

   You can include applications and other support files.
   The Package document automatically calculates the package dependencies that your application requires.

3. Review the contents of the **Package Dependencies** section, which displays package dependencies your application requires.

   You can include additional dependencies by clicking **Dependencies** and selecting packages under **Add Dependencies**.

4. In the **Information** section, confirm that the Package document filled the text boxes with information you want.

   The Package document automatically gathers information from the project and your NI User Account to complete the **Information** section. The values in the text boxes become metadata in the package. NI Package Manager reads the metadata of packages to sort them and display information about them to users.

5. On the **Document** tab, choose the target on which to build the package contents in **Build target**.

6. Select the package output type from **Output type**.

7. In **Output directory**, specify the location to save the package output.

8. Click **File**»**Save all**.

9. Click **Build**.

   The **Build Queue** tab shows the status of the build process.

What to do next

Right-click the package and click **Locate directory in Windows Explorer** to locate the package in the output directory. You can distribute your software content using NI Package Manager or other distribution channels.

RELATED INFORMATION

Packaging a Library

Use a Package document (`.lvdist`) to build a library into a package or package installer.

The Package document contains the build configuration and metadata, such as version, maintainer, and display name, for your package.

Before you begin

Before you begin, *create a library* of source files using a Library document (`.gcomp`).

Procedure

1. Click **File**»**New**»**Package/Installer**.

   The Package document opens in the project.

2. Drag the library or libraries you want to distribute from the **Project Files** tab to the **Files** section.

   You can include applications and other support files.

   > **NOTE**
   > Make sure that the Destination is set to **Public Add-ons** and that the company information is filled in. Your libraries will be installed in `G Web Development Software[version]/addons/[company name]/[product name]`.

   The Package document automatically calculates the package dependencies that your library requires.

3. In the right rail, click **Create palette file**.

4. Use the pull-down to choose whether you will create a **G diagram palette** or a **G panel palette** library.

5. Review the contents of the **Package Dependencies** section, which displays package dependencies your library requires.

   You can include additional dependencies by clicking **Add dependencies** and selecting packages.

6. In the **Information** section, confirm that the values in the Package document are correct.

   The Package document automatically gathers information from the project and your NI User Account to complete the **Information** section. The values in the text boxes become metadata in the package. NI Package Manager reads the metadata of packages to sort them and display information about them to users.

7. In **Output type** on the **Document** tab, select **Package** for the output type. Select **Package Installer** if you want to create an installer that includes all of your packages.

8. In **Output directory**, specify the location to save the package output.

9. Click **File**»**Save all**.

10. Click **Build**.

    The **Build Queue** tab shows the status of the build process.

What to do next

Once the package successfully builds, right-click the package and click **Locate directory in File Explorer** to find and install your package to a target machine.

RELATED INFORMATION

Types of Package Outputs

With a Package document (`.lvdist`), you can create a package or package installer to distribute applications and/or add-ons to clients.

The following table helps you determine when to use a package or package installer.

Output Type	Definition	File Type	Installation Method	Configuration Options
Package	A single compressed file containing software intended for installation on a target machine.	`.nipkg`	NI Package Manager or SystemLink **NOTE** In most cases, you must add the package to a feed that also includes the dependencies of the package before installing.	Add the package to a feed—clients of the software can subscribe to the feed and install the package from Package Manager via network access.

Output Type	Definition	File Type	Installation Method	Configuration Options
Package Installer	A folder containing the main software package, package dependencies, and an executable file that extracts the package and installs it on a target machine. **NOTE** By default, a package installer includes all of the package dependencies in the output so clients of the software can install the package without network access.	.exe	Run the package installer file. **NOTE** Web applications can be hosted and used directly via SystemLink.	• Reference package dependencies using feeds instead of including the dependencies in the output—when a package installer that references feeds runs, it retrieves dependencies from the feed locations. • Require installation of the exact hierarchy and versions of all the dependencies—when clients of the software install the package, the installer installs the exact versions of the dependencies found on the machine that built the package installer when it was built. **NOTE** This is recommended only for installing on a clean machine because it might upgrade or downgrade installed packages on the target machine.

RELATED INFORMATION

Collaborating on Web Applications

Learn about tools that help your team streamline interactions with project files during software development.

Procedure

Consider using one of the following tools when working on a project with a team of developers.

These tools are designed to support developer efficiency and preserve the integrity of your code base.

- *Package Dependencies Document*—Store a list of packages a project uses so you can set up a development system.
- *Source Code Control*—Back up and manage versions of your source files.

RELATED INFORMATION

Package Dependencies

In projects, a *package dependency* is a package installed on the development system and used in the project.

The *Package Dependencies* document (`.sls`) stores a list of packages a project uses so you can set up a development system with the required packages. With the Package Dependencies document, you can do the following:

- Share a project that allows a recipient to easily set up their development system with the package dependencies of the project.
- Create a project that serves as a template. Other developers can use the template project to set up their development system and begin development of the project.
- Update the list of package dependencies any time you add a package dependency to the project. Share the updated list with other developers so they can see new package dependencies you add to the project.

The Package Dependencies document is a Salt State file (`.sls`). For more information about Salt States, visit the SaltStack Documentation website and search for the SALT.STATE.PKG state module.

RELATED INFORMATION

Sharing a Project and Including Package Dependencies on page 116

Sharing a Project and Including Package Dependencies

Use the Package Dependencies document to store a list of packages a project uses so you can set up a development system with the required packages.

Procedure

1. Create a project that uses NI software, drivers, or third-party packages.

2. *Capture package dependencies on the source system*—Identify the packages a project requires and store a list of those packages in the Package Dependencies document.

   NOTE
   When you capture package dependencies, you must have the required packages installed on the source system and the code in the project must be in working condition.

3. Copy the project folder and save it on the target system.

4. *Resolve package dependencies on the target system*—Set up a development system by installing packages listed in the Package Dependencies document.

Capturing the Package Dependencies of a Project

Identify the packages a project requires and store a list of those packages in the Package Dependencies document.

Procedure

1. On the **Project Files** tab, right-click the project and select **Capture package dependencies**.

   The Package Dependencies document opens, scans the project, and displays a list of packages the project requires.

2. (Optional) Make changes to the project, return to the Package Dependencies document, and click **Recapture dependencies**.

3. Click **File**»**Save all**.

What to do next

After you capture the package dependencies of a project, copy the project folder, save it on another development system, and use the Package Dependencies document to resolve package dependencies on that system.

RELATED INFORMATION

Sharing a Project and Including Package Dependencies on page 116

Resolving Missing or Mismatched Package Dependencies on a Development System on page 117

Resolving Missing or Mismatched Package Dependencies on a Development System

Set up a development system by installing packages listed in the Package Dependencies document.

Before you begin

Before you can resolve package dependencies on a development system, copy the project folder from the system on which the project was originally developed, then save the folder on the target system.

Procedure

1. Launch the project, then open the Package Dependencies document.

   When you open the Package Dependencies document, it searches the system to identify missing or mismatched packages for the project.

2. Click **Resolve**.

   The Package Dependencies document displays checkboxes next to the missing or mismatched packages. You can review the packages before installing them with NI Package Manager. By default, the Package Dependencies document selects all missing or mismatched packages.

   NOTE
   NI Package Manager does not support downgrading packages.

3. Click **Resolve selected**.

   The Package Dependencies document launches NI Package Manager.

4. Follow the prompts in NI Package Manager to install the packages you selected.

   Depending on the type of package you install, you may need to restart G Web Development Software or the development system.

RELATED INFORMATION

Sharing a Project and Including Package Dependencies on page 116

Capturing the Package Dependencies of a Project on page 117

Recommendations for Developing a Project Stored in a Source Code Control Repository

Because graphical code interacts differently with source code control systems than text-based code does, follow NI's recommendations for developing a project stored in a source code control (SCC) repository.

NOTE
The following information uses examples from Subversion (SVN) and Git. However, you can apply these concepts when interacting with any source code control system.

Recommendation	Details
Create modular code within the project.	*Modularity* is a software design technique where units of code are collected together into logical and functional groups called *modules*. Modularity reduces the likelihood that changes to one module of code will introduce unexpected changes to another module of code. To build modularity into your project, create VIs and subVIs that handle specific, limited tasks. Use applications and libraries (`.gcomp`) when you want to group files together to perform a cohesive set of tasks. In team-based development environments, modular code helps create logical divisions of work between developers.
Avoid making simultaneous changes in a VI.	The likelihood of broken code increases when multiple developers modify the same VI simultaneously. Merge conflicts are possible when developers submit conflicting changes in a VI. If you use modular design techniques in the project, you can assign modules to individual developers on your team to reduce the likelihood that multiple developers will modify a VI simultaneously. Some source code control systems allow you to lock files when you check them out. Use this feature to prevent others from modifying a VI while you have it checked out. If you cannot avoid simultaneous development of VIs in a project, use ***NI Compare*** to view differences between two versions of a VI, then manually combine the differences between each version.
If you make changes to source files in the project, communicate with other developers on your team.	Modifications you make to source files in the project can produce changes to the project file as well as other files. Communicating with your team when you make changes helps other developers to quickly identify conflicts with their work. Using a collaboration tool can help streamline communication on a development team. Some collaboration tools, as well as some source code control systems, can auto-generate emails when changes to the repository occur.

Recommendation	Details
Configure the project directory in the centralized repository to ignore submissions to the following project sub-folders: • `Builds` • `.cache`	Changes within these sub-directories do not affect the functionality of source code in the project. In SVN, apply the `svn:ignore` property to the sub-directories. In Git, list the sub-directories in the `.gitignore` file.
Perform code reviews before submitting code to the central repository.	Code reviews improve the quality of the code in your project and preserve the integrity of your source code as it changes during project development. Consider implementing a tiered review framework on your development team. For example: • Peer reviews verify that the submission does not contain defects. • Owner reviews verify that the submission adheres to architectural guidelines. As a code reviewer, you can use ***NI Compare*** to compare two versions of a project file or VI.
Create a Package Dependencies (`.sls`) document and keep it up to date during project development.	When you include a Package Dependencies (`.sls`) document in the project, developers can update this file as they create code, then other developers can use it to keep their development systems in sync.
For VIs, disable the auto-merge utility in your source code control system.	The underlying source of a VI file is XML-based text. When merging two versions of a VI, the auto-merge utility might combine lines of text from each version in a way that corrupts the VI and prevents the VI from opening. If using the auto-merge utility on a VI breaks code, you may need to revert the VI to an older version to recover your code.
Minimize occurrences of renaming and moving files.	When you rename or move a file, SCC systems delete the file and create a new file with the new name or location you provide. This operation erases any version history of the file with its previous name or location. Other developers could introduce a breaking change when they submit files that use the old name. You might encounter this behavior when you move a VI file to an application or library.

Source Code Control Models

Source code control systems are built using either a centralized or distributed model.

Centralized Model —

In centralized source code control, a project exists in a single central repository, usually on a server. Users create a local repository on a client where they make modifications to the project, then upload their changes directly to the central repository.

Apache Subversion® (SVN) is one example of an open-source system using the centralized model.

Distributed Model —

In distributed source code control, a project exists in a single central repository. Clients have two local repositories: a clone of the central repository and a working repository where users make modifications. Users submit modifications to the working repository often during development. When users finish development work, they push their changes to the central repository.

Git™ is one example of an open-source system using the distributed model.

Recommendations for Interacting with a Centralized Source Code Control System while Developing a Project

Learn how to interact with a centralized source code control provider while developing a project.

NOTE
You can apply these concepts when interacting with any centralized source code control provider.

Checking Out Files and Making Modifications

Recommendation	Details
Lock webVI files to prevent other developers from making simultaneous changes.	Locking webVI files when you check them out reduces the chances of code-breaking merge conflicts.
Update your local copy of the project regularly during development.	Keeping your local copy of the project in sync with the central repository ensures you are developing code against the most recent version of the project. Use the `<centralized source control provider> update` command, where `<centralized source control provider>` is, replace with the name of your provider.

Recommendation	Details
Exit G Web Development Software before updating your remote copy.	If you leave G Web Development Software open when you update your local copy, it is possible to corrupt files in the project.

Checking In Files

Recommendation	Details
Check in all files within the project folder that have changes.	You may notice that some files have changes even if you did not modify them. In particular, the project file will frequently change due to changes in other files. Your source code control provider should scan the project folder for changes. Use this feature to identify all files with modifications.
Ignore the following directories when you submit to the central repository: • `Builds` • `.cache`	Changes within these sub-directories do not affect the functionality of source code in the project. The project directory in the central repository may already be configured to ignore the folders. If not, use commands to configure your source code control provider to ignore the folders.
Add a descriptive comment to your commit.	Descriptive comments help you and other developers interpret changes you submit to the repository.
After you commit files to the repository, notify other team members.	Other developers may need to change their code based on changes you make. Some collaboration tools, as well as some source code control systems, can auto-generate emails when changes to the repository occur.

RELATED INFORMATION

Recommendations for Developing a Project in a Distributed Source Code Control System

Learn how to interact with a distributed source code control provider while developing a project.

NOTE
The following information uses terminology and commands from Git. However, you can apply these concepts when interacting with any distributed source code control provider.

Obtaining a Working Copy of a File and Making Modifications

Recommendation	Details
Merge from the central repository regularly during development.	Keeping your local copy of the project in sync with the central repository ensures you are developing code against the most recent version of the project. Fetch and merge to update code in your local repository.
Commit files to your local repository often.	Commit to your local repository as often as you want. It is common to commit locally many times per day.

Committing and Pushing Files

Recommendation	Details
Push all files with changes in the project folder.	Push any files with changes. Most source code control providers scan for changes to the project folder to help you identify changed files.
Ignore the following directories when you submit to the central repository: • `Builds` • `.cache`	Changes within these sub-directories do not affect the functionality of source code in the project. The project directory in the central repository may already be configured to ignore the folders. If not, include the directories in a `.gitignore` file. Refer to Git documentation for more information.
Add descriptive comments when you commit to your local repository or push to the central repository.	Descriptive comments help you and other developers interpret changes you submit to the repository.

RELATED INFORMATION

Merging Files and Resolving Conflicts

Manually editing and merging files helps you maintain stable source code when merge conflicts occur.

Recommendation	Details
Merge project (`.lvproject`) files manually.	Merge project files manually rather than using the auto-merge utility in your source code control system. One expected merge conflict in the project file is the checksum, which records the state of the file when it was last loaded. When manually merging the project file, you can select the checksum from either version. G Web Development Software always reloads the project and generates a new checksum.
Manually edit a VI to resolve conflicts between two versions.	Avoid performing merge operations on VIs. If you identify a conflict between two versions of a VI, view the two versions in ***NI Compare*** to locate differences, then edit the code to satisfy the requirements of a project.

Code Snippets

A *code snippet* is a PNG image of code that includes the functionality of a VI file.

You can drag a snippet directly onto the diagram and G Web Development Software populates the code depicted in the snippet. You can embed snippets in documents, forum posts, and emails to quickly share and collaborate on code. Snippets are also useful for keeping sections of code that you use frequently handy.

RELATED INFORMATION

Creating a Code Snippet

Use code snippets to store and share G Web Development Software code as PNG files that become graphical code when you drag them onto the diagram.

Procedure

1. Open a project.
2. On the diagram, highlight the code you want to capture as a snippet.
3. Click **Edit**»**Create snippet from selection**.

4. Choose where to save the snippet, give it a descriptive name, and click **Save**.

NOTE
If you edit a snippet in an image editor, the editor will remove the VI information from the file and you will not be able to drop the snippet onto the diagram.

TIP
Sharing snippets is most useful for simple code. If your snippet includes dependencies, such as subVIs or classes, anyone who uses your snippet must have those dependencies in their project or they will see a missing dependency error.

RELATED INFORMATION

Adding a Code Snippet to the Diagram

Use code snippets to drag sections of code onto your diagram.

Before you begin

Before you begin, make sure you have any subVIs included in the code snippet in your project.

Procedure

1. Save the snippet to your machine.

   NOTE
   If you edit a snippet in an image editor, the editor will remove the VI information from the file and you will not be able to drop the snippet onto the diagram.

2. Open your project and navigate to the diagram of the VI you want to add the snippet to.

3. Drag the snippet from its location on your machine to the diagram.

   The code depicted in the snippet appears on the diagram.

RELATED INFORMATION

Communicating Data with a Web Application

Send and receive data from files and web services with your web application.

- *Access resource files from a WebVI*—Use WebVIs to include CSS, JavaScript, and other files in your web application.

- *Access data from a web service*—Learn more about sending requests to and receiving responses from a web service, including same-origin and cross-origin requests, and configuring the LabVIEW Web Service for access to your web application resources.
- *Communicate with a web service using a WebVI*—Use WebVIs to communicate with web services, such as SystemLink Tags and Messages, on the same network as your web application or cloud web services.

RELATED INFORMATION

Accessing Resource Files from a WebVI

You can include CSS, JavaScript, images, and other types of files in your web application and access them from a WebVI.

NOTE
For information about including JavaScript files for a JavaScript Library Interface document, refer to the Integrating a JavaScript File Into a Web Application help topic.

Before you begin

Before you begin, create or open a web application project.

Procedure

1. In the **Project Files** tab, right-click the application document file (`.gcomp`) and select **Import files**.

   NOTE
   You must save the application document before you can import files.

2. Navigate to the file you want to add and click **Open**.
3. Open the WebVI you want to reference the resource file from, and choose one of the following options based on file type.

Type of file	How to reference the file from your WebVI
CSS	In the `<head>`/`head>` tags, add the following lines of code: `<link rel="stylesheet" href="my-style-sheet.css" />` Refer to the *Mozilla Developer Network* for more information on using CSS to customize the appearance of your UI.

JavaScript	In the <head>↙head> tags, add the following lines of code: `<script src="my-script.js"></script>` Refer to the *Mozilla Developer Network* for more information on using JavaScript to add custom functionality to your web application.
Image	• For images that are hosted externally, complete the following steps: 1. Switch to the panel and add a URL Image control to the panel. 2. On the **Item** tab, enter the path to your image file in the **Source URL** field. • For images on disk, complete the following steps: 1. Import the image into your application document file (`.gcomp`). 2. Switch to the panel and drag the image from the **Project Files** tab onto the Panel.

RELATED INFORMATION

Integrating a JavaScript File Into a Web Application on page 88

Connect a JavaScript file to your web application and call functions you defined in a JavaScript Library Interface (JSLI) in your web application.

Integrating a JavaScript File Into a Web Application on page 88

Connect a JavaScript file to your web application and call functions you defined in a JavaScript Library Interface (JSLI) in your web application.

Retrieving Data From a Web Service

Before you begin

Before you begin, open or create a web application project.

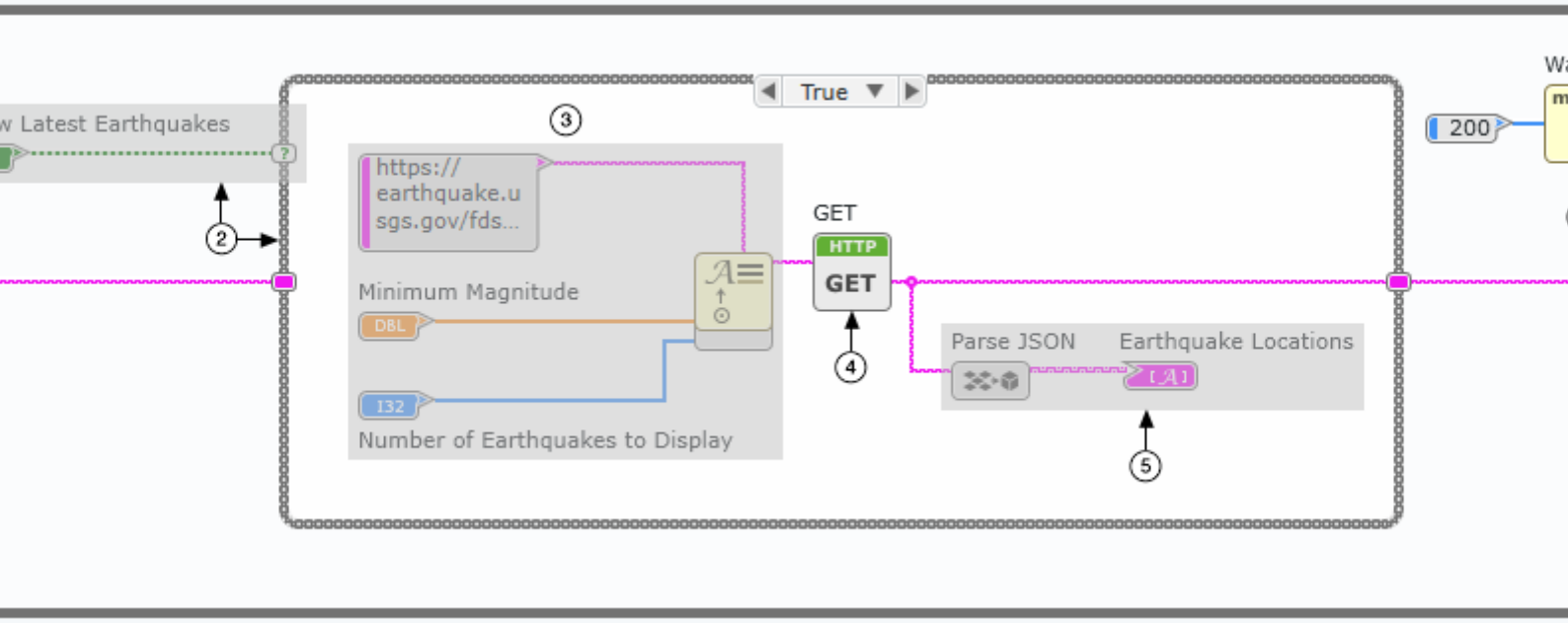
What to Use

- **GET**
- **While Loop**
- **Wait (Milliseconds)**
- **Case Structure**

What to Do

Create the following diagram to retrieve data from a web service and display that data on the panel.

Customize the gray sections for your unique programming goals.



①	If you want your web application to run continuously in a web browser, place your code in a `While Loop` with a `False` constant wired to the **condition** terminal. Otherwise, your web application runs only once in the web browser. **TIP** You can run your web application again by refreshing the page.
②	Place the code that calls the web service within a `Case Structure`. The `Case Structure` prevents the GET node from making a request to the web service in each iteration of the `While Loop`. Making a GET request each time the `While Loop` iterates is unnecessary because the data you're accessing probably doesn't change as quickly as the loop iterates. Most web services also limit how many requests you can make per second and may even ban your IP address if you make too many requests.
③	Enter the URL of the web service that you want to call. You can replace this code with a string constant containing the URL of a web service. You can also create code that programmatically creates a URL based on user input. In this example, `Minimum Magnitude` and `Number of Earthquakes to Display` determine what values make up the URL.
④	The GET node sends a request to the web service and returns data from that web service. In this example, the GET node sends a request to the U.S. Geological Survey web service and returns the latest earthquake data.

⑤	Create a subWebVI that parses the data that the web service returns. The web service in this example returns data in the JSON format, which is what most web services return. Other common data formats are XML, CSV, and YML. If a web service returns data in a format other than JSON, you can use other `String` nodes to parse that data.
⑥	To reduce load and improve performance when you run your built web application on a web browser, add a `Wait` node to any WebVI that uses an infinite `While Loop`.

Troubleshooting

If the GET node doesn't return any data or returns unexpected data, verify the following conditions:

- The URL is correct. Test this by navigating to the URL in a web browser.
- The website or web server you want to access is running. Test this by navigating to the URL in a web browser.
- You have a working internet connection.

If each of these conditions is true, try one of the following:

- To check the status of an HTTP GET request, wire an indicator to the **status code** output of the GET node.
- To check detailed information about the HTTP GET request, including its status, wire an indicator to the **headers** output of the GET node.

Refer to the *W3C* website for more information about HTTP status codes and headers.

Examples

Search within the programming environment to access the following installed examples:

- *Call a 3rd Party Web Service*
- *Call LabVIEW Web Service*

RELATED INFORMATION

Considerations When Accessing Data from Web Services

Communicating Data with Web Services using WebVIs

Creating a LabVIEW Web Service

SystemLink Data Services

Considerations When Accessing Data from Web Services

If you want to create a web application that sends requests to and receives responses from a web service, you need to know the origin of the web service that hosts the web application as well as the origin of the target web service.

For example, a web application hosted at http://localhost:8080/Demo/MyExample.html has the origin http://localhost:8080. A server origin contains three parts:

Part	Definition	Example
Scheme	Protocol the web service uses.	http://
Host	Domain name or IP address of the service.	localhost
Port	TCP port of the web service. If you don't specify a port, the web service uses the default port.	8080

Once you know the origin of the host web service and the target web service, determine whether requests from the web application to the target web service will be same-origin requests or cross-origin requests. If both origins are identical, requests from the web application to the web service are same-origin requests. If there is any difference between the two origins, requests from the web application to the web service are cross-origin requests.

Browsers running web applications do not impose restrictions on same-origin requests. Web applications that perform cross-origin requests are subject to the *Cross-Origin*

Resource Sharing (CORS) mechanism. By default, a web browser blocks all cross-origin requests made to a target web service that is not configured to support CORS.

RELATED INFORMATION

Types of Cross-Origin HTTP Requests on page 131

Enabling CORS for a Web Service on page 132

Configuring CORS for a LabVIEW Web Service on page 134

You may need different CORS configurations for each LabVIEW Web Service in your application during development and deployment.

Sending a Credentialed Cross-Origin HTTP Request on page 141

Hosting a Web Application During Development on page 102

Host WebVIs running in the G Web Development Software editor with NI Web Server or the G Web Development Software embedded web server.

Types of Cross-Origin HTTP Requests on page 131

Enabling CORS for a Web Service on page 132

Configuring CORS for a LabVIEW Web Service on page 134

You may need different CORS configurations for each LabVIEW Web Service in your application during development and deployment.

Sending a Credentialed Cross-Origin HTTP Request on page 141

Hosting a Web Application During Development on page 102

Host WebVIs running in the G Web Development Software editor with NI Web Server or the G Web Development Software embedded web server.

Types of Cross-Origin HTTP Requests

Cross-origin HTTP requests, either simple or non-simple, determine whether the browser asks the target web service before sending a request.

Simple requests are cross-origin HTTP requests that do not require prior approval from the target web service before the request can be sent by the browser. An HTTP request must meet the following criteria to execute as a simple request:

- Use a GET, HEAD, or POST node to make the request.
- Include only *CORS-safelisted request-headers* in the request.
- Set the Content-Type request header to one of the following values:
  - `application/x-www-form-urlencoded`
  - `multipart/form-data`
  - `text/plain`

Non-simple requests are cross-origin HTTP requests that must get approval from the target web service to send the actual HTTP request. The web browser sends a CORS *preflight request* to the target web service to ask for approval. The response to the CORS preflight request determines if the web browser can proceed to send the actual HTTP request. A cross-origin HTTP request executes as a non-simple request if it violates any of the criteria for a simple request.

CORS also enables *credentialed requests*. Credentialed requests may use HTTP cookies and HTTP Authentication headers, or allow TLS client certificates. By default, browsers do not include credentials with a cross-origin HTTP request. However, you can use the `Configure CORS` node in a WebVI to include credentials with a cross-origin request. The `Configure CORS` node has no effect on same-origin HTTP requests.

RELATED INFORMATION

Considerations When Accessing Data from Web Services on page 130

If you want to create a web application that sends requests to and receives responses from a web service, you need to know the origin of the web service that hosts the web application as well as the origin of the target web service.

Hosting a Web Application on the NI Application Web Server on page 100

To make your web application available to other users, host your build output on a web server that is accessible to other users.

HTTP Nodes

Configure CORS

Enabling CORS for a Web Service

To incorporate resources from a different origin into your web application, configure CORS for the server hosting those resources.

Procedure

1. Refer to the web service documentation or contact the web service administrator to verify if the web service allows cross-origin requests using CORS.

   NOTE
   The target web service must enable cross-origin requests to allow HTTP requests from a WebVI executing in the development environment. Refer to ***Hosting a Web Application During Development*** for more information.

2. Determine whether the web service requires credentialed CORS requests.

   If the web service requires credentialed requests, refer to *Sending a Credentialed Cross-Origin HTTP Request* to configure your application to send credentialed requests.

3. Ensure the web service administrator includes an `Access-Control-Allow-Origin` header in HTTP responses to enable cross-origin HTTP requests to the web service.
   - For non-credentialed simple CORS requests, set the `Access-Control-Allow-Origin` header to one of the following values:
     - The wildcard value (*) that allows any origin to access a resource. NI recommends using this value only during the development of your application.
     - The origin of the web application performing the HTTP request.
   - For credentialed simple requests, ensure the web service includes the following header values in responses to the web application:
     - `Access-Control-Allow-Origin` header set to the origin of the web application performing the HTTP request.
     - `Access-Control-Allow-Credentials` header set to True.

What to do next

Refer to the *Mozilla Developer Network documentation on HTTP access control* for additional information on advanced CORS configurations, including:

- Responding to a CORS preflight request during a non-simple CORS request
- Handling headers for browser caches
- Accepting additional request headers from a browser

- Allowing the browser to have access to additional response headers

RELATED INFORMATION

Considerations When Accessing Data from Web Services on page 130

If you want to create a web application that sends requests to and receives responses from a web service, you need to know the origin of the web service that hosts the web application as well as the origin of the target web service.

Configuring CORS for a LabVIEW Web Service on page 134

You may need different CORS configurations for each LabVIEW Web Service in your application during development and deployment.

Sending a Credentialed Cross-Origin HTTP Request on page 141

Considerations When Accessing Data from Web Services on page 130

If you want to create a web application that sends requests to and receives responses from a web service, you need to know the origin of the web service that hosts the web application as well as the origin of the target web service.

Configuring CORS for a LabVIEW Web Service on page 134

You may need different CORS configurations for each LabVIEW Web Service in your application during development and deployment.

Sending a Credentialed Cross-Origin HTTP Request on page 141

Configuring CORS for a LabVIEW Web Service

You may need different CORS configurations for each LabVIEW Web Service in your application during development and deployment.

NOTE

The following content only applies to LabVIEW Web Services deployed to the LabVIEW Application Web Server. NI recommends deploying LabVIEW Web Services to the NI Web Server instead.

To determine the origin of a web application executing in the development environment, refer to *Hosting a Web Application During Development*. During development using a LabVIEW Web Service, configure CORS to test your application in the LabVIEW editor.

LabVIEW Web Services only allows simple cross-origin requests and cannot respond to CORS preflight requests from a web browser. Use the following table to find the best CORS configuration for your web application.

NOTE

You do not need to enable CORS if your WebVI and LabVIEW Web Service have the same origin. To achieve this, host the built WebVI in the Public Content Folder of the LabVIEW Web Service the WebVI is making HTTP requests to when running your web application.

Goal	CORS Configuration	Configuration Instructions
Share your web application resources publicly and receive requests from any origin. **NOTE** This configuration is the least secure. Any origin could access your web service.	• No credentials • Unfiltered Origins	*Configure a LabVIEW Web Service CORS for Open Access*
Allow specific origins you define or well-known origins to access your web application resources that do not require credentials.	• No credentials • Filtered Origins	*Configure a LabVIEW Web Service CORS for Filtered Access*
Allow specific origins you define or well-known origins to access your web application resources and you want to enable sharing credentials, such as cookies.	• Credentials • Filtered Origins	*Configure a LabVIEW Web Service CORS for Credentialed Access*

RELATED INFORMATION

Considerations When Accessing Data from Web Services on page 130

If you want to create a web application that sends requests to and receives responses from a web service, you need to know the origin of the web service that hosts the web application as well as the origin of the target web service.

Configure a LabVIEW Web Service CORS for Open Access on page 135

Configure a LabVIEW Web Service CORS for Filtered Access on page 137

Configure a LabVIEW Web Service CORS for Credentialed Access on page 139

NI Web Server Manual

Configure a LabVIEW Web Service CORS for Open Access

Configure CORS to allow any origin access to your WebVI resources.

This CORS configuration is the least secure configuration. Any origin can access your web service and application resources. For example, any web page the browser visits could run scripts that make requests to the web service.

NOTE

The following content only applies to LabVIEW Web Services deployed to the LabVIEW Application Web Server. NI recommends deploying LabVIEW Web Services to the NI Web Server instead.

What to Use

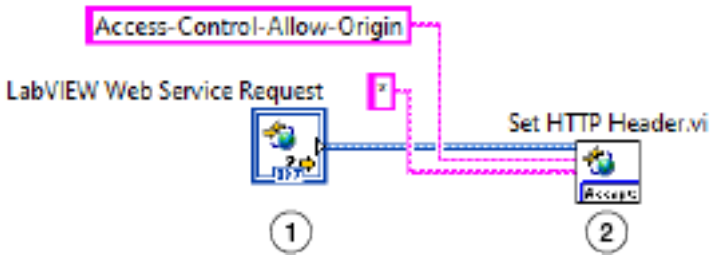
You can find the Web Services API on the Connectivity palette in LabVIEW.

NOTE
LabVIEW NXG does not support creating web services.

- LabVIEW Web Service Request
- Set HTTP Header

What to Do

Create the following diagram in a Web Resources VI to configure a LabVIEW Web Service to allow CORS from any origin.



1	A new Web Resources VI automatically adds the `LabVIEW Web Service Request` class to the block diagram and terminal pane.
2	`Set HTTP Header` sets the HTTP header value in response to a request. Use the **header**`Access-Control-Allow-Origin` to indicate if the origin making the request can access the response of your Web Service VI. Use wildcard (*) as the **header value** to allow any origin to access your WebVI resource.

Troubleshooting

If you encounter errors, try the following troubleshooting strategy:

- Verify the **header** and **header value** are correct.

RELATED INFORMATION

NI Web Server Manual

Configure a LabVIEW Web Service CORS for Filtered Access

Configure CORS to allow specific origins you define or well-known origins access to your WebVI resources.

NOTE
The following content only applies to LabVIEW Web Services deployed to the LabVIEW Application Web Server. NI recommends deploying LabVIEW Web Services to the NI Web Server instead.

What to Use

You can find the Web Services API on the Connectivity palette in LabVIEW.

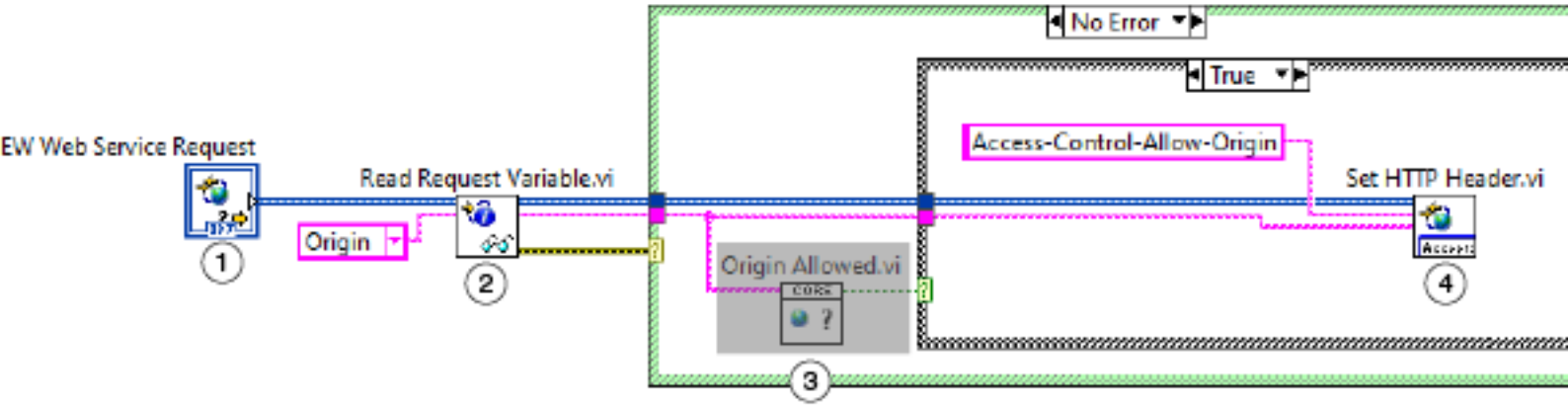
NOTE
LabVIEW NXG does not support creating web services.

- LabVIEW Web Service Request
- Read Request Variable
- Set HTTP Header

What to Do

Create the following diagram in a Web Resources VI to configure a LabVIEW Web Service to allow CORS with specific or well-known origins.

Customize the gray sections for your unique programming goals.



1	A new Web Resources VI automatically adds the `LabVIEW Web Service Request` class to the block diagram and terminal pane.

②	`Read Request Variable` checks the request for the variable you define. To check the origin of the request, enter `Origin` for the value of **variable**. Use a `Case Structure` to create two cases: a case to continue if an origin is found in the request and a case to do nothing if an origin is not found in the request. **NOTE** Web browsers in a same-origin configuration and HTTP clients outside of web browsers may not have an `Origin` header in the request. These should be handled by the web resource.
③	Define how to filter the origins you want to access your web resources. Some common implementations include the following: • Have a strict list of origins to check against • Check for a prefix on the origin, such as http://localhost • Use the LabVIEW Web Service Request and Read Request Variable to check for additional information, such as Remote Address Use a `Case Structure` to create two cases: a case to continue if the origin passes the filter you implement and a case to do nothing if the origin does not pass the filter.
④	`Set HTTP Header` sets the HTTP header value in response to a request. Use the **header**`Access-Control-Allow-Origin` to indicate if the origin making the request can access the response of your Web Service VI. The **header value** populates with the origin the filter approves.

Troubleshooting

If you encounter errors, try the following troubleshooting strategies:

- Check the implementation of your filter for errors.
- Verify the **header** is correct.

RELATED INFORMATION

NI Web Server Manual

Configure a LabVIEW Web Service CORS for Credentialed Access

Configure CORS to allow specific origins you define, well-known origins, and origins with credentials access to your WebVI resources.

NOTE
The following content only applies to LabVIEW Web Services deployed to the LabVIEW Application Web Server. NI recommends deploying LabVIEW Web Services to the NI Web Server instead.

What to Use

You can find the Web Services API on the Connectivity palette in LabVIEW.

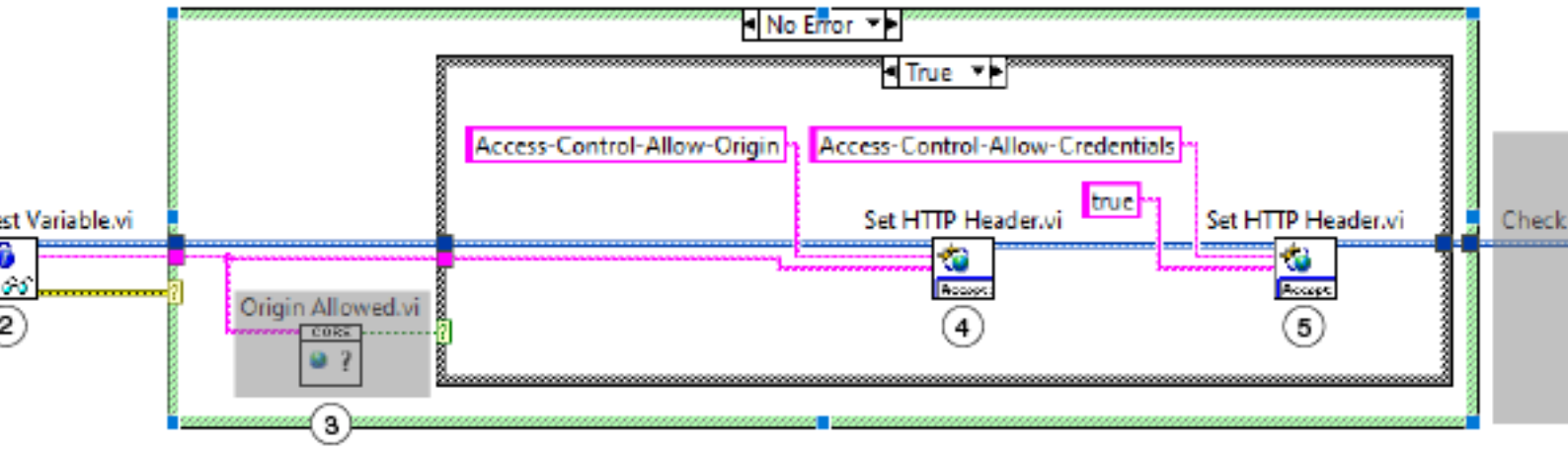
NOTE
LabVIEW NXG does not support creating web services.

- LabVIEW Web Service Request
- Read Request Variable
- Set HTTP Header

What to Do

Create the following diagram in a Web Resources VI to configure a LabVIEW Web Service to allow CORS with specific or well-known origins and origins with credentials.

Customize the gray sections for your unique programming goals.



1	A new Web Resources VI automatically adds the `LabVIEW Web Service Request` class to the block diagram and terminal pane.

②	`Read Request Variable` checks the request for the variable you define. To check the origin of the request, enter `Origin` for the value of **variable**. Use a `Case Structure` to create two cases: a case to continue if an origin is found in the request and a case to do nothing if an origin is not found in the request. **NOTE** Web browsers in a same-origin configuration and HTTP clients outside of web browsers may not have an `Origin` header in the request. These should be handled by the web resource.
③	Define how to filter the origins you want to access your web resources. Some common implementations include the following: • Have a strict list of origins to check against • Check for a prefix on the origin, such as http://localhost • Use the LabVIEW Web Service Request and Read Request Variable to check for additional information, such as Remote Address Use a `Case Structure` to create two cases: a case continue if the origin passes the filter you implement and a case to do nothing if the origin does not pass the filter.
④	`Set HTTP Header` sets the HTTP header value in response to a request. Use the **header**`Access-Control-Allow-Origin` to indicate if the origin making the request can access the response of your Web Service VI. The **header value** populates with the origin the filter approves. **NOTE** The wildcard (*) value is not valid for credentialed cross-origin requests. You must choose a filtered origin.
⑤	`Set HTTP Header` sets the HTTP header value in response to a request. Use the **header**`Access-Control-Allow-Credentials` to indicate if the origin making the request can access the response of your Web Service VI and accept credentialed information, such as cookies. Use the **header value**`true` to allow the filtered origin to access your Web Service VI. You must make a corresponding change in the Configure CORS node in your WebVI. See *Sending a Credentialed Cross-Origin HTTP Request* for more information.

6	Add code that only functions if the origin has supported credentials defined in the request. The code above uses session VIs that require browser cookies to function.

Troubleshooting

If you encounter errors, try the following troubleshooting strategies:

- Check the implementation of your filter for errors.
- Verify the **header** is correct for each Set HTTP Header.
- Check the implementation of the Configure CORS node in your WebVI.

RELATED INFORMATION

Sending a Credentialed Cross-Origin HTTP Request on page 141

NI Web Server Manual

Sending a Credentialed Cross-Origin HTTP Request

Enable credentials in your WebVI, such as including and storing HTTP cookies, adding HTTP Authorization headers, or permitting TLS client certificates to interact with a web service.

Before you begin

About this task

In addition to configuring a web service to support cross-origin credentialed requests, configure the client handle to include credentials during a CORS requests.

NOTE

Same-origin requests do not have to perform CORS configuration on the client handle of a WebVI to include credentials in a request. During a same-origin request the result of using the Configure CORS node is ignored.

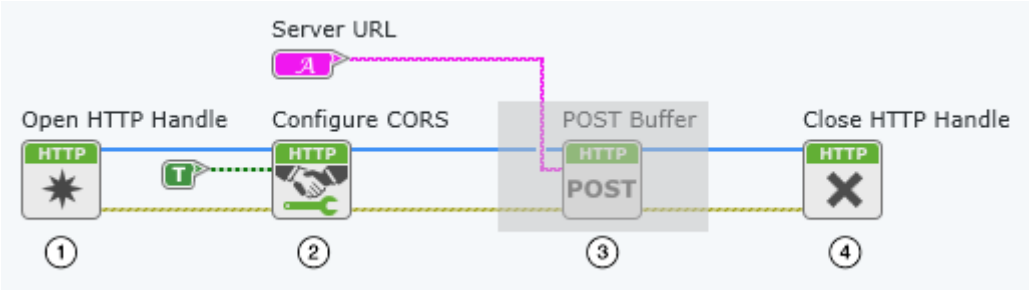
What to Use

- `Configure CORS`
- `Open HTTP Handle`
- `Close HTTP Handle`

What to Do

Create the following diagram to send a credentialed cross-origin HTTP request.

Customize the gray sections for your unique programming goals.



①	`Open HTTP Handle` creates a client handle that preserves the headers you want to add to HTTP requests the application sends.
②	`Configure CORS` adds configuration information, such as whether credentials should be included in CORS requests, to the client handle.
③	Configure the HTTP node to access resources for your application.
④	`Close HTTP Handle` closes the client handle and deletes any authentication credentials and HTTP headers associated with the handle. **NOTE** The browser cache independently stores and manages cookies a credentialed CORS request or same-origin request creates. Closing the client handle does not affect the cookies in the browser cache.

Troubleshooting

Some servers may require that your application not include credentials with a cross-origin HTTP request. If the server you send requests to responds with the `Access-Control-Allow-Origin` header set to `*`, the wildcard value, the HTTP request fails with a CORS configuration error. To resolve the error, set the **include credentials during CORS** input of

`Configure` CORS to False. To enable credentialed access for a LabVIEW Web Service, see *Configure a LabVIEW Web Service CORS for Credentialed Access*.

RELATED INFORMATION

Considerations When Accessing Data from Web Services on page 130

If you want to create a web application that sends requests to and receives responses from a web service, you need to know the origin of the web service that hosts the web application as well as the origin of the target web service.

Configuring CORS for a LabVIEW Web Service on page 134

You may need different CORS configurations for each LabVIEW Web Service in your application during development and deployment.

Communicating Data with Web Services Using WebVIs

You can communicate with web services on the same network and cloud web services using WebVIs.

Use the following tables to choose the best communication option for your needs.

Table 1 : Communicating with Web Services on the Same Network

Goal	Web service to use	How to implement
Track a piece of data, such as a hardware measurement.	Tags Service	*Transferring Data Using Tags*
Send commands, status updates, and data between web applications and servers.	Messages Service	*Sending Messages Between Systems*
Share waveform data stored in a TDMS file.	TDM Reader API Service	*Reading Measurement Data from TDMS Files*
Host a WebVI to make it accessible from web browsers.	NI Web Server	*Hosting a Web Application on the NI Web Server*
Create a custom HTTP REST API to interface with an existing system, such as a database.	LabVIEW Web Service	*Creating and Accessing a Web Service* **NOTE** Download the latest LabVIEW Help for the most up-to-date content. *Configuring CORS for a Web Service*

Table 1 : Communicating with Web Services on the Same Network (Continued)

Goal	Web service to use	How to implement
		Hosting a Web Application on the LabVIEW Application Web Server
Stream data with low latency.	3rd party WebSocket service **NOTE** G Web Development Software does not support listening for WebSocket connections at this time.	*WebSocket G APIs*

Table 2 : Communicating with Cloud Web Services

Goal	Web service to use	How to implement
Share tags and messages securely over the internet.	SystemLink Cloud Tags and Messages	*Connecting to SystemLink Cloud from G Web Development Software* *Sharing Data Across Systems*
Host a WebVI securely on the internet.	SystemLink Cloud Visualizations	*Hosting a Web Application on SystemLink Cloud*
Make local tags and messages available securely on the internet.	SystemLink Cloud Connector	*Connecting to SystemLink Cloud*

RELATED INFORMATION

Retrieving Data From a Web Service on page 127

Considerations When Accessing Data from Web Services on page 130

If you want to create a web application that sends requests to and receives responses from a web service, you need to know the origin of the web service that hosts the web application as well as the origin of the target web service.

Hosting a Web Application on a Server on page 98

To make your web application available to other users, host your build output on a web server that is accessible to other users.

Hosting a WebVI in SystemLink

Host a WebVI in SystemLink to securely share it with users on the server.

Before you begin

Before hosting a WebVI in SystemLink, *create a package (.nipkg) in G Web Development Software* that contains the WebVI.

Procedure

1. In SystemLink Web Application, click **Custom Applications**»**WebVIs**.
2. Click **Import**.
3. Select the package you want to upload from your local machine.

   The WebVI you import must be 100 MB or smaller.
4. Enter a name and optional description for the WebVI.
5. Select the workspace you want to host the WebVI in.
6. Click **OK**.
7. Click the WebVI to run it.
8. To update an existing WebVI, select the WebVI and click ↑ to upload the updated file.

Results

Users in the workspace you selected who have WebVI permissions can interact with the WebVI you uploaded.

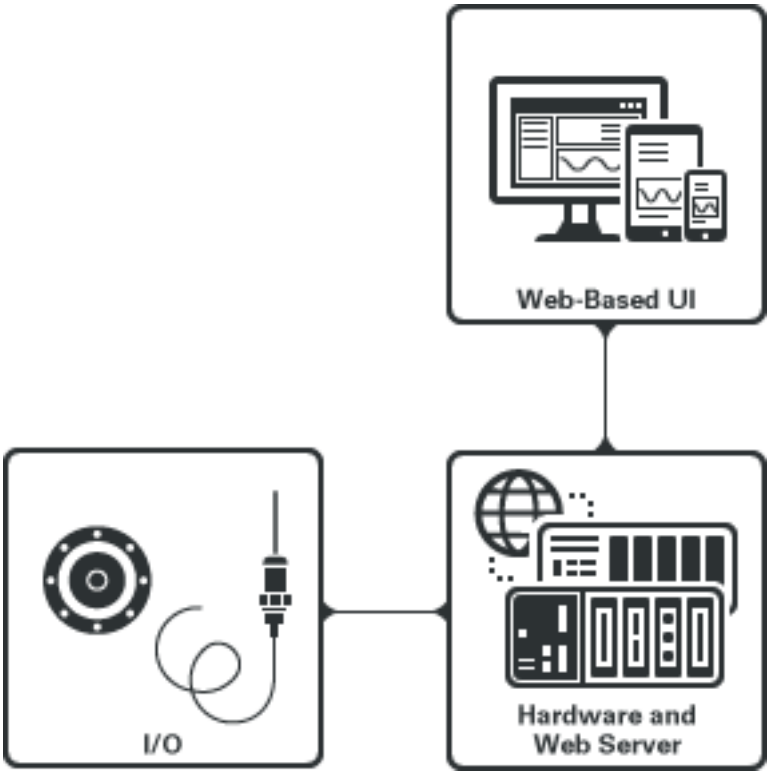
Using Hardware with a Web Application

Use a web application to share measurement data and interact with hardware.

The following diagrams show three architectures for how to use your hardware with a web application. Use the diagrams to choose the best architecture for your application.

NOTE
You can use both real-time and non-real-time hardware devices with your web application.

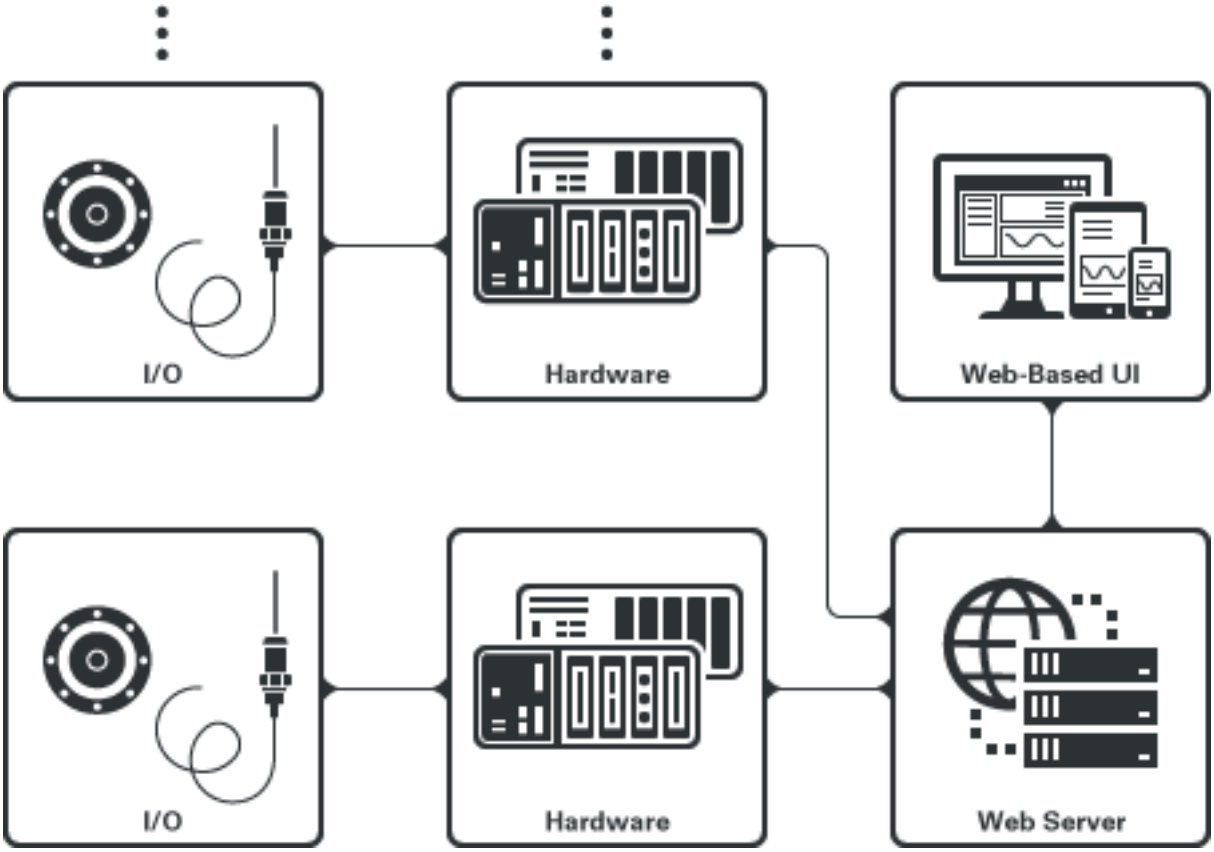
Access a Web Application on Your Hardware



In this architecture, the web service runs on the hardware itself and the hardware also stores the web application. This architecture works best for light-weight web applications, such as browser-based simple configurations and human-machine interfaces for your hardware.

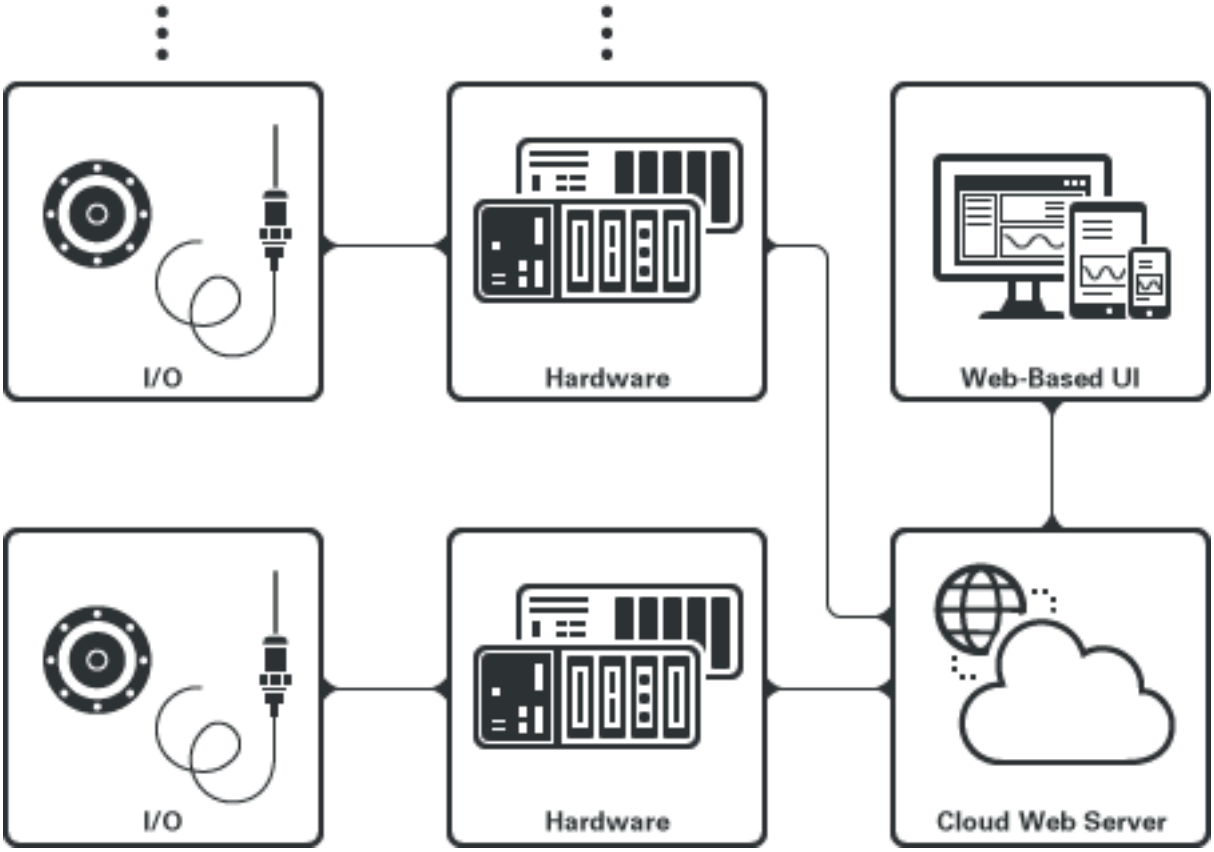
NOTE
You cannot use SystemLink APIs using this architecture.

Access a Web Application on a PC



In this architecture, the web service runs and web application is stored on a PC on the same network as your hardware. This architecture keeps the server burden off of your hardware and works best for sharing measurement data dashboards and applications using SystemLink Tag and Message APIs.

Access a Web Application on the Cloud



In this architecture, your hardware shares data with the web application hosted in the cloud. This architecture works best for accessing the web application from a desktop or mobile device over the internet to securely share your measurement data.

RELATED INFORMATION

Using a Web Application with a CompactRIO Device

Leverage the following architectures to use your web application with a CompactRIO device.

- Access a web application on your CompactRIO.
- Access a web application on a PC.
- Access a web application on the cloud.

Use the following table to determine the best architecture for your system and web application.

Architecture	Use case	How to access	How to implement

Access a web application on your CompactRIO. **NOTE** You cannot use SystemLink APIs using this architecture.	Light-weight web applications: • Browser-based simple configurations • Human-machine interfaces	Enter the IP address of your CompactRIO in a browser on the same local network as the device.	• *Create and Access a LabVIEW Web Service* on the CompactRIO. • *Host using the LabVIEW Application Web Server*. • *Configure cross-origin resource sharing (CORS)* for your LabVIEW Web Service running on a CompactRIO while developing the web application in the LabVIEW NXG editor. **NOTE** NI recommends disabling CORS access to the LabVIEW Web Service when you complete development and deploy the web application to the CompactRIO. For more information about how to protect your web application and data, refer to *Security in NI Web Technology*.

Access a web application on a PC.	• Measurement data dashboards • Use SystemLink Tag and Message APIs • Avoid putting the server burden on your CompactRIO	Enter the IP address of the PC in a browser on the same local network as the PC and CompactRIO.	• Access the data from your CompactRIO device using one of the following options: ◦ *Use NI Web Server Data Services*, such as SystemLink Tag and Message APIs. ◦ *Create and Access a LabVIEW Web Service* on the PC. • *Host using the NI Web Server* or a third-party web server. • *Configure cross-origin resource sharing (CORS)* if your development PC and hosting PC are different machines.

Access a web application on the cloud.	• Use a desktop or mobile device to access the web application over the internet • Host and securely share your measurement data with SystemLink Cloud	Enter the URL for the web application in a browser connected to the internet.	• Use SystemLink Cloud to share data across systems by directly connecting your hardware or make local tags and messages available using SystemLink Cloud Connector API. • ***Host using SystemLink Cloud Server*** to take advantage of the accessibility and security of SystemLink. • ***Connect to SystemLink Cloud using an API key***. **NOTE** Refer to ***SystemLink Cloud FAQ*** to learn more about SystemLink Cloud requirements and features.

Converting LabVIEW NXG Web Module Projects to G Web Development Software Projects

Convert your LabVIEW NXG Web Module project (`.lvproject`) to a G Web Development Software project (`.gwebproject`).

Before you begin

Before you begin, ensure that all files are unlocked and sourced within the project you want to convert. If your files are locked or cannot be found, your project will not convert.

Procedure

1. Launch G Web Development Software.
2. Click **File**»**Open Project**»**Browse**.
3. In the **Open Project** dialog box, filter the available files by selecting **LabVIEW NXG Project** in the file type pull-down.
4. Select the project you want to convert and click **Open**.
5. Click **Continue** when prompted.

   NOTE
   G Web Development Software removes unsupported files to successfully complete conversion. The conversion report lists any changes made to your code.

6. Click **Close** and your converted project automatically opens.

RELATED INFORMATION

Differences between LabVIEW and G Web Development Software

Learn the differences between LabVIEW and G Web Development Software so that you can identify the areas in your application that may change during conversion.

The following table lists terminology differences between LabVIEW and G Web Development Software.

NOTE
This table is not an exhaustive list of all terminology differences between LabVIEW and G Web Development Software.

LabVIEW	G Web Development Software
Application Directory	*Relative Paths*
Unbundle By Name/Bundle By Name (Clusters)	*Cluster Properties*
CTL (Strict Typedef)	*Type Definitions*
Formula Node	No equivalent behavior.
Global Variable	No equivalent behavior.
LabVIEW Class	No equivalent behavior.
Local Variable	*Duplicate Terminal*
Get Waveform Components	*Waveform Properties*
VI Server	*Panel Manipulation*

The following table lists behaviors in G Web Development Software that differ from LabVIEW.

NOTE
This table is not an exhaustive list of all behavioral differences between LabVIEW and G Web Development Software for which you may need to modify your application or settings when migrating code.

LabVIEW	G Web Development Software
Build Specification - Installer	*Package or Package Installer*
Call Library Function Node	*JavaScript Library Interface*
CTL (Control Definition)	No equivalent behavior.
CTL (Typedef)	*Type Definition*
Creating Actors and Messages with Actor Framework	No equivalent behavior.
DataSocket API	No equivalent behavior. See *Communicating Data with a Web Application* for more information about other networking APIs.
Express VIs	SubVIs
Flattening and Unflattening to Binary	Use To Variant and Variant to Data VIs.

LabVIEW	G Web Development Software
Invoke Nodes	Invoke nodes are replaced with VIs that have corresponding behavior.
MathScript	No equivalent behavior.
Merge Errors	*Retain First Error* behaves similarly. Consider using *Multiplex Errors* for handling multiple errors on the same wire.
.NET Constructor Nodes and Invoke Nodes	No equivalent behavior. See *JavaScript Library Interface* for more information about calling JavaScript libraries.
Packed Project Libraries	No equivalent behavior.
Polymorphic VIs	An overload VI or a multi-mode element.
Sub-panels	No equivalent behavior.
String Encoding	The string data type is *UTF-8*. **NOTE** Casting other data types, such as binary, to a string may cause unexpected behavior when displaying or manipulating the string.
Tree Control	*Tree Control*
VI Package Manager	Use *NI Package Manager* for distributing and updating both source code and deployed applications.
XControls	No equivalent behavior.

The following table lists common keyboard shortcuts that differ between LabVIEW and G Web Development Software.

NOTE
This table is not an exhaustive list of keyboard shortcuts that differ between LabVIEW and G Web Development Software.

Command	LabVIEW	G Web Development Software
Create branch from wire	Click near a wire	Hold Ctrl to change to the wiring cursor while hovering over a wire (Default preference setting, which you can change.)
Create constants for all unwired inputs	<Ctrl-Space> then <Ctrl-Shift-D>	<Ctrl-M>
Create constants for required unwired inputs	No equivalent behavior.	<Ctrl-Shift-M>

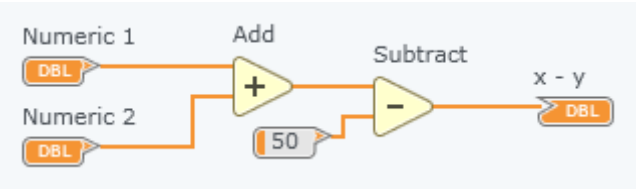
Command	LabVIEW	G Web Development Software
Go through case/event structures on diagram	<Ctrl-Scroll Wheel>	<Ctrl-Shift-Scroll Wheel>
Insert on wire	<Ctrl-Space> then <Ctrl-I>	Right-click on wire and select or search for new VI. Error wires automatically connected if applicable.
Pan on canvas	<Ctrl-Shift-Drag>	<Space-Drag>
Remove wire and rewire	<Ctrl-Space> then <Ctrl-R>	<Ctrl-Del>
Replace object	<Ctrl-Space> then <Ctrl-P>	Right-click on VI and select a new VI.
Show Panel and Diagram in separate windows	Default	Right-click on Panel/Diagram switcher or document tab and select **Float in new window**.
Show/hide panes	<Ctrl-\>	<Ctrl-T>
Wire objects	<Ctrl-Space> then <Ctrl-W>	<Ctrl-J>

Programming in G

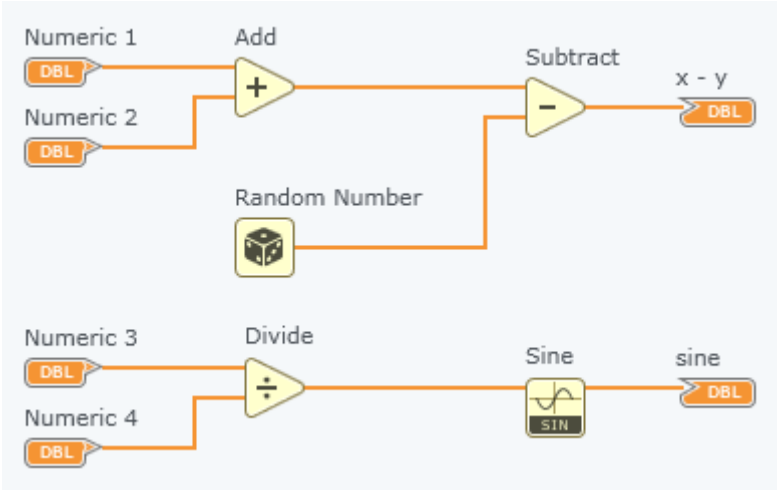
G Dataflow (G) is a graphical programming language in which nodes on a diagram execute when data is available for all required inputs.

A node in G Dataflow executes only when it receives data for all wired inputs. When a node finishes executing, it passes data along the wire to the next node in the dataflow path. The movement of data along the wires and through the nodes is called dataflow and determines the execution order of nodes on the diagram.

Because a node executes only when all of the inputs receive data, the `Subtract` node in the following G diagram cannot execute until Add finishes executing and passes the data to `Subtract`.



However, in the following G diagram, the Add, `Random Number`, and `Divide` nodes all meet the dataflow requirement of having the required input data they need to execute. Nodes not connected to each other by wires can execute in any order because the nodes do not depend on data from another node. You cannot determine which node executes first.



Because the flow of data, rather than the sequential order of commands, determines the execution order of nodes in G, you can create diagrams that have simultaneous operations. The image above illustrates operations that run simultaneously.

Nodes: Computational Units

Nodes are objects on the diagram that accept inputs, return outputs, and perform operations when a program runs.

They are analogous to statements, operators, functions, and subroutines in text-based programming languages.

Wires: Transferring Data between Nodes

Wires transfer data between nodes on a diagram by connecting the output terminal of one node to one or more input terminals of another node.

Wires have the following properties:

- A wire has a single data source.
- A wire can carry data from its single data source to more than one node that reads the data as an input.
- The color, style, and thickness of a wire represent the type of data passing between nodes.

Troubleshooting Broken Wires

A broken wire indicates that dataflow cannot run across the wire. You cannot run code that contains broken wires. Resolve each wire to make dataflow valid and run the code.

About this task

Broken wires look like the following image.

Make sure your code meets the following requirements to resolve each wire:

Procedure

- Each wire has a single data source.
- There are no unwired or overlapping tunnels when wiring through structures.
- All wires are connected to a node.
- Every data source is wired to at least one output.

  For example, you cannot wire two indicators together.
- The output and input of the same node are not wired together.
- Wire compatible data types together.

  For example, you cannot wire a Boolean output to a string input.
- Triple-click the wire to select the entire wire.

  You might see hidden wire segments that help you identify one of the causes listed above.
- Use the error list to find broken wires and an explanation.
- If you still cannot identify the cause of the broken wire, click **Remove Broken Wires** on the Document toolbar.

> **NOTE**
> **Remove Broken Wires** deletes all broken wires, even those you might not see.

RELATED INFORMATION

Wiring Best Practices

Poor wire organization might not produce errors, but it can make the diagram difficult to read and debug or make the code appear to do things it does not do. Use these best practices as you develop your diagrams.

- Use a left-to-right and top-to-bottom layout. Although the positions of diagram elements do not determine execution order, wiring from left to right keeps the diagram organized and easy to understand. Only wires and structures determine execution order.
- Control terminal wires should exit the right side of the terminal, and indicator terminal wires should enter on the left side of the terminal.
- Wire around objects. Do not cover wires with other objects.
- Use as few bends as possible.

RELATED INFORMATION

Wiring Shortcuts

You can use the following set of shortcuts to help you create wires more efficiently.

Editor Command	Shortcut	Action
Remove Broken Wires	Ctrl-B	Delete all broken wires from the diagram.
—	Esc Right-click Ctrl-Z	Delete a wire you are in the process of creating.
—	Single-click wire	Select one wire segment.
—	Double-click wire	Select a wire branch.
—	Triple-click wire	Select the entire wire.
—	Ctrl-click wire	Create a new wire branch from an existing wire.
—	Single-click while wiring	Tack down the wire segment and start a new wire segment.
—	Double-click while wiring	End the wire without connecting it to a node.
—	Tap spacebar while wiring	Switch the direction of a wire between horizontal and vertical.
Clean Up Diagram Clean Up Selection	Ctrl-U	Organize the diagram or the selected code to make it easier to understand.

Clean Up Wire	Select **Clean Up Wire** from the shortcut menu	Route a selected wire to decrease the number of bends in the wire and avoid crossing objects on the diagram.

Constants

A constant is a fixed piece of data that exists only on the diagram. Use a constant instead of a control when you know that a certain piece of data should always be the same regardless of other parts of your code.

You can use the following types of constants in graphical programming:

Universal constant —Represents commonly used values that are always the same, such as pi (π), the speed of light, or the character that represents pressing the Enter key.

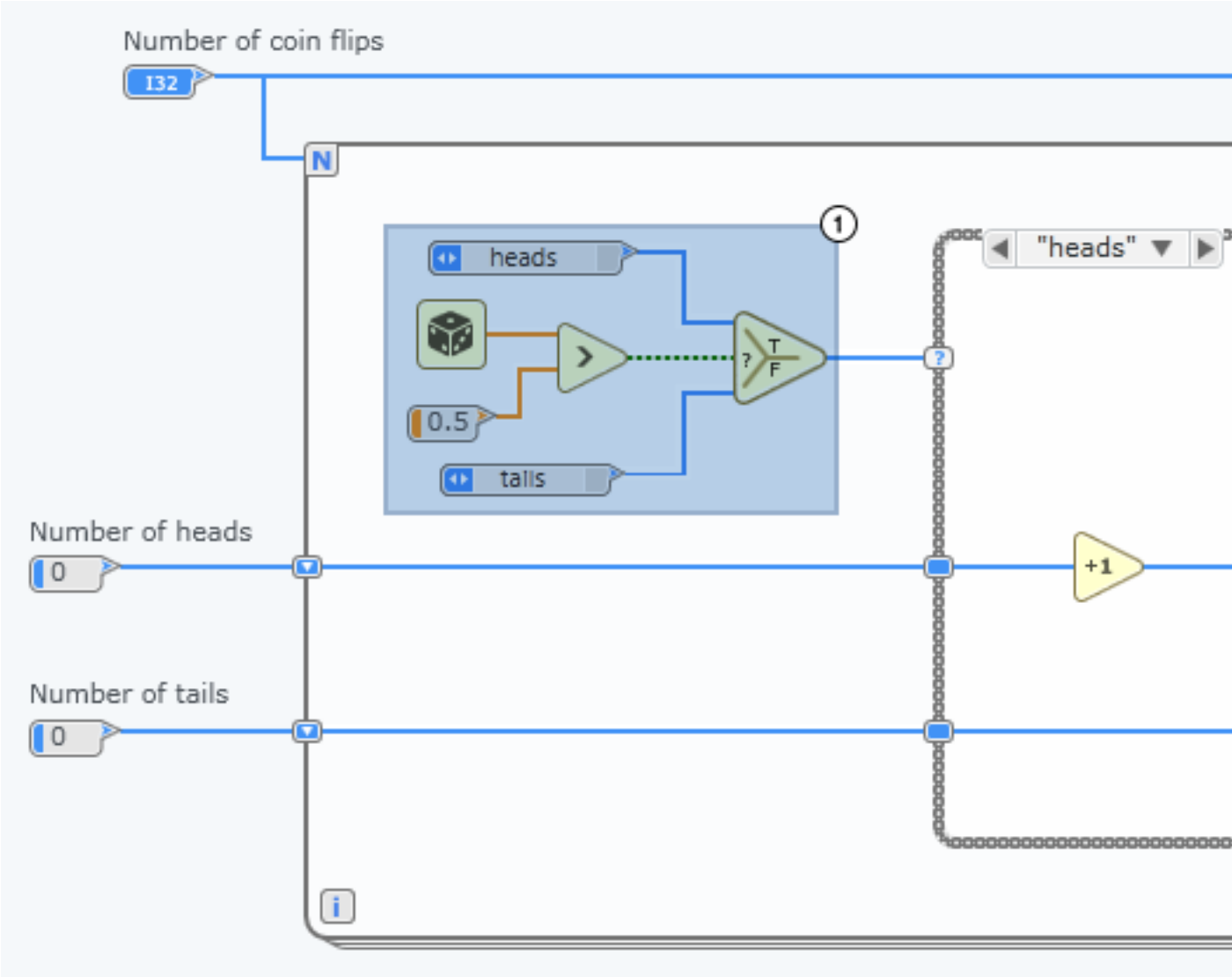
User-defined constant —Accepts user input. You define the values as you develop the code.

SubVIs

After you build a VI, you can call it from the diagram of another VI within the same project. A VI called from the diagram of another VI is a *subVI*. SubVIs contain reusable code and simplify the diagrams of calling VIs. SubVIs are analogous to functions or subroutines in other programming languages.

When you add an instance of a subVI to the diagram of another VI, the subVI appears as a single node, similar to a node from the palette. When data in the calling VI reaches the subVI node, the software executes the diagram of the subVI.

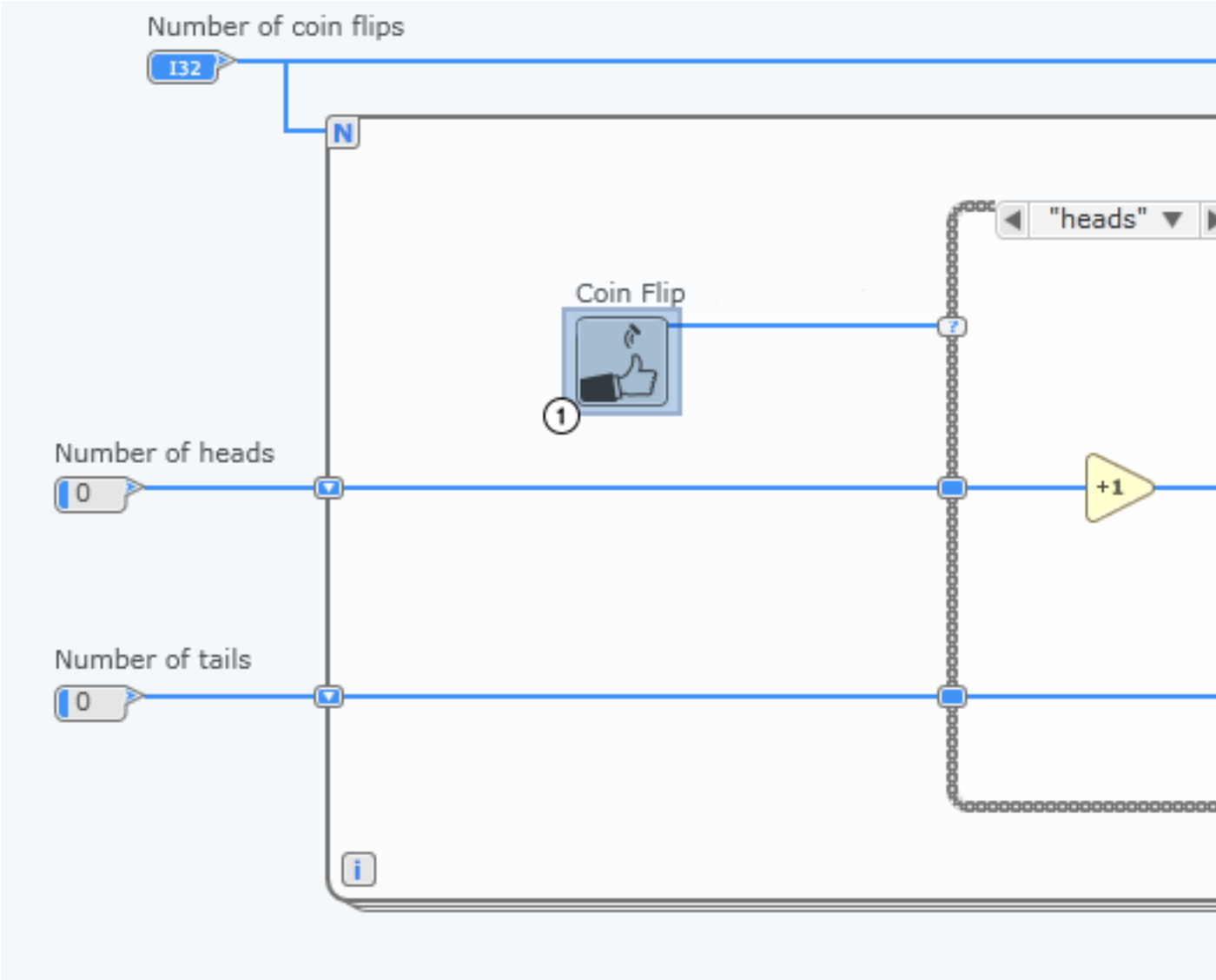
The following diagram shows two common situations that benefit from subVIs.



1. Users may have difficulty understanding the purpose of this section of code because it performs several operations. The more complex a diagram, the more time it takes to interpret sections of code.

2. The same code appears in two different places. Repeated code increases the likelihood of errors. For example, if you change one section of repeated code, you must remember to make the same change everywhere the code section occurs.

The following diagram shows two solutions that subVIs provide.



1. The Coin Flip subVI represents the highlighted code inside the For Loop of the previous diagram. The subVI simplifies the diagram and communicates the purpose of the code through its label and node icon.

2. The Percentage subVI represents one of the duplicate sections of highlighted code from the previous diagram. The Percentage subVI performs the same function in two places without repeating code. If you change code in the VI called by the Percentage subVI, both instances of the subVI change.

RELATED INFORMATION

Creating a SubVI from a Section of Existing Code

Procedure

1. Select the section of code you want to reuse.

2. Click **Edit**»**Create subVI from selection** to generate a new `.gvi` file in the project.

   To open this file, double-click the corresponding subVI node that contains the section of code you selected. The subVI diagram and connector pane include inputs and outputs for every wire that enters and exits the section of code. The panel automatically includes the corresponding controls and indicators.

   TIP
   To disable automatically placing items on the panel, click **File**»**Preferences**, select the **VI** tab, and disable **Place items on the panel when creating subVI from selection**.

3. Rename and save the new `.gvi` file.

   Choose a name that describes what function the VI performs so that other users can clearly understand any diagram that uses the VI as a subVI.

What to do next

To add the new subVI as a node on other diagrams, locate the subVI in the **Project Files** tab and drag it to the diagram where you want to use it.

NOTE
All instances of the subVI are automatically updated with most changes you make. However, if you remove or rearrange inputs and outputs on a subVI, wires attached to those inputs and outputs break in VIs that call the subVI. Fix the calling VIs by deleting the broken wires and rewiring the updated inputs and outputs.

VI Reentrancy

You can choose from three VI reentrancy options that determine how your subVI executes when it is called at more than one point in the dataflow at the same time. Reentrancy is the simultaneous execution of multiple calls for a subVI.

By default, each call to the subVI is executed one after another. This may cause your application to execute slowly or have unexpected behavior. To allow multiple calls to a subVI to execute simultaneously, set a reentrant execution state for the subVI.

Refer to the following table to determine which VI reentrancy option is best for your subVI.

What your subVI needs	Option(s) to use
Ability to maintain state	**Stateful** maintains state for each instance of the subVI. **None** maintains a single state across all instances of the subVI.
Minimize wait time when multiple VIs call your subVI at the same time	**Stateful** eliminates wait time completely. **Stateless** results in wait time if there are more calls to the subVI than there are clones. However, the wait time is always within a certain range.

What your subVI needs	Option(s) to use
Determinism, or the same wait time for each call to the subVI	**Stateful** guarantees the same exact wait time for each call to the subVI. This minimizes jitter in applications involving time-sensitive data.
Minimize memory usage	**None** uses the least memory. **Stateless** uses less memory than **Stateful** if your subVI needs the other features of reentrancy.
Minimize call overhead	**Stateful** is the most efficient for calling a subVI many times, such as within a loop.

RELATED INFORMATION

Choosing the Right VI Reentrancy Option for a SubVI

Procedure

1. On the **Document** tab, in the **Behavior** section, click **Properties** to access VI Reentrancy options for your subVI.
2. In the resulting dialog box, select the **Execution** tab.
3. Choose one of the following options:

Option	Description
None	Allocates a single data space for use by all instances of the subVI. Therefore, the software executes multiple, simultaneous calls to a subVI one at a time in an indeterminate order. Because every instance of the subVI shares the same data space, all calls to the subVI also share a single state, which preserves the values of controls and uninitialized shift registers among calls.
Stateless	Allocates one copy, or clone, of the subVI for each processor on your machine. Each call to the subVI can then access one of these shared clones instead of waiting on the main subVI. If there are not enough shared clones for each call to the subVI, additional clones are allocated on demand. Each instance of the subVI may access a different clone each time it is called.
Stateful	Allocates a separate clone for each instance of the subVI. Each instance of the subVI stores data in its own pre-allocated clone.

RELATED INFORMATION

Minimizing SubVI Overhead

Inlining a subVI into its calling VIs helps minimize the overhead associated with calling the subVI.

Before you begin

When you inline a subVI, G Web Development Software inserts the compiled code of the subVI into the compiled code of the calling VI. If you then make changes to the subVI, G Web Development Software recompiles all calling VIs of that subVI to include those changes. Essentially, inlining a subVI removes the need to call the subVI at run time. Instead, the subVI code executes inside the compiled code of the calling VI.

A subVI that you want to inline cannot have any of the following characteristics:

- Contains recursion
- Contains certain diagram nodes, such as control references. These nodes generate edit-time error messages if you enable subVI inlining.

About this task

Complete the following steps to inline a subVI.

Procedure

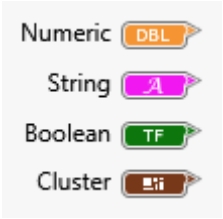
1. On the **Document** tab, in the **Behavior** section, click **Properties** to access the property settings for the subVI.
2. Place a checkmark in the **Inline subVI into calling VIs** checkbox.
3. Set **VI Reentrancy** to **Stateless** or **Stateful** to configure reentrant execution.
4. Click **OK**.

RELATED INFORMATION

Terminals

Terminals transfer data between the diagram and panel, between the diagram and other nodes, or between duplicates of the same terminal on the diagram.

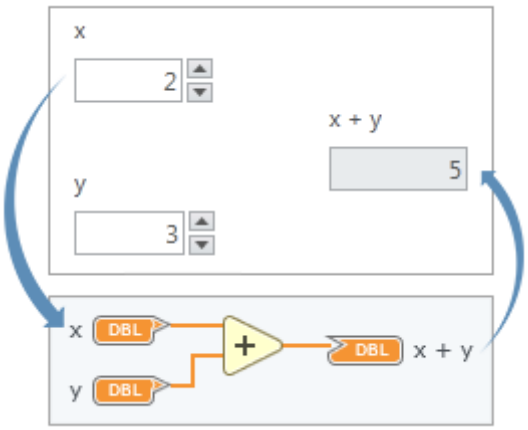
As the following example shows, the color and symbol of each terminal indicate the data type of the terminal.



Data Transfer between the Panel and the Diagram

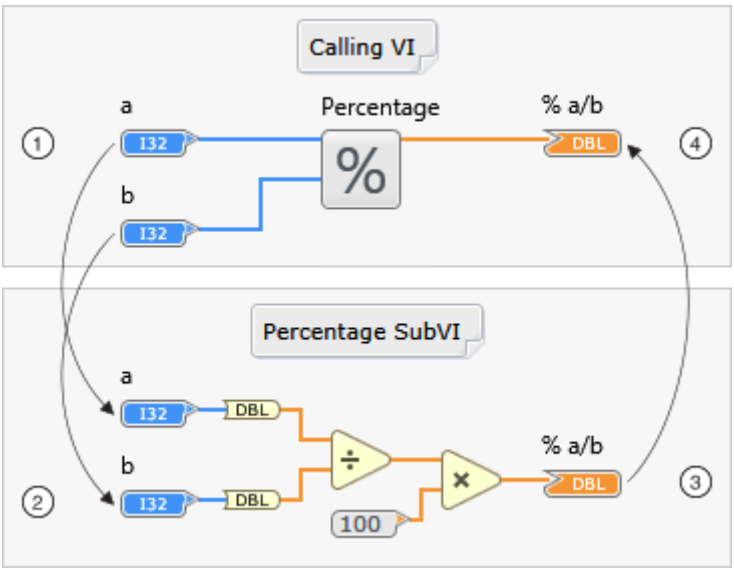
Data in a control flows from the panel to the diagram through a corresponding input terminal. Data in an output terminal flows from the diagram to the panel through a corresponding indicator.

The following image shows the flow of data between the panel and diagram of a VI.



Dataflow between the Diagram and Another VI

Terminals transfer data between the diagram and other VIs via subVIs. The following image shows the flow of data between terminals of the calling VI and a subVI.



1. The calling VI passes data to the Percentage subVI through the input terminals of the subVI node.
2. Data from the calling VI flows through the input terminals on the diagram of the subVI.
3. As the subVI executes, data flows to the output terminal of the subVI diagram.
4. Data flows from the output terminal of the subVI diagram to the output terminal of the subVI node in the calling VI.

Dataflow between Duplicates of the Same Terminal

You can use a duplicated output terminal to write to an indicator at multiple places on the diagram. You can also use a duplicated input terminal to read from a control at multiple places on the diagram.

NOTE
Terminals are duplicates of one another if they have the exact same label. You can duplicate a terminal by right-clicking the terminal and selecting **Create Duplicate Terminal**. Copying and pasting a terminal does not create a duplicated terminal.

In the following example, the **String** indicator displays a different message depending on which `While Loop` is executing. This example also uses a duplicate of the **Run** button to control the execution of both `While Loops`.

Figure 3 : Panel

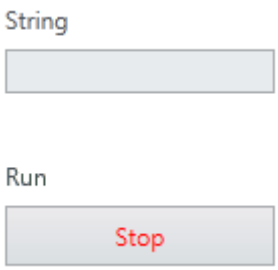
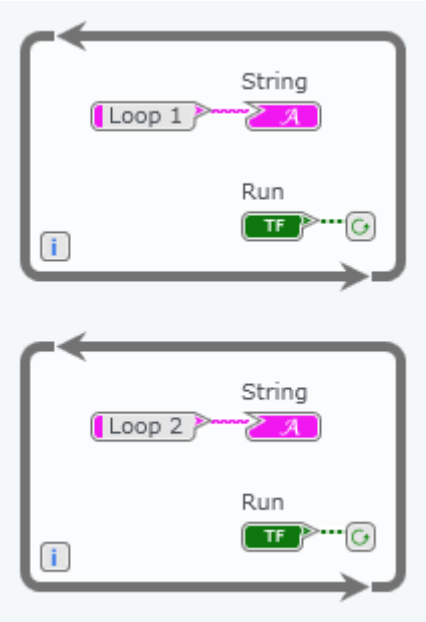


Figure 4 : Diagram



RELATED INFORMATION

Terminals on page 164

Dataflow between the Diagram and Another VI on page 165

Data Transfer between the Panel and the Diagram on page 165

Opening, Processing, and Closing Files

What to Use

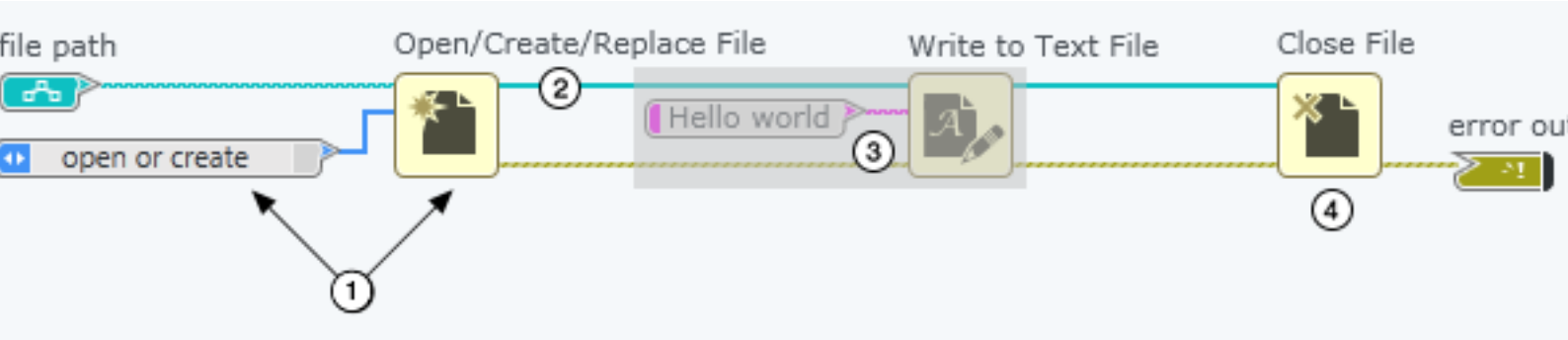
- **Open/Create/Replace File**
- A read or write node from the **Storage** palette category

- **Close File**

What to Do

Create the following diagram to programmatically open, write to, and close a text file.

Customize the gray sections for your unique programming goals.



①	Open, create, or replace a file by using the `Open/Create/Replace File` node and specifying its precise behavior. To access the enumerated list of possible behaviors, create a constant from the **operation** input of the node.
②	After the file opens, a unique identifier called a refnum represents the file. A file refnum is required for most nodes that process files.
③	Process the file by using a file I/O node from the `Storage` palette category. You can interact with binary or text files. In this example, `Write to Text File` writes `Hello world` to the file represented by the file refnum.
④	Close the file to release the object from memory and avoid potential errors. If you do not close a file, the reference cannot be closed until the VI that opened the file finishes executing.

Troubleshooting

- Some file I/O nodes are considered high-level nodes and perform all three steps of a file I/O process—open, read/write, and close. Avoid placing high-level file I/O nodes, such as `Read from Spreadsheet File` and `Write to Spreadsheet File`, in loops because these nodes perform open and close operations each time they run.

Strategies for Improving VI Execution Speed

Factors like inefficient memory use, poorly designed panels, and too many I/O calls can negatively affect the execution speed of your VI. However, there are several strategies you can use to ensure that your VI runs more efficiently.

RELATED INFORMATION

Strategies for Improving Execution Speed through Memory Management

Effective memory management can improve the execution speed of your VI. By minimizing memory usage, you can help alleviate slow VI execution speeds, which are affected by allocation and deallocation operations. Execution speed is indirectly affected by the allocation of more space in memory because operations slow down if they reach the memory's capacity.

The more you allocate, deallocate, or move data in memory, the more memory the VI uses. Memory can also indirectly affect execution speed because allocating more space in memory leaves less memory for other operations, causing them to slow down if they reach the memory's capacity.

When creating a VI, consider the following guidelines:

- Use consistent data types and watch for coercion dots when passing data to subVIs.
- Avoid using hierarchical data types such as arrays of clusters containing large arrays or strings.
- Reduce the use of nested subVIs inside of other VIs, as these can cause an exponential increase in the number of VI clones needed to maintain separate data spaces for each call to a subVI.

Avoid the following, which can create additional and unnecessary copies of data:

- Repeatedly resizing data—Clearing space in memory or adding and removing data from memory creates overhead and can take time.
- Overusing duplicated array and string terminals—Creating more than one corresponding object on the diagram can increase memory usage.
- Unintentionally coercing data—Coercion dots appear on nodes when data types are coerced. Data type coercion can decrease execution speed and increase memory usage, especially when you use complex data types or large arrays because they can increase memory usage.
- Displaying large arrays and strings on panels—Displaying the smallest array or string on the panel will ensure that copies of data remain small. The larger the array or string, the larger the copy of contained data.

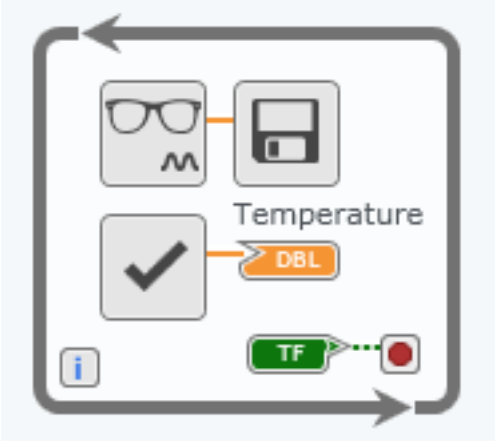
Guidelines for Designing Efficient Panels

Optimizing your panel can improve the execution speed of your VI. Design and configure your panel according to the following guidelines:

- Remove unnecessary panel objects and avoid adding objects to the panel that aren't necessary for controlling input to the diagram or observing output data.
- Avoid transparent and overlapped panel objects unless necessary.
- When using charts, reducing Chart History Length in the Configuration pane will set the maximum number of data points drawn to your chart.
- When using graphs and charts, turn off auto-scaling, axis labels, anti-aliased line drawing, and axis grids to prevent drawing more elements. A graph with fewer elements takes less time to redraw every time the graph updates.

Using Parallel Loops to Increase VI Execution Speed

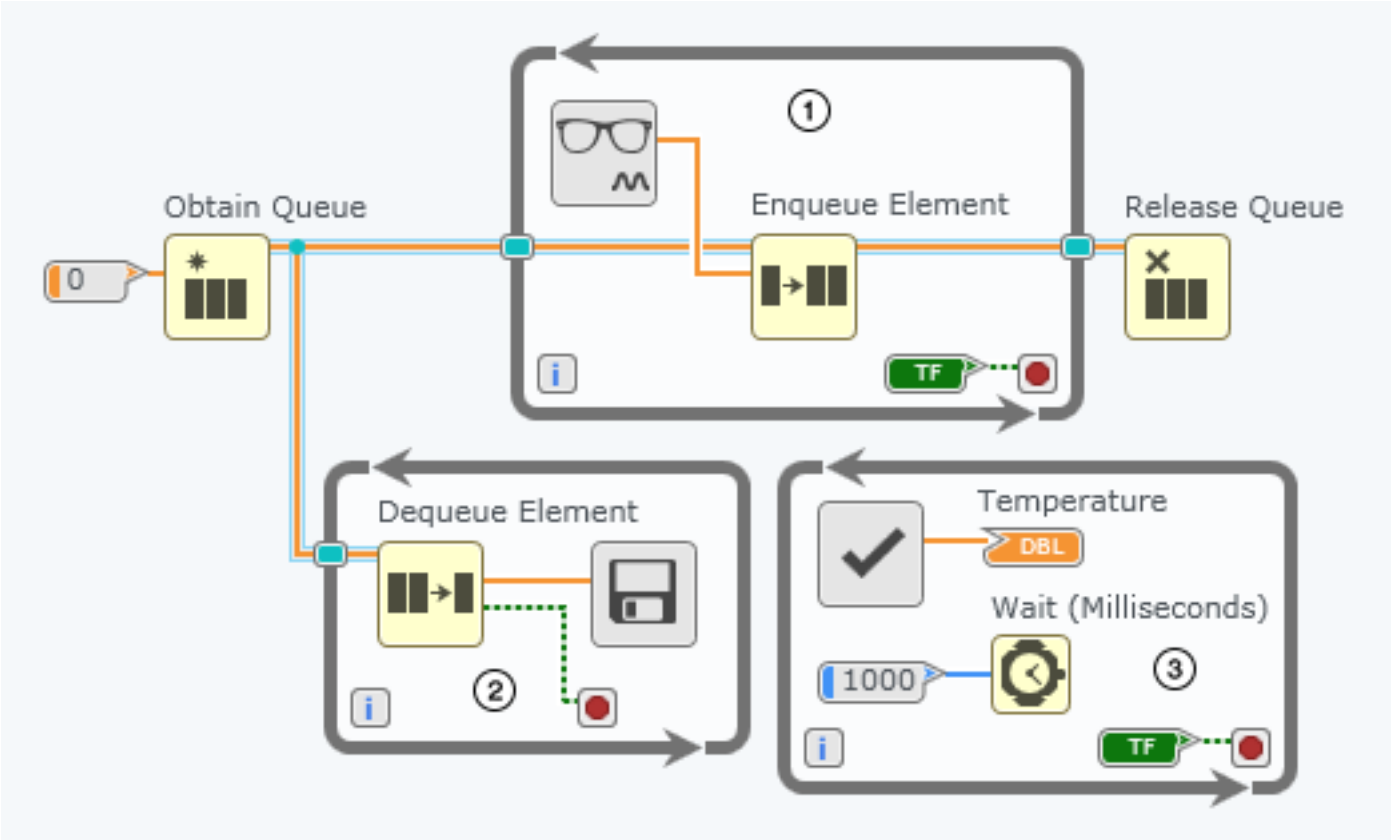
When multiple loops run in parallel, the VI switches between them periodically. Parallel loops can help decrease execution time by simultaneously running independent tasks within individual loops. In the following example diagram, a single loop contains multiple independent tasks. This loop is inefficient because execution speeds can vary due to the lack of loop timing. In this example, the DAQ operation must wait for the slower I/O function and a status check to complete each iteration.



Since all parallel loops share the same processor resources, consider the following practices to make sure other loops do not interrupt the timing or execution of your important tasks:

- Important Operations—Because you want this type of operation to execute as frequently as possible, do not add timing nodes inside the loop to delay its execution.
- Executions Dependent on Dequeue Element node—The Dequeue Element node can halt the execution of a loop until an element is ready in the queue. This node allows more important loops to use more of the processor's time.
- Low-Importance Operations—Because this type of operation is the least important, you can use a Wait node to run the operation less often and yield execution time to the other loops.

The following is an example of a recreated VI using the considerations:



1. The Read DAQ loop reads data from a DAQ device and sends the data to a queue. Do not add timing nodes because you want this type of operation to execute as frequently as possible.

2. The Write to File loop can execute more slowly as it is logging data points for later analysis. The execution of this loop can be dependent on elements being ready in the queue because it does not need to execute as frequently.

3. The Check Temperature loop displays temperature data to the user and uses a Wait node to run less often and yield execution time to the other loops because it is the least important.

Repeating Operations

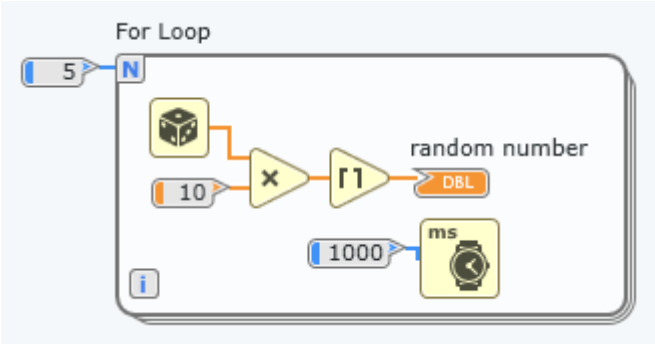
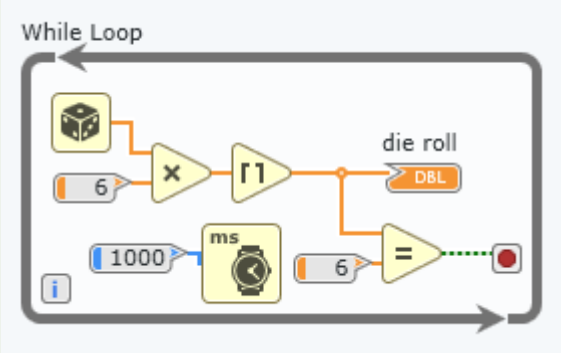
When building an application, you may need to repeat code a set number of times or until a specified condition is met. For example, you may have a section of code that you know you want to run three times. You may want your code to keep running until a user clicks a stop button. You can use loops to do this.

About this task

A *loop* is a programming element that executes the same code multiple times, or iterations.

Types of Loops

You can use the following loops to repeat code in G Dataflow: a `For Loop` and a `While Loop`.

Loop	Behavior	Example	Diagram
`For Loop`	Repeats the code inside the loop for a set number of iterations.	Generate five random numbers between 1 and 10.	
`While Loop`	Repeats the code inside the loop until a specified condition is met.	Continue rolling a die until the value of the die is 6.	

Repeating Operations until a Condition Occurs

About this task

In some situations, you might know that you want to repeat an operation, but you do not know exactly how many times you want to repeat that operation. Instead, you know only that the operation should repeat until a certain condition occurs. For example, you might have a program you want to run repeatedly until a user clicks a stop button or until the code inside the loop produces a particular value.

Use a `While Loop` to repeat code until a specified condition is met. A `While Loop` behaves similarly to a do `while` loop in other programming languages.

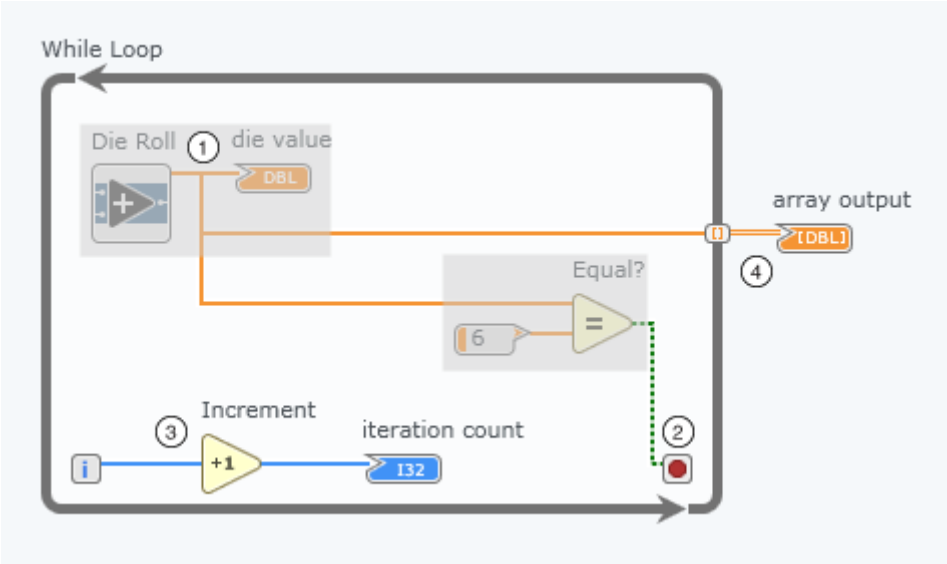
What to Use

`While Loop`

What to Do

Create the following diagram to repeat an operation until a condition occurs.

Customize the gray sections for your unique programming goals.



①	Place the code you want to repeat on the subdiagram of a `While Loop`.
②	To specify the condition under which the loop stops, create code that produces a True Boolean value when the desired stop condition occurs and wire that Boolean value to the condition terminal. By default, the condition terminal is configured to **Stop if True**. To stop the loop for a False Boolean value instead of a True value, right-click the condition terminal and select **Continue if True**. You can also specify when the loop stops by wiring an error cluster to the condition terminal. In this situation, the Boolean value of the **status** of the error is used to determine when the loop stops.
③	If you want to know how many loop iterations executed before the stop condition occurred, use the iteration terminal to return the current loop iteration count. The iteration terminal is zero-indexed, meaning it ranges from 0 to *n* -1. Use `Increment` to obtain the true number of loop iterations. **NOTE** A `While Loop` always executes at least one time.
④	If you want to collect the result of each loop iteration in an array, use an auto-indexing output tunnel to pass values out of the loop. To enable auto-indexing for an output tunnel, right-click the tunnel and select **Append Mode**»**Auto Index Values**. If you do not enable auto-indexing, the loop returns only the value from the last loop iteration.

Troubleshooting

If the execution speed of the loop is too fast, place a `Wait` node on the subdiagram of the loop to specify a wait period.

Repeating Operations a Set Number of Times

About this task

Instead of creating the same code on a diagram multiple times, you can write code in a single location and use a `For Loop` to programmatically repeat it. For example, you might want to read a specific number of measurement samples from a device. You can place the code that performs the measurement operation on the subdiagram of a `For Loop` and configure the loop to repeat the operation as many times as your application requires.

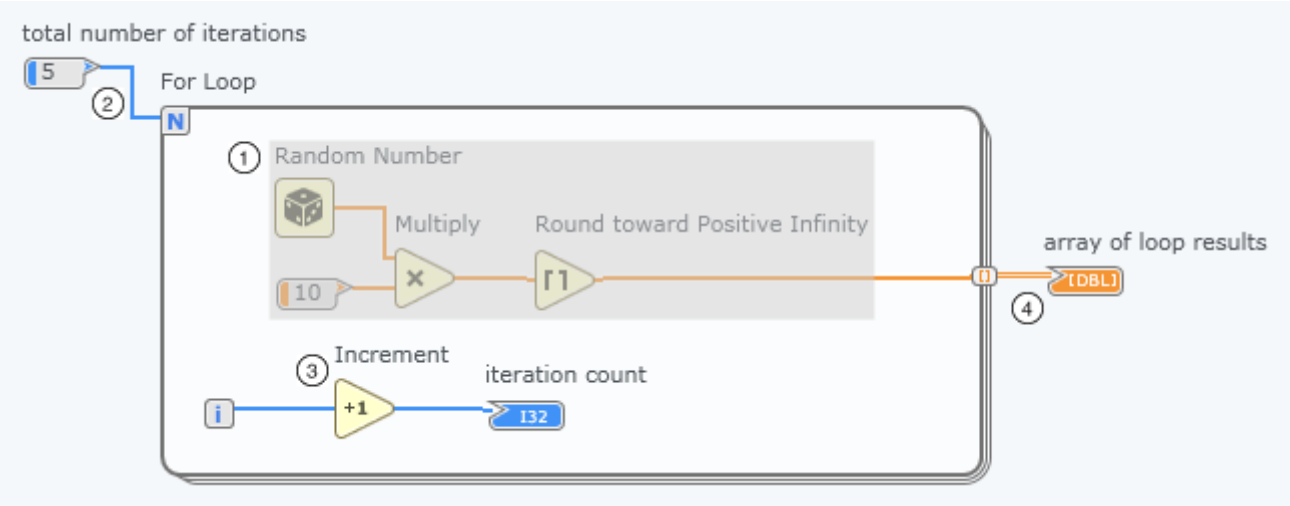
What to Use

- **`For Loop`**
- Count terminal of the **`For Loop`**

What to Do

Create the following diagram to repeat an operation a set number of times.

Customize the gray sections for your unique programming goals.



①	Place the code you want to repeat on the subdiagram of a `For Loop`.

②	Specify how many times the For Loop repeats its subdiagram by wiring the desired value to the count terminal. **TIP** If you want the loop to execute once for each element in an array, wire the array to the loop using an auto-indexing tunnel instead of wiring a value to the count terminal.
③	If you want to know which loop iteration is currently executing, use the iteration terminal to return the current loop iteration count. The iteration terminal is zero-indexed, meaning it ranges from 0 to *n* -1. Use Increment to obtain the true number of loop iterations.
④	If you want to collect the result of each loop iteration in an array, use an auto-indexing output tunnel to pass values out of the loop. To enable auto-indexing for an output tunnel, right-click the tunnel and select **Append Mode**»**Auto Index Values**. If you do not enable auto-indexing, the loop returns only the value from the last loop iteration.

Troubleshooting

- If the For Loop unexpectedly fails to execute, verify that the value wired to the count terminal is greater than zero.
- If the execution speed of the loop is too fast, place a Wait node on the subdiagram of the loop to specify a wait period.

Repeating Operations Once for Every Element in an Array

About this task

While working with an array of data, you might want to access individual elements within the array. Although you can use a combination of Array nodes to accomplish this task, the For Loop includes an auto-indexing tunnel that you can use to access the individual elements with minimal additional code.

Use a For Loop with an auto-indexing input tunnel to process one element of an array during each iteration of the loop. A For Loop with an auto-indexing input tunnel behaves similarly to a for each loop in other programming languages.

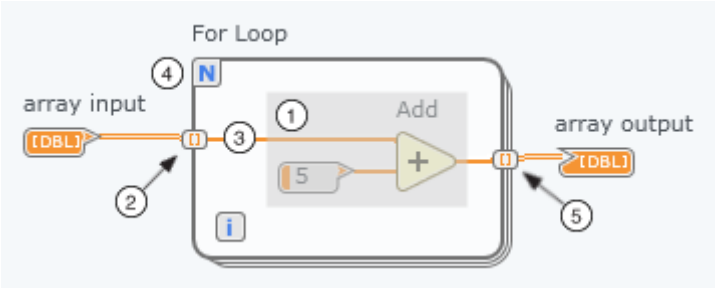
What to Use

For Loop with an auto-indexing input tunnel

What to Do

Create the following diagram to repeat an operation once for every element in an array.

Customize the gray sections for your unique programming goals.



①	Place the code you want to repeat on the subdiagram of a For Loop.
②	When you wire an array to a For Loop, the For Loop processes one element of the array at a time. This configuration, known as auto-indexing, occurs by default. The input tunnel appears as a white box with brackets to indicate that it is an auto-indexing tunnel. **NOTE** A For Loop can process multiple arrays one element at a time using multiple auto-indexing input tunnels. In this situation, the loop uses the smallest array size to determine the number of loop iterations. For example, if two auto-indexed arrays enter the loop with 10 and 20 elements respectively, the loop executes 10 times, processing all elements of the first array but only the first 10 elements of the second array.
③	You can access each individual element of the input array by wiring the auto-indexing tunnel to the code on the subdiagram of the loop. The wire entering the auto-indexing tunnel carries 1D array data, whereas the wire leaving the auto-indexing tunnel carries scalar data. **NOTE** For multidimensional input arrays, the wire leaving the auto-indexing tunnel carries array data that is one dimension smaller than the input array. For example, if the input array terminal passes a 2D array to the auto-indexing tunnel, the wire leaving the auto-indexing tunnel carries 1D array data.
④	By not wiring a value to the count terminal when auto-indexing is enabled, the loop automatically iterates once for each element in the array. **NOTE** Wiring a value to the count terminal while auto-indexing is enabled causes the For Loop to use the smallest of the choices between the count terminal and the input array size to determine the number of loop iterations. For example, if an auto-indexed array enters the loop with 10 elements and you wire a value of 15 to the count terminal, the loop executes 10 times.
⑤	If you want to collect the result of each loop iteration in an array, use an auto-indexing output tunnel to pass values out of the loop. To enable auto-indexing for an output tunnel, right-click the tunnel and select **Append Mode**»**Auto Index Values**. If you do not enable auto-indexing, the loop returns only the value from the last loop iteration.

Troubleshooting

- If the For Loop iterates an unexpected number of times, note that if you enable auto-indexing for more than one input tunnel or if you wire a value to the count terminal, the actual number of loop iterations becomes the smallest of the choices.
- If the For Loop processes the entire input array in a single loop iteration instead of one element per iteration, verify that the input tunnel is auto-indexing the array. An auto-indexing tunnel appears as a white box with brackets, whereas a non-indexing tunnel appears as a solid box.
- If the execution speed of the loop is too fast, place a Wait node on the subdiagram of the loop to specify a wait period.

Synchronizing the Execution of Multiple Loops

About this task

By default, each loop iteration executes as quickly as possible based on the code inside the loop. However, consider a program that includes multiple loops that contain code requiring different amounts of time to execute. You might want to control the rate at which those loops execute in order to synchronize their execution and ensure that the loops begin each iteration at the same time. Use Wait Until Next Multiple to do this.

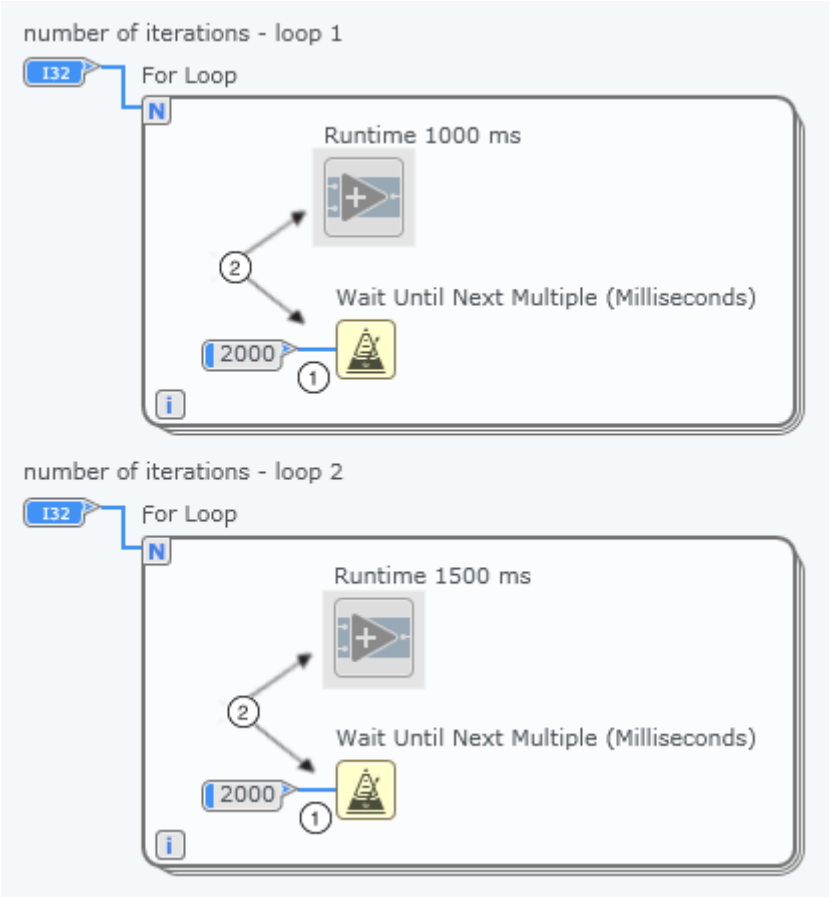
What to Use

- **While Loop** or **For Loop**
- **Wait Until Next Multiple**

What to Do

Create the following diagram to synchronize the execution of multiple loops.

Customize the gray sections for your unique programming goals.



①	Wire the same value to the inputs of the `Wait Until Next Multiple` nodes placed on the subdiagram of each loop. The loops wait until the value of the system clock becomes a multiple of the specified input before beginning each iteration. Therefore, the loops begin each iteration at exactly the same time.
②	When specifying a value for the input of `Wait Until Next Multiple`, ensure that the value is greater than the time required to execute the code inside the loop. If a loop contains code that takes longer to execute than the time specified, `Wait Until Next Multiple` has no effect on the execution speed of the loop.

Accessing Data from the Previous Loop Iteration

About this task

After a loop completes a single iteration, sometimes you want to use the value of one of its calculations in the next loop iteration. You can use a shift register to pass data from the most recent iteration to the next iteration.

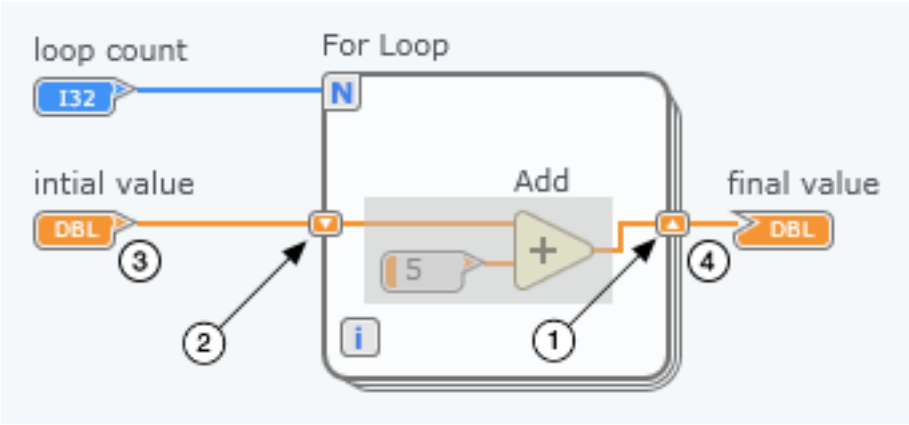
What to Use

- **`For Loop`** or **`While Loop`**
- Shift register

What to Do

Create the following diagram to pass data from the most recent loop iteration to the next iteration.

Customize the gray sections for your unique programming goals.



①	To share data with the next loop iteration, pass the data into a shift register on the right border of the loop. This right shift register passes that data to a corresponding left shift register. To create a pair of shift registers, right-click the loop border and select **Create Shift Register**.
②	The left shift register contains data passed from the right shift register. Access this data by wiring the output of the left shift register to the code inside the loop.
③	For the first loop iteration, an initialized shift register returns the value wired to its input. If you do not initialize the shift register, it shares the data it held the last time the loop executed, even if that data is from a previous execution of the VI containing the loop.
④	After the loop executes, access the data from the last iteration by wiring the output of the right shift register to the rest of the program. Only values from the last iteration exit the loop through the output of the right shift register.

Examples

In the previous example, the value the left shift register passes into the loop changes after each iteration. The following table records the data each shift register contains after each loop iteration if the **initial value** wired to the left shift register is 10 and the **loop count** is 3.

Shift Register	Value after First Iteration	Value after Second Iteration	Value after Third Iteration
Left shift register	10	15	20
Right shift register	15	20	25

Accessing Data from Multiple Past Loop Iterations

About this task

A shift register passes values from one loop iteration to the next, but sometimes you need to access values from more than just the previous iteration.

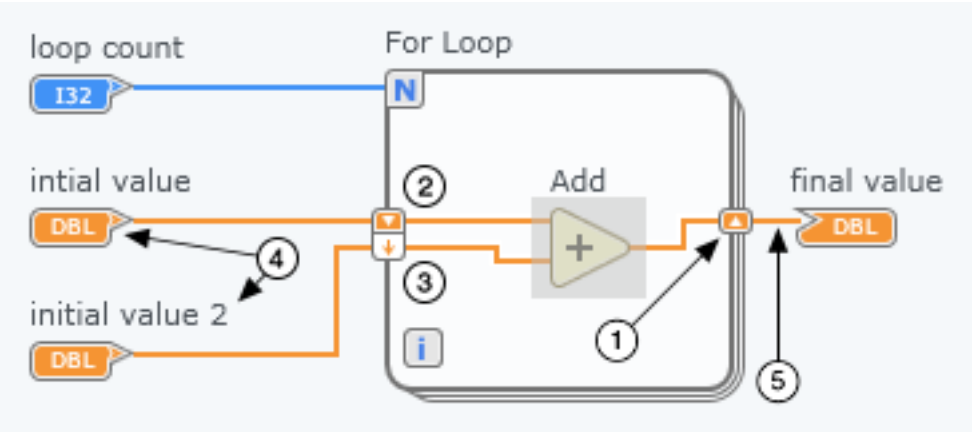
What to Use

- **For Loop** or **While Loop**

What to Do

Create the following diagram to pass data from the two most recent loop iterations to the current iteration.

Customize the gray sections for your unique programming goals.



1	A right shift register always passes data to the next loop iteration, regardless of whether you want to share data among one or multiple loop iterations. To create a pair of shift registers, right-click the loop border and select **Create Shift Register**.

②	After each iteration, the first left shift register contains the data passed from the right shift register. Access this data by wiring the output of the first left shift register to the code inside the loop.
③	The second left shift register contains the data from the second most recent iteration.
④	Just like for single shift registers, specify an initial value for each stacked shift register to ensure that each one has a predictable value for the first loop iteration. Each left shift register that lacks an initial value uses the most recent value it contained, even if that value is from a previous loop execution.
⑤	After the loop executes, access the data from the last iteration by wiring the output of the right shift register to the rest of the program. Only values from the last iteration exit the loop through the output of the right shift register.

Examples

In the previous example, the value contained in each shift register changed after each loop iteration. The following table records the data each shift register contains after each loop iteration, assuming the following conditions:

- The **loop count** is 3.
- The **initial value** wired to the first left shift register is 5.
- The **initial value 2** wired to the second left shift register is 2.

Shift Register	Note	First Iteration Value	Second Iteration Value	Third Iteration Value
First left shift register	The first left shift register receives new values from the right shift register.	5	7	12
Second left shift register	The second left shift register receives new values from first left shift register.	2	5	7
Right shift register	The right shift register receives values from the subdiagram of the loop.	7	12	19

Executing Code Based on a Condition

About this task

Use a Case Structure to incorporate decision logic into your program. A Case Structure contains one or more subdiagrams, or cases, exactly one of which executes when the structure executes.Case Structures behave similarly to switch statements or if-then-else statements in other programming languages.

What to Use

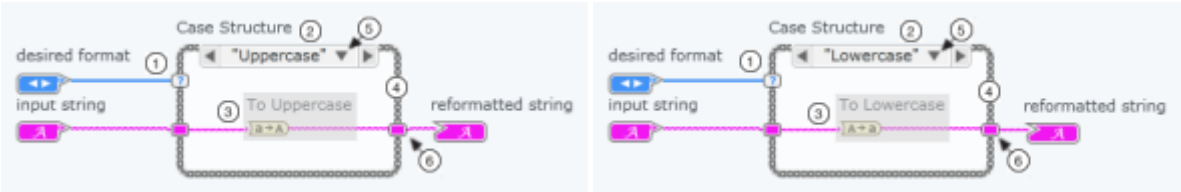
- **Case Structure**

> **TIP**
> If you are choosing between only two values based on a Boolean input, you can use the **Select** node instead of a Case Structure with a Boolean selector.

What to Do

Create the following diagram to execute different code based on a given condition.

Customize the gray sections for your unique programming goals.



①	Wire the data that you want to use to make a decision to the selector terminal. The data type of the value you wire to the selector terminal defines the various cases that are available in the Case Structure. The selector terminal accepts Boolean, string, integer, floating point, enumerated type, and error cluster data.
②	Use the case selector label to define the various cases in which different code executes. The case name displayed in the case selector label matches the selector terminal value(s) for which the corresponding subdiagram executes. You can define each case using a single value or a range of values. You can also use the case selector label to specify a default case.
③	Place the code you want to execute in each case on the corresponding subdiagram of the Case Structure.
④	To add, duplicate, or delete cases, right-click the border of the Case Structure and select the desired option from the **Cases** shortcut menu.
⑤	To view the different subdiagrams in the Case Structure, click the down arrow on the case selector label and select the desired case.

⑥	Every case must provide data to all output tunnels in order for the `Case Structure` to execute. If any case does not provide data to an output tunnel, an error occurs, and the output tunnel appears as a white box with a colored border. To resolve this error and allow the `Case Structure` to execute without explicitly wiring all cases, right-click the unwired output tunnel and select **Default If Unwired**. The output tunnel returns the default value for its data type if the subdiagram that executes does not provide that output tunnel with data. When set to **Default If Unwired**, the output tunnel appears as a white box with a colored border and a dash in the center. When all cases provide data to a particular output tunnel, that output tunnel appears as a solid box.

Troubleshooting

- If the default case executes unexpectedly, verify that the input values wired to the selector terminal match the values in the case selector label exactly.
- An edit-time error occurs when there are values of the selector data type that do not correspond to any subdiagram in the `Case Structure`. You must either define a default case to handle out-of-range values or create a case for every possible input value. For example, if the selector is an integer data type and you specify cases for 1, 2, and 3, you must specify a default case to execute if the input value is 4 or any other unspecified integer value.

Adjusting the Execution Speed of a Loop

About this task

By default, each loop iteration executes as quickly as possible based on the code inside the loop. However, you might want to control the rate at which a loop executes in order to specify a fixed time interval for each iteration, reduce the speed at which an indicator changes value, or conserve processing resources.

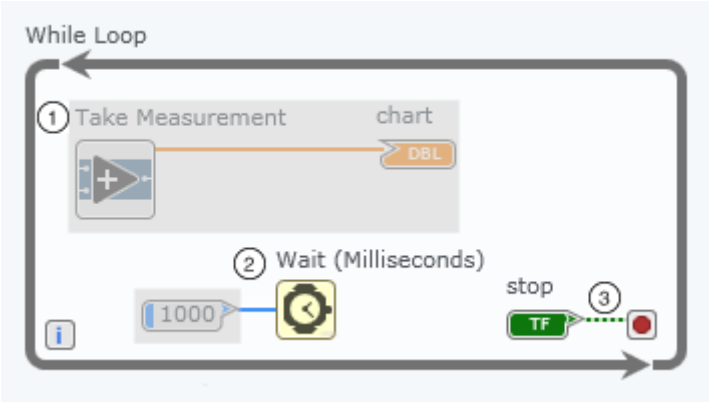
What to Use

- **`While Loop`** or **`For Loop`**
- **`Wait`**

What to Do

Create the following diagram to adjust the execution speed of a loop.

Customize the gray sections for your unique programming goals.



①	Place the code you want to repeat on the subdiagram of a `While Loop` or `For Loop`.
②	To specify the amount of time to wait between loop iterations, wire the desired time duration to the input of the `Wait` node. The `Wait` node waits until the value of the operating system's counter increases by an amount equal to the input you specify. The total running time of the loop is equal to the greater of the time it takes to run the code inside the loop and the wait time you specify.
③	Pressing the stop button does not interrupt the `Wait` node. For example, if the user presses the stop button in the middle of a loop iteration, the `While Loop` stops only after the specified wait period has elapsed and the code inside the loop has finished executing.

Troubleshooting

If a loop does not execute at the desired rate, verify that you have specified the input of the `Wait` node correctly. Consider the following common mistakes:

- If you are converting from another unit of time, verify that your conversion calculation is correct.
- If your code takes longer than expected to execute, note that the total running time of the loop is equal to the greater of the time it takes to run the code inside the loop and the wait time you specify.

Preventing Data Loss When Transferring Data Between Loops

Prevent data loss and promote efficiency when transferring data between producer and consumer loops by using queue nodes.

About this task

By creating a queue of data that is ready to be consumed, a consumer loop can process a greater volume of data at more regular intervals without losing data. This is especially useful if data from multiple network channels need to be processed in the order in which they arrive.

What to Use

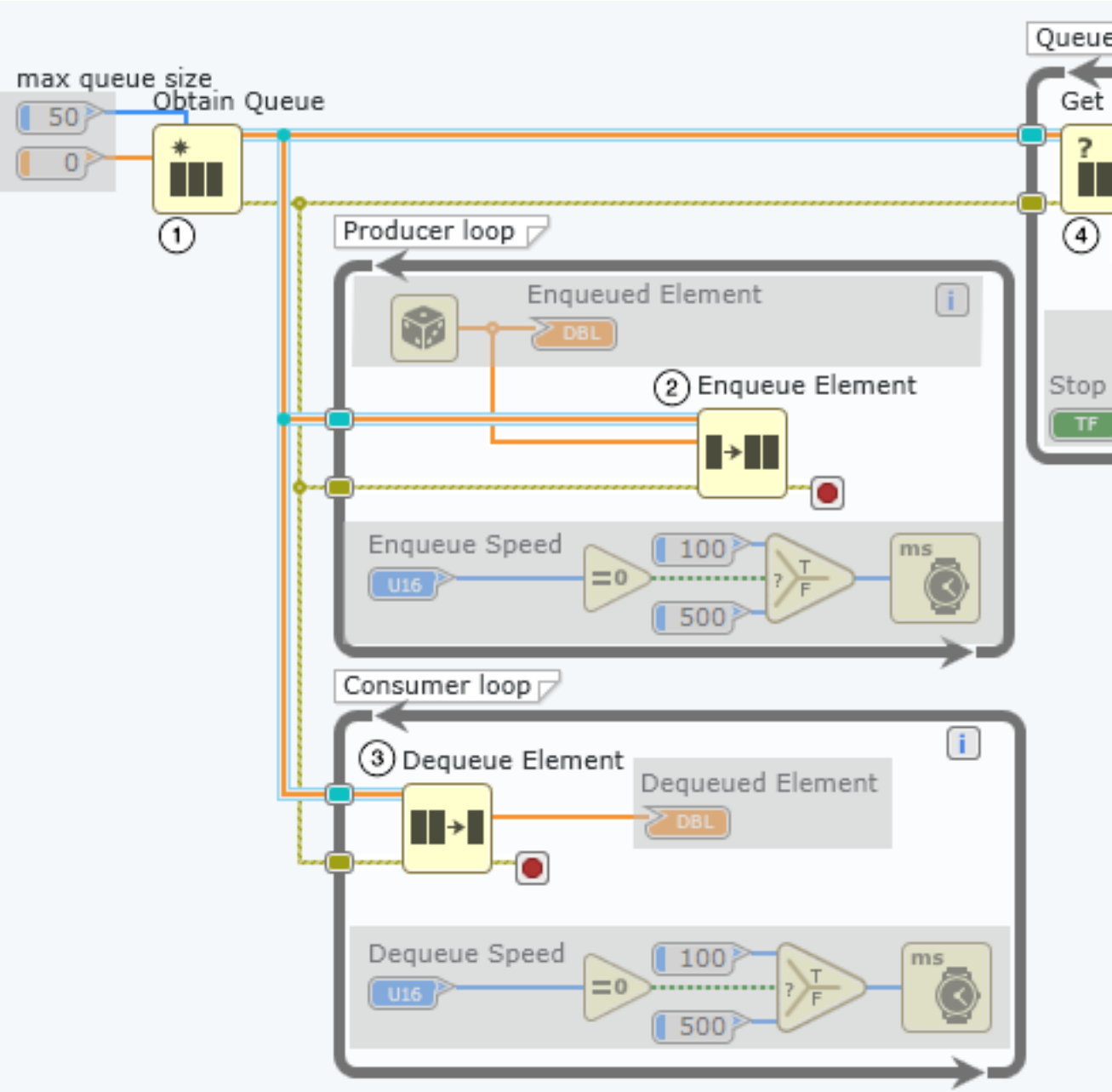
TIP
Since each queue is bound to one data type, each unique data type in a producer loop requires a unique queue, which could complicate the diagram. To avoid complicating the diagram with multiple queues, use a cluster or array of clusters to write a variety of data types in a single queue.

- **Obtain Queue**
- **Enqueue Element**
- **Dequeue Element**
- **Get Queue Status**
- **Release Queue**

What to Do

1. Create the following diagram to automate the reading and writing of data without the risk of losing data.

   Customize the gray sections for your unique programming goals.



1	Use `Obtain Queue` to set the data type for the queue and to set the maximum number of elements the queue can hold. In this example, the queue can hold up to 50 pieces of numeric data.

NOTE

If you call `Obtain Queue` inside a loop, this node creates a new reference each time the loop executes, and each new reference uses an additional four bytes of memory. To conserve memory, call `Obtain Queue` once before the loop executes.

②	Use `Enqueue Element` in the producer loop to add data to the back of the queue. If the queue is full, the producer loop stops adding elements to the queue until `Dequeue Element` removes data. If the producer loop runs slower than the consumer loop, the queue will empty and result in an underflow. **NOTE** If you want to ask a sensor for data at regular intervals instead of as often as possible, use `Wait` in the producer loop.
③	Use `Dequeue Element` in the consumer loop to remove data from the front of the queue so that data processes in the same order that it is added. If the queue is empty, the consumer loop waits to remove elements from the queue until `Enqueue Element` adds data. If the consumer loop runs slower than the producer loop, the queue will fill up and result in an overflow.
④	Use `Get Queue Status` in the queue status loop to return information about the current state of the queue, such as the number of elements currently in the queue. **NOTE** Using this node in parallel code can result in race conditions which can cause incorrect logic control and data loss.
⑤	Click the **Stop** button on the panel. The queue status loop finishes executing and calls `Release Queue`. This node invalidates the queue reference which causes the producer and consumer loops to error out and finish executing. **NOTE** Larger applications typically require a more sophisticated mechanism for stopping parallel loops. To see an example of such an architecture, launch the Queued Message Handler project in G Web Development Software.

Troubleshooting

- If a consumer loop processes data too slowly and results in an overflow, add a consumer loop to the diagram to raise the rate of data consumption.
- If a producer loop reads data too slowly and results in an underflow, raise the producer loop time if `Wait` is used. Otherwise add a producer loop to the diagram to raise the rate of data production.
- When multiple loops execute at the same time and change the same object, race conditions can occur which can result in data loss. Using `Get Queue Status` as a control mechanism that triggers an event between loops in code could result in race conditions if other loops change the contents of the queue before the event can be executed. This means that when the event does execute, it may do so based on outdated data and overwrite more current data. Improve the speed of your VI without the risk of creating race conditions by following the practices outlined in the Using Parallel Loops to Increase VI Execution Speed section of ***Strategies for Improving VI Execution Speed***. Use an event structure as a safer way to trigger your code using ***event-driven programming***.

Adding Code Snippets to a Custom Palette

Add items to a palette file (`.gpal`) that include chunks of code from your diagram.

Before you begin

Before you begin, complete the following prerequisite tasks:

- Enable `.gpal` option to place diagram contents when dropped.
- Enable *preview features*.
- Relaunch G Web Development Software.

Procedure

1. *Create a VI within your project.*
2. *Create a library.*
3. Add the VI to the library.
4. On the **Document** tab of a Library document, click **Create palette file**.
5. Select **G diagram palette** from the palette type drop-down menu.
6. Open your palette file (`.gpal`). Select your VI.
7. In the **Item** tab, under **Behavior**, select **Place diagram contents**.

   The block of diagram code from the VI will appear on the diagram in projects that include your custom `.gpal`.

   NOTE
   You cannot include a VI in an application if any of the exported VIs in the library are broken. If a VI in your library containing a snippet is broken, disable the VI's **Export** box before building the package.

Data Type Reference

You can use the following data types for controls and indicators, diagram terminals, wires, and node terminals.

Icon	Data Type	Details
SGL	Single-precision, floating-point numeric (SGL)	Minimum positive number: 1.40e-45 Maximum positive number: 3.40e+38 Minimum negative number: -3.40e+38 Maximum negative number: -1.40e-45
DBL	Double-precision, floating-point numeric (DBL)	Minimum positive number: 4.94e-324 Maximum positive number: 1.79e+308 Minimum negative number: -1.79e+308 Maximum negative number: -4.94e-324
CSG	Complex single-precision, floating-point numeric (CSG)	Same as single-precision, floating-point, with a real and an imaginary part.
CDB	Complex double-precision, floating-point numeric (CDB)	Same as double-precision, floating-point, with a real and an imaginary part.
I8	8-bit signed integer numeric (I8)	A positive or negative integer stored using 8 bits, or a single byte. Range: -128 to 127
I16	16-bit signed integer numeric (I16)	A positive or negative integer stored using 16 bits, or two bytes. Range: -32,768 to 32,767
I32	32-bit signed integer numeric (I32)	A positive or negative integer stored using 32 bits, or four bytes. Range: -2,147,483,648 to 2,147,483,647

Icon	Data Type	Details
I64	64-bit signed integer numeric (I64)	A positive or negative integer stored using 64 bits, or eight bytes. Approximate Range: -1e19 to 1e19
U8	8-bit unsigned integer numeric (U8)	A positive integer stored using 8 bits, or a single byte. Range: 0 to 255
U16	16-bit unsigned integer numeric (U16)	A positive integer stored using 16 bits, or two bytes. Range: 0 to 65,535
U32	32-bit unsigned integer numeric (U32)	A positive integer stored using 32 bits, or four bytes. Range: 0 to 4,294,967,295
U64	64-bit unsigned integer numeric (U64)	A positive integer stored using 64 bits, or eight bytes. Approximate Range: 0 to 2e19
	timestamp	A time and date. Minimum time: 01/01/1600 00:00:00 UTC maximum time: 01/01/3001 00:00:00 UTC
	enumerated type	A list of items from which to select.
TF	Boolean	A TRUE/FALSE value.
	string	A series of text characters.
[]	array	A collection of multiple pieces of data, each of which must be the same data type. The array icon shows the data type of the array's elements in square brackets and takes the color of that data type. As you add dimensions to the array, the thickness of the output wire increases.
	cluster	A collection of multiple pieces of data, each of which can be a different data type. Cluster data types appear brown if all elements in the cluster are numeric or pink if any elements in the cluster are of non-numeric types.
	error cluster	A cluster of information that specifies error conditions returned by a node.

Icon	Data Type	Details
	path	The location of a file or directory.
	waveform	A cluster of information that specifies an array of data, the start time at which the data was collected, and the change in time between each measurement.
	digital waveform	A cluster of information that specifies a digital table of data, the start time at which the data was collected, and the change in time between each measurement.
	reference number (refnum)	A unique identifier for an object, such as a file or a task.
	variant	A generic container for all other types of data.

RELATED INFORMATION

Numeric Data

You can program using a variety of numeric data types, each with different qualities and ranges. The differences among the numeric data types are the number of bits they use to store data and the data values they represent.

The following table describes the differences among the numeric data types.

Type	Description	Representation on the Diagram
Integers	Represent whole numbers. Signed integers can be positive or negative. Use unsigned integers when you know the integer is always zero or positive.	I8 U8 I16 U16 I32 U32 I64 U64
Floating-Point Numbers	Represent fractional numbers. Double-precision floating-point numbers store more digits than single-precision floating-point numbers.	SGL DBL
Complex Numbers	Represent a point in the complex numeric plane. Each value is comprised of two floating-point numbers, one representing the real part and the other representing the imaginary part.	CSG CDB

On the panel, you can use many types of controls and indicators to represent numeric values.

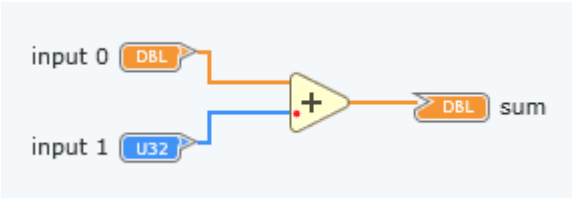
RELATED INFORMATION

Numeric Conversions

When you wire numeric data to a node that expects a different numeric data type, the node coerces the data to the most precise type.

A red coercion dot appears on the node input where the coercion takes place.

In the following diagram, a red coercion dot appears on the Add node because the node expects two inputs of matching numeric data types but receives a double-precision, floating-point number and an integer. Because `input 0` (a double-precision, floating-point number) is more precise than `input 1` (an unsigned, 32-bit integer), the node coerces `input 1` to a double-precision, floating-point number.



RELATED INFORMATION

Addressing Issues with Numeric Conversions on page 193

When you wire numeric data to a node that expects a different numeric data type, the node coerces the data to the most precise type. A red coercion dot appears where the coercion takes place. Because numeric coercions can affect memory usage, performance of a VI, or cause a VI to generate inaccurate results, use one of the strategies in the following table to eliminate or reduce the impact of a problematic coercion.

Addressing Issues with Numeric Conversions on page 193

When you wire numeric data to a node that expects a different numeric data type, the node coerces the data to the most precise type. A red coercion dot appears where the coercion takes place. Because numeric coercions can affect memory usage, performance of a VI, or cause a VI to generate inaccurate results, use one of the strategies in the following table to eliminate or reduce the impact of a problematic coercion.

Conversion Behaviors

By default, a data type with fewer bits changes to match the data type with more bits. If the number of bits is the same, unsigned data types are chosen over signed data types to retain as much precision as possible.

Conversion behaviors depend on the data types you are converting between or the node accepting the data. For example, if you wire two different data types to a node that expects matching data types, one of the inputs is converted to match the other.

The following table describes conversion behavior for different numeric data types.

Original Type	Converted Type	Conversion Behavior	Behavior Details
Floating-point number	Data is coerced to the nearest value possible.	This conversion is exact when the floating-point number is more precise than the integer. If the integer is more precise than the floating-point number, the floating-point data type maintains as much precision as possible in the conversion.	
Signed or unsigned integer	In-range values are coerced to a signed or unsigned integer. Out-of-range values are coerced to the minimum or maximum value of the integer.	This conversion is less precise if the floating-point number is an out-of-range value. For example, unsigned integers represent only non-negative integers. If you convert a negative floating-point number to an unsigned integer, the result is 0.	
Integer	If the source is smaller than the destination, the sign of a signed source is extended and zeros are placed in the extra bits of an unsigned source. If the source is larger than the destination, the destination retains only the least significant bits of the source value.	Out-of-range values are not coerced to the minimum or maximum value of an integer. The software attempts to retain as much precision as possible when converting between integer types.	

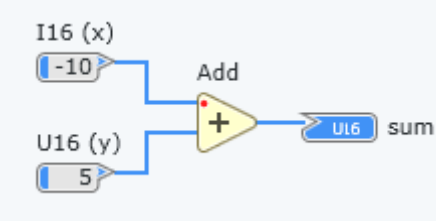
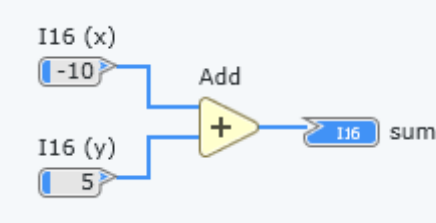
NOTE
Enums are treated as unsigned integers. For example, if you convert -1, a floating-point number, to an enum, the value is coerced into the range of the enum. If the range of the enum is 0 to 25, the value is converted to 0.

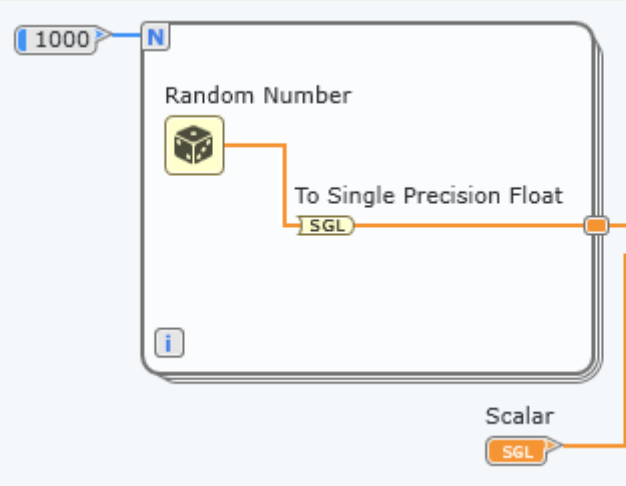
RELATED INFORMATION

Addressing Issues with Numeric Conversions

When you wire numeric data to a node that expects a different numeric data type, the node coerces the data to the most precise type. A red coercion dot appears where the coercion takes place. Because numeric coercions can affect memory usage, performance of a VI, or

cause a VI to generate inaccurate results, use one of the strategies in the following table to eliminate or reduce the impact of a problematic coercion.

Strategy	Details	Action	Example
Match input data types.	Create an object that matches the data type the node expects. If you wire data from a G type to a node or subVI that does not accept the data type of the G type, your VI may break or generate inaccurate data. Replace the node or subVI with one that matches the data type used in the G type, or replace the G type with a constant of the matching data type.	Right-click the coerced control, constant, or indicator on the diagram, select **Representation** from the shortcut menu, and select the matching data type.	In the following diagram, the red coercion dot c of the Add node indicates that the node is coe signed integer, to an unsigned integer to match of U16 (y). This causes Add to return a sum of 6 -5.  If you change the data type representation of I1 both inputs are signed integers, no coercion is r the node performs an accurate computation, as the following diagram. 

Strategy	Details	Action	Example
Use a conversion node.	Conversion nodes convert one data type to another. Conversion nodes have the same effect as the automatic coercions that occur when you wire numeric data to a node that expects a different numeric data type. However, when you use a conversion node you can control where in your VI the conversion takes place, which can improve VI performance in certain cases.	To add a conversion node from the `diagram palette`, navigate to **Data Types**»**Numeric**»**Conversion** and select the desired conversion node.	You may want to use a conversion node insid generates an array so the conversion is perform generates the array. This allows you to avoid buffer. In the following diagram, `To Single Float` converts the `Random Number` c single-precision number inside the `For Loop` Add node from converting a large amour 

RELATED INFORMATION

Numeric Conversions on page 192

When you wire numeric data to a node that expects a different numeric data type, the node coerces the data to the most precise type.

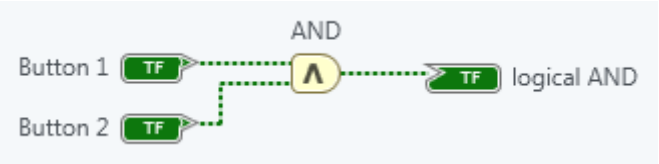
Numeric Conversions on page 192

When you wire numeric data to a node that expects a different numeric data type, the node coerces the data to the most precise type.

Boolean Data

Boolean data represents True and False values.

On the diagram, Boolean values are represented by green objects and wires.

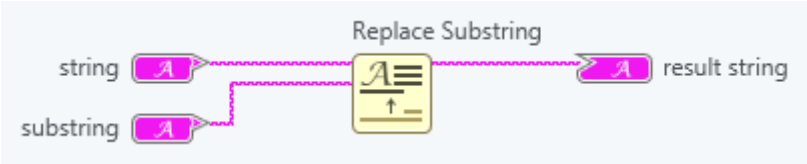


On the panel, Boolean data is represented as switches or buttons with mechanical actions that represent physical switches and buttons.

RELATED INFORMATION

String Data

Strings represent text. A *string* is a sequence of zero or more letters, numbers, and special characters. On the diagram, strings are represented by pink objects and wires.



The string data type cannot represent binary data. For binary data, use an array of U8 integers. For text, use a string data type. Casting any data type to a string may cause unexpected behavior when displaying or manipulating the string.

You can use built-in nodes to format, parse, and manipulate strings.

On the panel, strings appear as tables, text entry boxes, and labels.

RELATED INFORMATION

Parsing a String into Smaller Pieces

About this task

Parsing a string into smaller pieces allows you to perform operations on individual words or groups of characters in the string. These words or groups of characters are often referred to as *tokens*. A token is defined as either the next set of characters that appears before a separator character, called a *delimiter*, or one of a specified set of operators.

What to Use

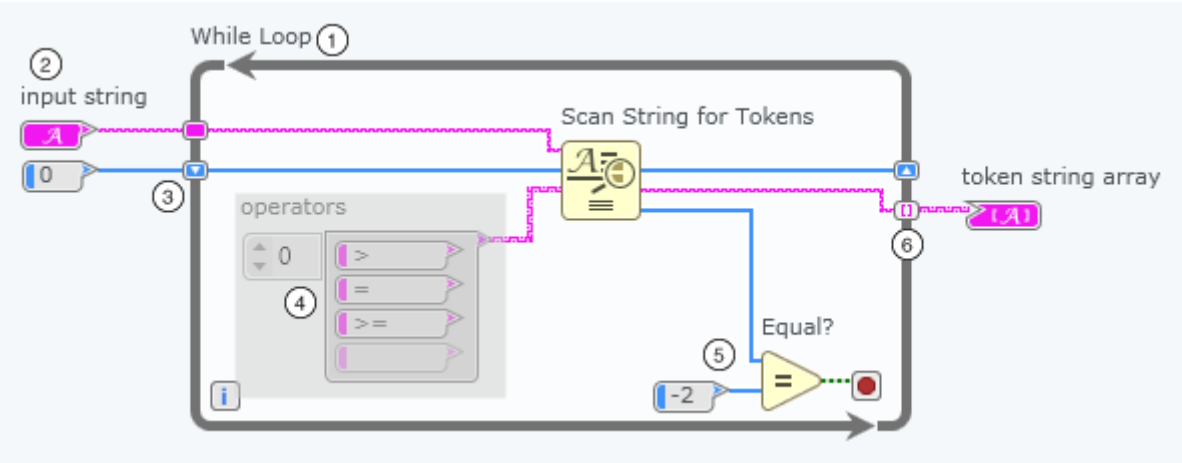
- **Scan String for Tokens**

- **While Loop** with shift registers

What to Do

Create the following diagram to parse a string into smaller pieces.

Customize the gray sections for your unique programming goals.



①	In order to identify all the tokens in a string, you must use a `While Loop`. Inside a `While Loop`, `Scan String for Tokens` scans the entire input string, returning all tokens it identifies until it reaches the end of the string. If you do not use `Scan String for Tokens` inside a `While Loop`, the node stops scanning as soon as it identifies the first token in a string and returns only that token.
②	The **input string** input of `Scan String for Tokens` contains the string to scan for tokens.
③	To start each scan at the location within the string where the preceding scan ended, pass the **offset past token** output of `Scan String for Tokens` through a shift register and back into the **offset** input of the same node. Initialize this shift register with the location within the string at which you want to begin parsing. Use 0 if you want `Scan String for Tokens` to begin its operation at the beginning of the string each time the program runs.
④	Add any strings that you want `Scan String for Tokens` to identify as tokens to the **operators** input array. The node identifies these strings as tokens even if they are not surrounded by any delimiters.
⑤	Detect the end of the input string by comparing the **token index** output of `Scan String for Tokens` to -2.

⑥	Use the auto-indexing output tunnel of the `While Loop` to collect the individual tokens in an array. This array contains all text that appears between delimiters as well as any strings specified in the **operators** input array that are found in the input string. `Scan String for Tokens` does not return delimiters as tokens but instead uses them to determine where tokens begin and end.

Troubleshooting

- If `Scan String for Tokens` does not identify a token that you expect it to identify, check to make sure the **operators** input array does not contain regular expression notation or any invisible characters. `Scan String for Tokens` does not process regular expressions. Also check for correct capitalization of items in the **operators** input array, as scanning is case-sensitive.

Examples

input string	operators	delimiters	token string
4>=0	[>, =, >=]	\s, \t, \r, \n (default)	[4, >=, 0]
a==b c!=d	[==, !=]	\s, \t, \r, \n (default)	[a, ==, b, c, !=, d]

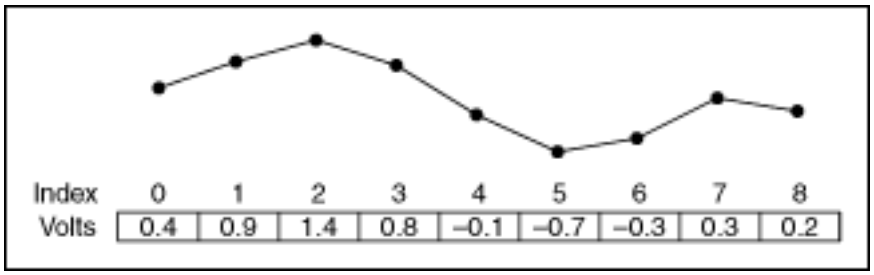
input string	operators	delimiters	token string
G2 X0.5Y1.0 i0.5j0 z-0.05	[X, Y, Z, i, j, z]	\s, \t, \r, \n (default)	[G2, X, 0.5, Y, 1.0, i, 0.5, j, 0, z, -0.05]
C1_1.11C2_2.22C3_3.33	None	C, _ (add to **delimiters** array) \s, \t, \r, \n (default)	[1, 1.11, 2, 2.22, 3, 3.33]

Arrays

An *array* combines data of the same data type into one data structure. An array consists of elements and dimensions. Elements are the data that make up the array. Dimensions represent the length, height, or depth of an array.

Consider using arrays when you work with a collection of similar data and when you perform repetitive computations. You can build arrays of numeric, Boolean, path, string, waveform, and cluster data types. You cannot create arrays of arrays. However, you can use a multidimensional array or create an array of clusters where each cluster contains one or more arrays.

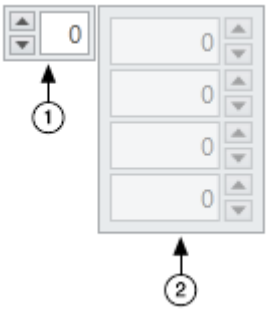
An example of an array is a series of numeric voltage measurements. The index indicates the position of each element in the array.



RELATED INFORMATION

Array Controls

The following image is an example of a one-dimensional numeric array control.

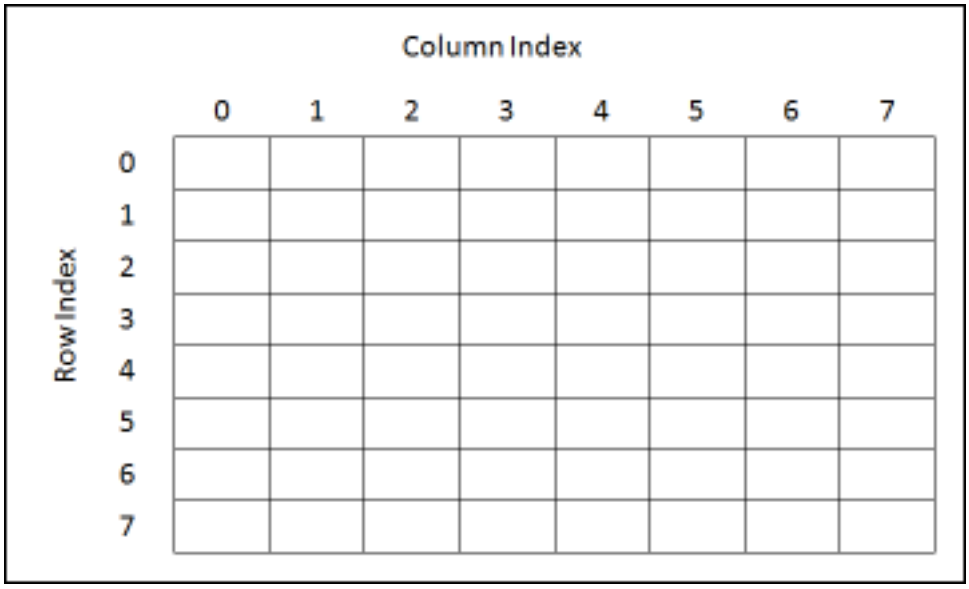


1. Index display—Determines which element to display at the top of the array control. An array index is zero-based.	**2.** Element display—Value in the array.

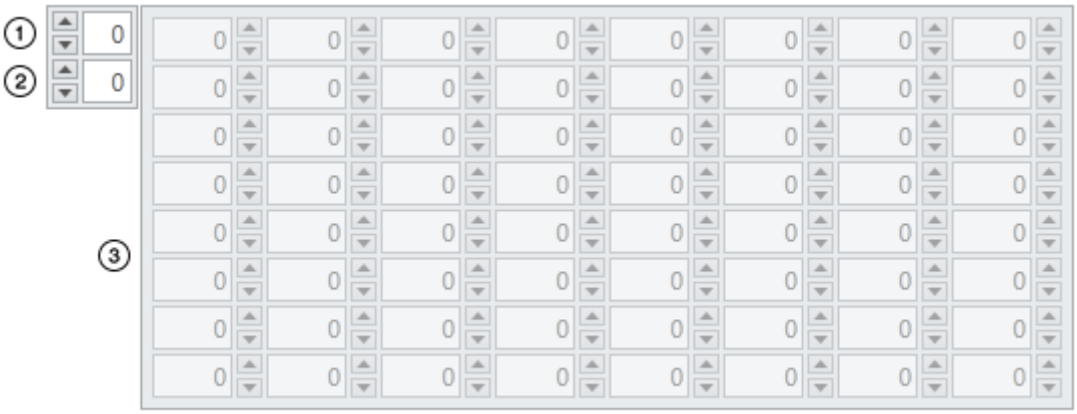
RELATED INFORMATION

Two-Dimensional Arrays

A two-dimensional (2D) array stores elements in a grid. It requires a column index and a row index to locate an element.



2D array controls and indicators have two index displays to navigate to specific indexes.



1. Top index display—Controls the row index to display in the array control.

2. Bottom index display—Controls the column index to display in the array control.

3. Element display—Values in the 2D array.

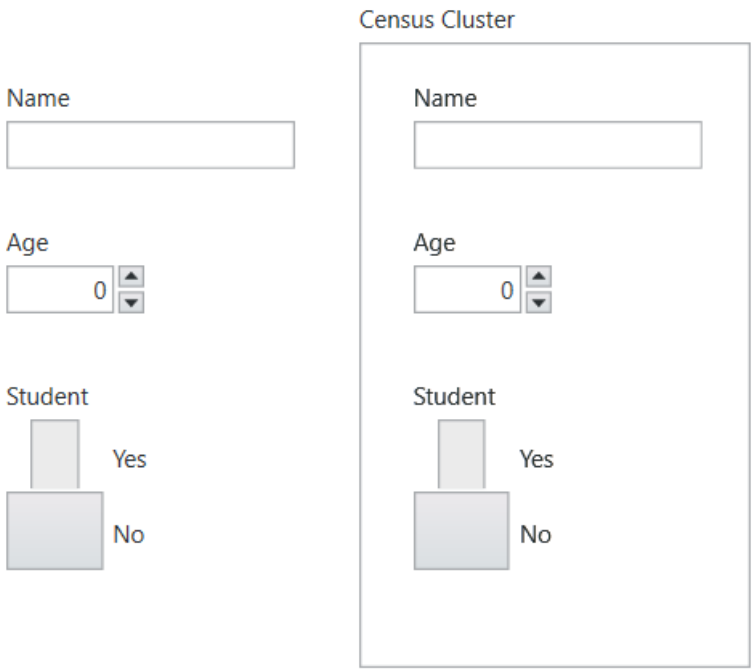
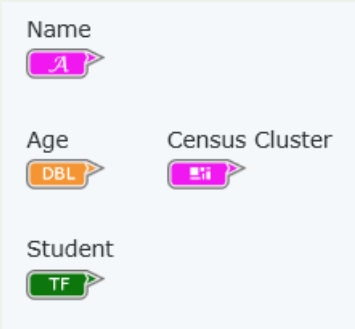
RELATED INFORMATION

Clusters: Building and Interacting with Grouped Data

Like elements of arrays, elements of a cluster are ordered. Unlike elements of arrays, the elements of a cluster are not ordered by their positions in the cluster but by the order in which the elements are added to the cluster. The first object you place in the cluster is element 0, the second is element 1, and so on. You can verify the order of elements in a cluster and reorder them as desired.

A cluster groups data elements of mixed data types. Grouping related data elements into clusters allows you to keep them together while you program. For example, you can use a cluster to store a person's name, age, and whether the person is a student instead of storing this data in three separate controls.

The following images show **Name**, **Age**, and **Student** controls as individual elements versus the same controls in a single **Census Cluster** control.

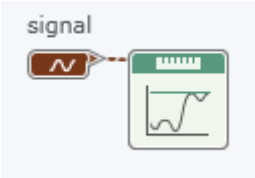
On the panel	
On the diagram	

RELATED INFORMATION

Waveforms

One common example of a cluster is the waveform. You might be familiar with the waveform data type if you have used it to collect data from a signal.

On the diagram, waveform objects and wires appear brown.



A waveform cluster is made up of three data types: a timestamp, an array of numerics, and a numeric constant. You can use these three pieces of data to create a signal that you can plot on a chart or graph.

Item	Details
t0	Start time—A timestamp associated with the first measurement point in the waveform.
Y	Waveform data—An array of Y values associated with the waveform data.
dt	Delta t—The time interval in seconds between any two points in the signal.

Because this data type is so common, you can access a waveform control from the palette on the panel, and you can access built-in nodes for waveforms from the palette on the diagram.

RELATED INFORMATION

Comparison Modes for Array and Cluster Data

When using certain Comparison nodes to compare data contained in arrays or clusters, you can select from two comparison modes.

Compare elements when you want to compare the individual corresponding elements of the input arrays or clusters.

Compare aggregates when you want to compare the input arrays or clusters in their entirety.

NOTE
You can select a comparison mode for any Comparison node with two or more inputs, such as `Greater?`, `Equal?`, and `Less or Equal?`. Comparison nodes with only one input, such as `Greater Than 0?`, compare the data contained in arrays or clusters to an implied second input, such as 0 or not a number. Because this second input is always a scalar, you can only compare elements.

Example

The following table describes the behavior of `Greater?` in each of the two comparison modes.

Comparison Mode	Input Array 1 (x)	Input Array 2 (y)	Result (x > y?)	Comments
Elements	[1,2,3,4]	[2,1,4,3]	[False, True, False, True]	`Greater?` compares the individual corresponding elements and returns an array of Boolean values that represent the result of each comparison.
Aggregates	[1,2,3,4]	[2,1,4,3]	False	In **Aggregates** mode, a Comparison node processes corresponding elements sequentially, checking for inequalities. The node stops as soon as it encounters corresponding elements that are not equal or reaches the end of one of the input arrays or clusters and performs the comparison on those elements. The first elements in the input arrays are not equal, so `Greater?` stops and returns a single Boolean value representing the result of 1 > 2?

You can change the comparison mode for a selected Comparison node by selecting **Elements** or **Aggregates** on the **Item** tab in the **Compare** section.

RELATED INFORMATION

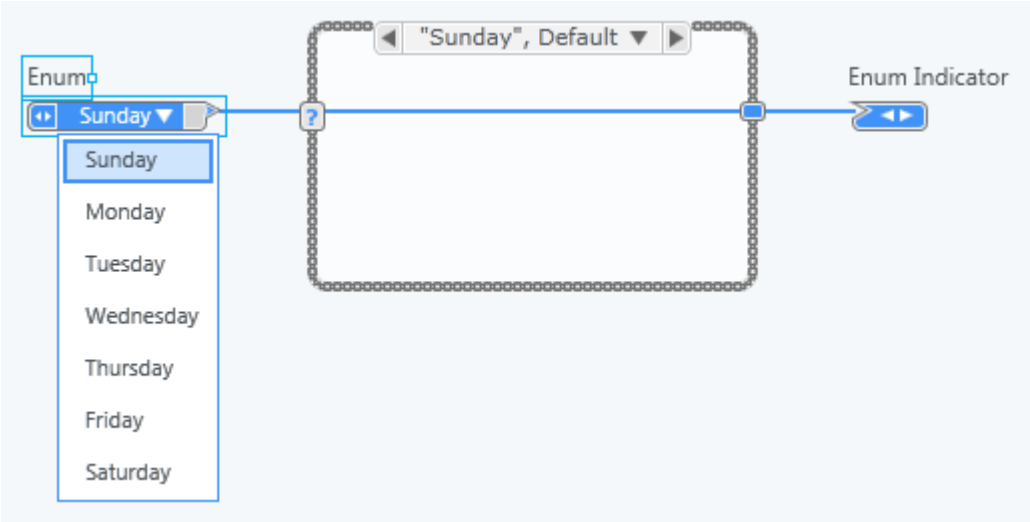
Comparison Nodes

Enum Data

The enumerated type, or enum, is a container for a finite set of user-named values. Each item in the set has both a string and a numeric value.

On the diagram, an enum appears as a pull-down list of names represented as numeric data.

You can use the named values to drive the logic of your program. For example, you can wire an enum constant or control to the selector of a `Case Structure` to create cases that correspond to each item in the enum.



You can use the numeric values within the enum for numeric calculations or comparisons. If you wire an enum control to a numeric node or indicator, the enum value is coerced to a numeric value. All arithmetic functions except `Increment` and `Decrement` treat enum data the same as an unsigned integer.

Type Definitions

Type definitions are custom data types that you define using the G language and can reuse throughout your project.

You define a type definition in a G Type document.

RELATED INFORMATION

Creating a Custom Data Type Using a G Type Document on page 205

Updating Custom Data Type Instances Throughout Your Project on page 207

Modify a G Type document to update all the instances of a custom data type in your project.

Creating a Custom Data Type Using a G Type Document on page 205

Updating Custom Data Type Instances Throughout Your Project on page 207

Modify a G Type document to update all the instances of a custom data type in your project.

Creating a Custom Data Type Using a G Type Document

Create a type definition, or custom data type, to reuse a control or constant throughout your project. You define the type definition with the control you add to the panel of a G Type document.

About this task

An example of when a type definition would be useful is keeping state enums of a state machine in sync. Because instances of a type definition share a source file, you can update the source enum, the type definition, with additional states as needed and propagate the changes to all instances rather than having to update each copy individually.

Procedure

1. From the **Project Files** tab, expand the **New** menu and select **Type Definition** to create a G Type document.
2. In the G Type document, add a single control, array of controls, or cluster of controls that you want to reuse throughout your application.

   The data type of the control you add to the G Type document panel defines the data type of the type definition.

   NOTE
   Adding more than one control outside of an array or cluster or more than one array or cluster of controls to a G Type document makes the type definition unusable.

3. In the **Item** tab, configure the control with the settings you want to persist to each instance of the type definition.
4. Save the G Type document.

Results

Drag the type definition from the **Project Files** tab to the panel or diagram to add an instance of the type definition to your code.

RELATED INFORMATION

Type Definitions on page 205

Updating Custom Data Type Instances Throughout Your Project on page 207

Modify a G Type document to update all the instances of a custom data type in your project.

Type Definitions on page 205

Updating Custom Data Type Instances Throughout Your Project on page 207

Modify a G Type document to update all the instances of a custom data type in your project.

Updating Custom Data Type Instances Throughout Your Project

Modify a G Type document to update all the instances of a custom data type in your project.

Procedure

1. Open the G Type document that defines the control you want to modify.

   TIP
   You can right-click an instance of the type definition control or constant you want to modify and select **Open Type Definition** to open the associated G Type document. On the diagram, type definition controls and constants display a black marker on the left side to help you identify them.

2. In the G Type document, modify the control to your needs.

3. Save the G Type document to propagate the control updates to all instances of the type definition.

RELATED INFORMATION

Type Definitions on page 205

Creating a Custom Data Type Using a G Type Document on page 205

Type Definitions on page 205

Creating a Custom Data Type Using a G Type Document on page 205

Programming Design Patterns

A design pattern is a code structure or practice that solves common programming problems.

Here are a few examples of design patterns and the problems they solve.

- Event-driven programming allows your code to respond to events during execution.
- The state machine design pattern allows you to execute different parts of your code based on the state of your application.

RELATED INFORMATION

Event-Driven Programming

Event-driven programming is a programming pattern in which you create diagram code that executes outside the dataflow structure as a result of an event occurring.

Use event-driven programming when you want diagram code to execute in response to an event. An *event* is anything that happens to the user interface during run time that provides a notification to other parts of a program.

Events are handled by the `Event Structure`. An `Event Loop`, which consists of an `Event Structure` within a `While Loop` waits for events, responds to those events, then returns to waiting for the next event. Place the code that handles the event in the appropriate event case of the `Event Structure`.

For example, if you want an event to occur when the user presses a specific key on the panel, you can design an application to include a loop that waits until the key press event occurs, handles the event, and returns to wait for the next event. While the loop waits for the event to occur, other parts of the application can continue to run. How the application handles each event depends on the code written for that specific event. Some sections of the program may execute frequently because the events those sections handle occur frequently, and other sections of the program may not execute at all because the events never occur.

RELATED INFORMATION

Event Sources

You can configure the `Event Structure` to handle events from any supported source.

- User Interface—mouse clicks, key presses, and so on.
- Programmatically generated events—events triggered by diagram code, without any user interaction.

Each event is associated with a control on the panel of the VI, the panel window of the VI, a pane of the VI, or the application.

NOTE
The application generates events for a VI only while that VI runs or when another running VI calls that VI as a subVI.

RELATED INFORMATION

User Interface Events

User interface events are events that occur when a user performs a particular action on the panel.

Each time a user interface event occurs, the diagram executes the code you write for the event. Compared with polling the user interface in a loop for interactions, user interface events reduce the CPU requirements of your program, simplify the diagram code, and ensure that the diagram can respond to all user actions.

You can select from two categories of user interface events: notify and filter.

Event Type	Description	Example	Use Case
Notify	Informs you that a user action occurred.	Configure an `Event structure` to execute in order to handle a Value Change event when the user changes the value of a control.	Use notify events only if you want to know the user performed a specific action.
Filter	Allows you to validate or change the event data before the software performs the default action associated with that event. You can also discard the event to prevent the change.	Configure an `Event structure` to discard a Key down? event, which prevents a user from clicking the mouse to select or change behavior on the panel.	Use filter events if you want to influence how the software handles certain user actions.

NOTE

Filter event names include a question mark to help you distinguish them from notify events. The software generates an associated notify event after most filter events if no event case discarded the event. The notify event has the same name as the filter event but without the question mark.

RELATED INFORMATION

General User Interface Event Reference

The following events occur when the user performs an action that involves the panel or any control on the panel.

Event	Description	Event Data Fields
Value Changed	Generated when the user changes the value of a control. You must read the terminal of a latched Boolean control in its Value Change event case.	• **Index**—Index of the event source within an individual event diagram. • **Time**—Value of the millisecond timer when the event occurred. • **Control Reference**—Reference to the control on which the event occurred. • **Old Value**—Value of the control before the data change. • **New Value**—Value of the control after the data change.

RELATED INFORMATION

Tree Control Event Reference

The following events occur when the user performs an action that involves a tree control.

Event	Description	Event Data Fields
Item closed	Occurs when the user closes an item in a tree control.	• **Time**—Value of the millisecond timer when the event occurred. • **Index**—Index of the event source within an individual event diagram. • **Control Reference**—Reference to the control on which the event occurred. • **Location**—Hierarchical location of the closed item. • **Data**—Data for the closed item, including the hierarchical location and associated data for the columns.

Event	Description	Event Data Fields
Item opened	Occurs when the user expands an item in a tree control.	• **Time**—Value of the millisecond timer when the event occurred. • **Index**—Index of the event source within an individual event diagram. • **Control Reference**—Reference to the control on which the event occurred. • **Location**—Hierarchical location of the expanded item. • **Data**—Data for the expanded item, including the hierarchical location and associated data for the columns.
Selection changed	Occurs when the user selects or deselects one or more items in a tree control.	• **Time**—Value of the millisecond timer when the event occurred. • **Index**—Index of the event source within an individual event diagram. • **Control Reference**—Reference to the control on which the event occurred. • **OldSelection**—Hierarchical location of the previously selected item. This event data field can be a string or an array of strings, depending on the selection mode of the tree control. • **NewSelection**—Hierarchical location of the selected item. This event data field can be a string or an array of strings, depending on the selection mode of the tree control. • **SelectedData**—Data for the selected item, including the hierarchical location and associated data for the columns. This event data field can be a cluster or an array of clusters, depending on the selection mode of the tree control.

RELATED INFORMATION

Programmatically Generated Events

Programmatically generated events are events that occur as a result of the execution of diagram code.

A user event is a type of programmatically generated event. Unlike user interface events, which require direct user interaction with front panel objects, user events allow you to create an application that responds to programmatic changes on objects. In order for an `Event Structure` to process a user event, you need to complete the following steps:

1. Create the event using the `Create User Event` node.

2. Register for the event using the `Register for Events` node.
3. Enable the dynamic event terminals on the `Event Structure`.

Programmatically generated events communicate among different parts of the program that have no dataflow dependency. For example, one part of an application may run a continuous loop to monitor a temperature sensor and generate a user event when the sensor reaches an alarm level. In a different part of the same application, an `Event Structure` executes to handle both the user event generated when the sensor reaches the alarm level and a user interface event that the `Event Structure` handles to notify the user of the alarm condition. Handling both user interface events and programmatically generated events in the same `Event Structure` allows you to implement more advanced architectures, such as queued state machines using events.

RELATED INFORMATION

Executing Code Based on an Event

To execute code based on an event, you must configure the `Event Structure` to handle events.

Before you begin

Before you configure events for the `Event Structure` to handle, review the *Recommendations for Event-Driven Programming*.

About this task

Complete the following steps to configure an `Event Structure` case to handle events.

Procedure

1. (Optional) If you want to configure the `Event Structure` to handle an external I/O event or a programmatically generated event, you first must dynamically register that event.
   a) Select the `Event Structure`. On the **Item** tab, click **Use dynamic events** to enable the `Dynamic Event Terminals`.
   b) Place a `Register for Events` node on the diagram and wire the output to the input of the `Dynamic Event Terminal`.
   c) Create diagram code for your event source and wire it to the inputs of `Register for Events`.
2. Click the event selector label at the top of the `Event Structure`, to view the list of available event sources from which you can choose.
3. Select one or more event sources for which you want to respond. Then select an event type for each event source.

The event case you configure appears as a selection option in the event selector label at the top of the Event Structure, and the Event Data Node displays the data common to all events handled in that case.

4. Create the code you want the Event Structure to execute when the event occurs.

   Place this code inside the Event Structure.

5. (Optional) Use a Timeout event to configure an Event Structure to wait a specified amount of time for an event to occur.

   If no event occurs before the specified amount of time passes, the Event Structure stops handling events.

6. Add additional event cases as required by your application.

   a) Select the Event Structure and click **+** on the **Item** tab to add additional event cases.
   b) Repeat steps 2 through 4 for each event case you want to configure.

Executing Code Based on a User Event

About this task

User events are defined on the diagram and triggered by the execution of diagram code. Unlike user interface events, which require direct user interaction with front panel objects, user events allow you to create an application that responds to programmatic changes on objects. Additionally, you can handle data associated with the event in its case of the Event Structure.

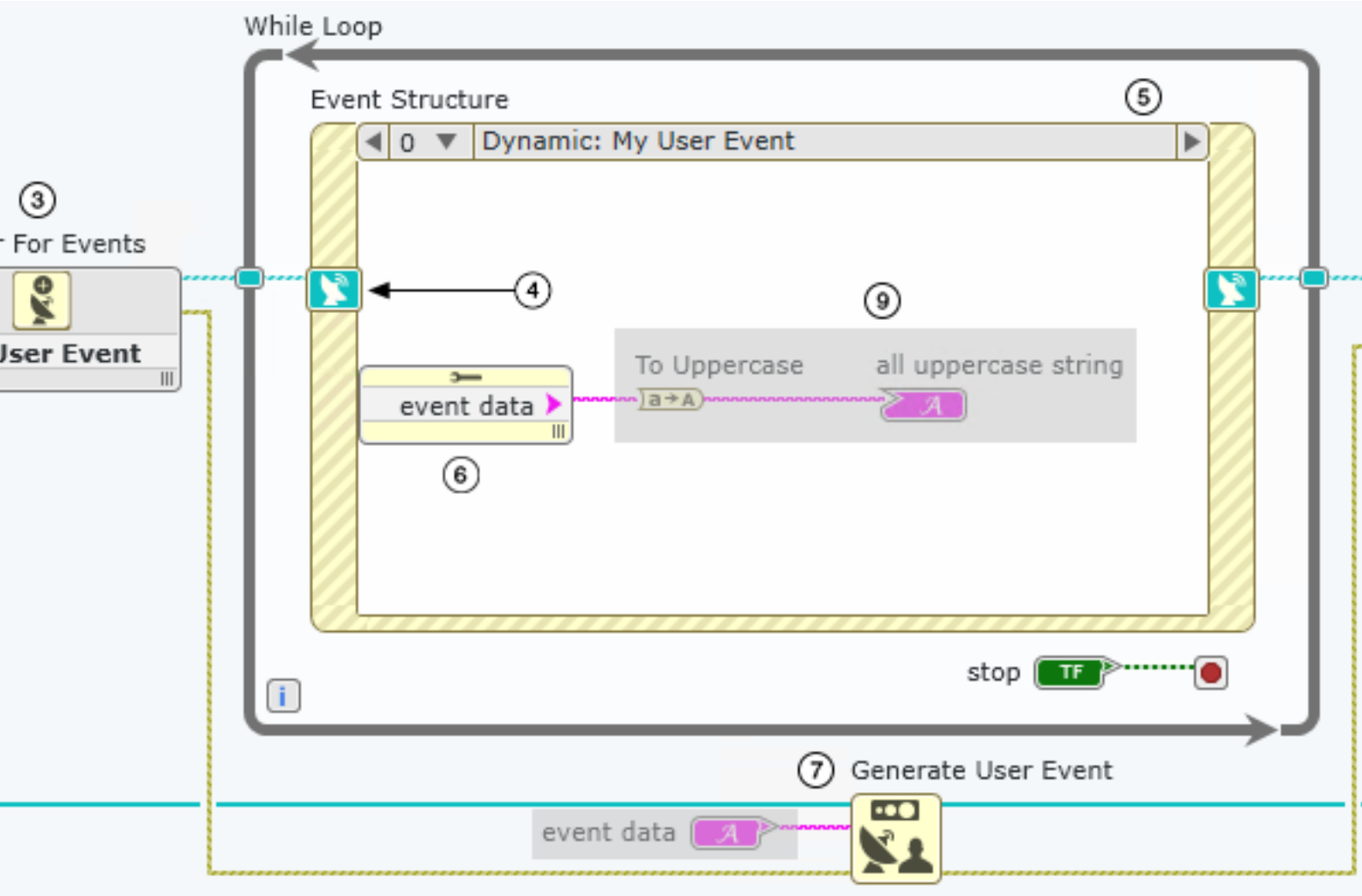
What to Use

- **Create User Event**
- **Register for Events**
- **Event Loop**
- **Generate User Event**
- **Unregister for Events**
- **Destroy User Event**

What to Do

Create the following diagram to build a user event that notifies the Event Structure to handle the event and any associated data.

Customize the gray sections for your unique programming goals.



①	To define a user event, wire a constant of the same data type as the event to `Create User Event`. The input for `Create User Event` is an individual element or a cluster whose data type defines the data type for the event.
②	`Create User Event` returns a reference to the user event that you create. Use this reference to register for events and generate a user event. To give a unique name to the user event you create, select `Create User Event` and edit the **Event name** field on the **Item** tab. In this diagram, the name of the user event is My User Event.
③	Use `Register For Events` to dynamically register the user event. You must dynamically register the user event so that the `Event Structure` can handle it when it occurs.

④	To display the `Dynamic Event Terminal`, click the `Event Structure` and select **Use dynamic events** on the **Item** tab. Wire the **event registration refnum** output of `Register For Events` to the `Dynamic Event Terminal` on the left side of the `Event Structure`. After you select an event, the user event data items appear in the `Event Data Node` on the left border of the `Event Structure`.
⑤	Click the event selector label at the top of the `Event Structure` to select one or more event sources for which you want to respond. Then select an event type for each event source. The name of the user event appears under the **Event type** drop-down list when you select **Dynamic** as the event source.
⑥	Use the `Event Data Node` to access data elements associated with a particular event. To add more data items to the `Event Data Node`, vertically resize the node.
⑦	Use `Generate User Event` to broadcast the user event you wire to this node and the associated event data to each `Event Structure` registered to handle the event. The data value that you wire to this node must match the data type of the user event. **NOTE** If the user event is not registered, `Generate User Event` has no effect. If the user event is registered but no `Event Structure` is waiting on it, the VI queues the user event and data until an `Event Structure` executes to handle the event.
⑧	To conserve memory resources, use `Unregister For Events` and `Destroy User Event` to release the associated events and the user event reference.
⑨	Create code that executes whenever this user event occurs.

Recommendations for Event-Driven Programming

Because handling events in G Dataflow is different from handling events in other programming languages, follow NI's recommendations for event-driven programming.

RELATED INFORMATION

General Recommendations

Recommendation	Details
Place the Event Structure in a While Loop that terminates when events can no longer occur.	Using a While Loop ensures that the Event Structure can handle events whenever events occur. Without a While Loop, the Event Structure handles only the first event it receives. If the VI queues events after the Event Structure finishes executing, the Event Structure cannot handle those events.
To ensure that every queued event is handled, design the VI to finish running soon after the Event Structure stops executing.	The VI uses an event queue to store events and handle them in the order in which they occur. Because the VI queues events for the entire time that it is running, the VI can queue events after dataflow causes an Event Structure to finish executing.
Use caution when placing an Event Structure within a Case Structure.	If a Case Structure prevents an Event Structure from executing to handle events, the Event Structure cannot handle events when they occur. If you have enabled panel event pausing, the user interface of the VI can become unresponsive. For example, if an event case launches an application that requires text entry from the user, the user might begin typing before the application appears on the panel. If panel event pausing is enabled, once the application launches and appears on the panel, it processes the key presses in the order in which they occurred. If panel event pausing is disabled, the key presses might be processed elsewhere on the panel. To enable or disable panel event pausing, use the **Pause panel event processing until complete** checkbox on the **Item** tab of the Event Structure.
Wire each Event Structure that handles dynamic events to a unique Register for Events node.	Branching the wire from a Register for Events node allows multiple Event Structures to pull events from one queue, resulting in a race condition that may cause unpredictable behavior.

Recommendation	Details
When you use multiple Event Structures in the same VI, observe the following guidelines: • Place each Event Structure in its own While Loop. • Wire your diagram code so that the Event Structures execute simultaneously. • Wire your diagram code so that the Event Structures stop handling events simultaneously.	You can use multiple Event Structures in the same VI, especially if you register for events dynamically. For example, you can place two Event Structures within two separate While Loops that are subject to different stop conditions.
Use Unregister for Events and Destroy User Event to unregister and destroy user events.	The VI unregisters all events and destroys existing user events automatically when the top-level VI finishes running. However, to conserve memory resources, unregister and destroy user events explicitly, especially in an application that runs for a long period of time.

Notify Events

Recommendation	Details
Avoid configuring one Event Structure case to handle multiple notify events of different data types.	A single case in an Event Structure can handle multiple notify events. However, if the events are not the same type, only the event data fields common to all events that the case handles appear on the Event Data Node.
Do not use the Panel Close notify event for shutdown code.	Including shutdown code in your VI allows you to stop the VI in a controlled manner and protect data. If you use the Panel Close notify event to execute shutdown code, the VI can abort before the shutdown code executes.

Events and Loops

Recommendation	Details
Avoid placing code that takes a long time to execute within the Event Structure because the UI will not be able to respond to additional user input until the operation finishes.	If no Event Structure executes promptly to handle an event and panel event pausing is enabled, the user interface of the VI may become unresponsive. If you have enabled panel event pausing, the user interface of the VI can become unresponsive. For example, if an event case launches an application that requires text entry from the user, the user might begin typing before the application appears on the panel. If panel event pausing is enabled, once the application launches and appears on the panel, it processes the key presses in the order in which they occurred. If panel event pausing is disabled, the key presses might be processed elsewhere on the panel. To enable or disable panel event pausing, use the **Pause panel event processing until complete** checkbox on the **Item** tab of the Event Structure.
Avoid placing two Event Structures in one loop.	If you place two Event Structures in a single loop, the loop cannot iterate until both Event Structures handle an event. For example, if you place two Event Structures in a single While Loop and configure the first Event Structure to handle a Mouse Down event and configure the second Event Structure to handle a Key Down event, if the user clicks the mouse button twice with no intervening Key Down event, the user interface will not respond to the second until the user presses a key.

Events and Latched Boolean Controls

Recommendation	Details
When using events with latched Boolean controls, place the control inside the event case so that the mechanical action of a latched Boolean control behaves correctly.	When you trigger an event on a Boolean control with a latched mechanical action, the Boolean control does not reset to its default value until the diagram reads the Boolean control. For example, if you wire a latched stop Boolean control to the condition terminal of a While Loop, create an event case that handles the latched stop Boolean control in the Event Structure.
When you wire a latched Boolean control, place the Boolean control in its Value Change event case.	

Recommendation	Details
Use a Case Structure within the event case to handle undo operations for a latched Boolean control.	When you perform an undo operation on a Boolean control with a latching mechanical action, the Event Structure handles the undo operation as a Value Change event, which might return unexpected results. Use the True case of the Case Structure to specify how you want to handle the Boolean control and the False case to specify how you want to handle the undo operation.

State Machine Design Pattern

You can use the state machine design pattern to implement decision making algorithms where a set of distinguishable states exists.

These states, or subdiagrams of code, carry out specific operations within a program. Only one state executes at a time while the machine is active.

After all of the code within a state executes, the state outputs a transition value and initiates a state transition. This transition advances the program from the finished state to the next state indicated by the transition value.

During a state transition, data from the completed state is sent to the upcoming state. Once the state machine executes its final state, data output by the machine becomes available for other parts of the program to use.

RELATED INFORMATION

When to Use a State Machine

You can apply the state machine design pattern to a wide variety of programming tasks.

The following tasks are examples of situations for which state machines are well suited:

- Responding to user interface interactions where the user's action determines which state executes.
- Process testing where each state carries out a step of the process.
- Breaking down difficult to manage applications into smaller, easily maintainable chunks of code.

However, a state machine is not well suited for every programming situation. If your program needs to run parallel processes, you may want to choose a different design pattern.

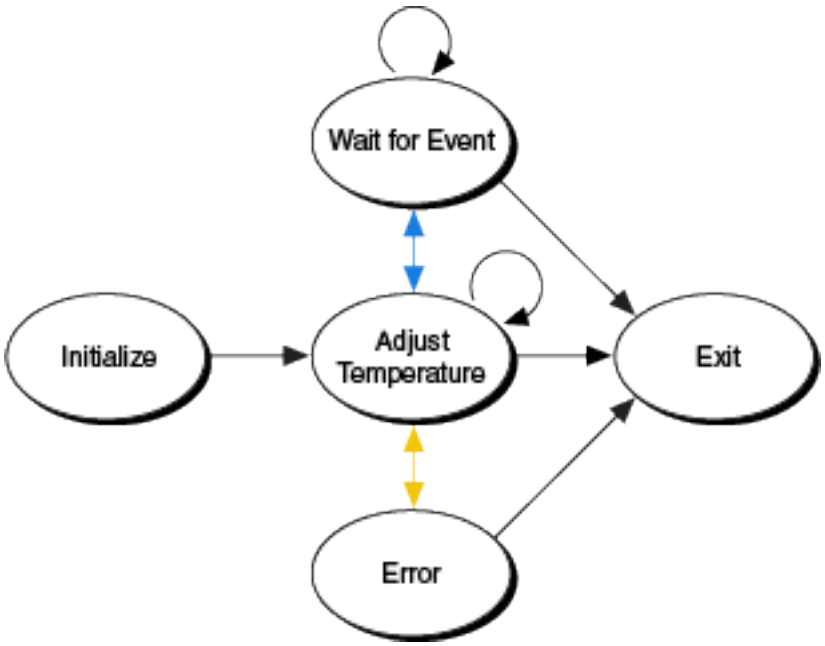
RELATED INFORMATION

State Diagrams

To keep track of every state and interaction in your program, design a state diagram of your state machine before you start programming.

A state diagram is an illustration of all the states in a state machine and how these states interact with each other. Each circle represents a state, and each arrow represents a possible state transition. While you create your code, you can follow the diagram flowchart to recall how to structure the state machine and how the states within the machine interact.

The following image shows an example of a basic state diagram for a program that changes the temperature of a room over time.



The state diagram shows all possible state transitions in the program. You can use the state diagram to understand how the states within the program interact before analyzing the code that defines the interactions.

- Each arrow points toward the endpoint of a state transition. For example, the initialize state can only transition to the adjust temperature state. On the other hand, the program can transition back and forth between the adjust temperature state and wait for event state.
- A circular arrow, seen on the adjust temperature and wait for event states, indicates that the state can transition to itself.

- A blue arrow indicates that a transition occurs because of a user action, and a yellow arrow indicates that a transition occurs because of an error in the program.

RELATED INFORMATION

Standard States To Consider When Planning Your Program

When you design a state machine, create a distinct initialize state for the program. You can also add a specific state to handle user input or provide custom error handling, depending on your program needs.

States to Include in Every State Machine You Create

Specify one entry point and one exit point for a state machine to control the code that executes each time the state machine starts up and shuts down.

Initialize state —The first state a state machine executes, which includes any application initialization code. Common uses for initialization code include opening file references and hardware references to use later and bundling control references into the data cluster to unbundle later.

NOTE
Web applications run infinitely once initialized.

A state machine runs continuously until the `condition terminal` on the `While Loop` receives the stop value determined by the user. Directing your state machine toward a single exit rather than accounting for multiple exit points allows you to control the shutdown code that executes each time the machine stops. Using a single exit state also helps prevent accidental, premature, or partial state machine shutdowns.

States to Consider Depending on Your Program Needs

If you are designing a program that implements user interface actions or contains specialized error handling, consider including these states in your state machine.

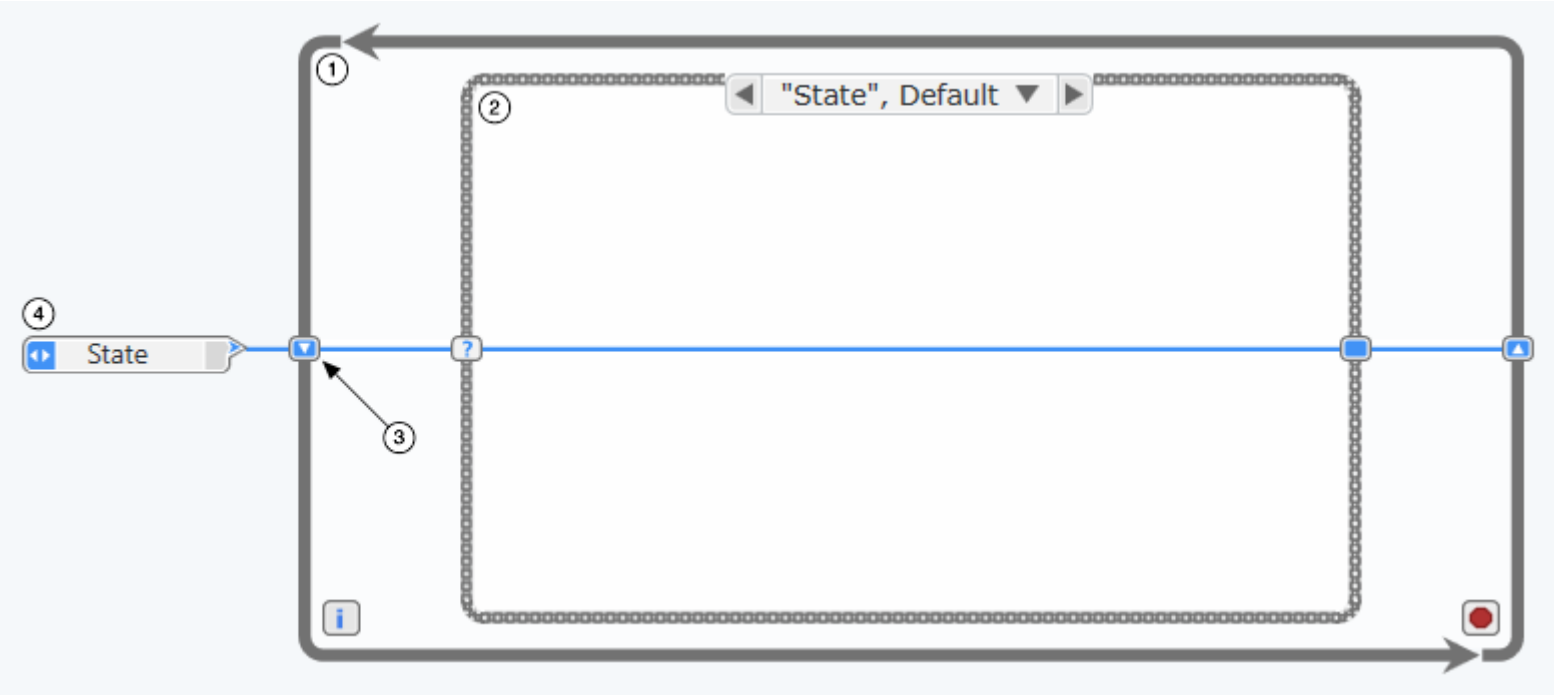
Wait for event state —A state that accepts and implements user input.

Error handling state —A state that contains error handling code for the state machine.

RELATED INFORMATION

Diagram Components of a State Machine

A state machine has four components on the diagram: a `While Loop`, a `Case Structure`, an `Enum constant`, and a `shift register`.



1. While Loop—Sets the outer boundary for the state machine code and facilitates state transitions.
2. Case Structure—Contains a subdiagram for each state in the state machine.
3. Shift register—Passes data from a completed state to the upcoming state.
4. Enum constant—Contains a list of every state in the state machine. The state machine uses this constant to update the current state transition value as the program executes.

NOTE

Convert the `Enum constant` to a G Type and configure its list of values before using it in your state machine. This ensures that every `Enum constant` in your state machine has a consistent list of values that is easy to update.

RELATED INFORMATION

Common State Machine Transition Code

Transition code determines which case to execute in the next `While Loop` iteration. You can use the transitions detailed below as a starting point for the transition code in your program.

Table 3 : One State Transition Value

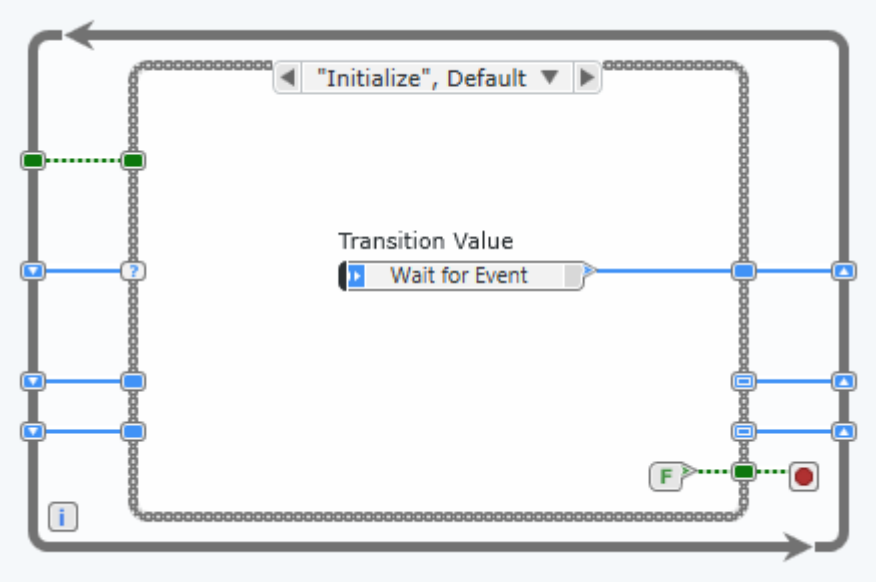
State Transition	Example Image
Use an `Enum constant` transition when you always want to transition to a specific state after the code in the current state executes. For example, the Initialize and Exit states in a state machine often use an `Enum constant` transition.	

Table 4 : Two State Transition Values

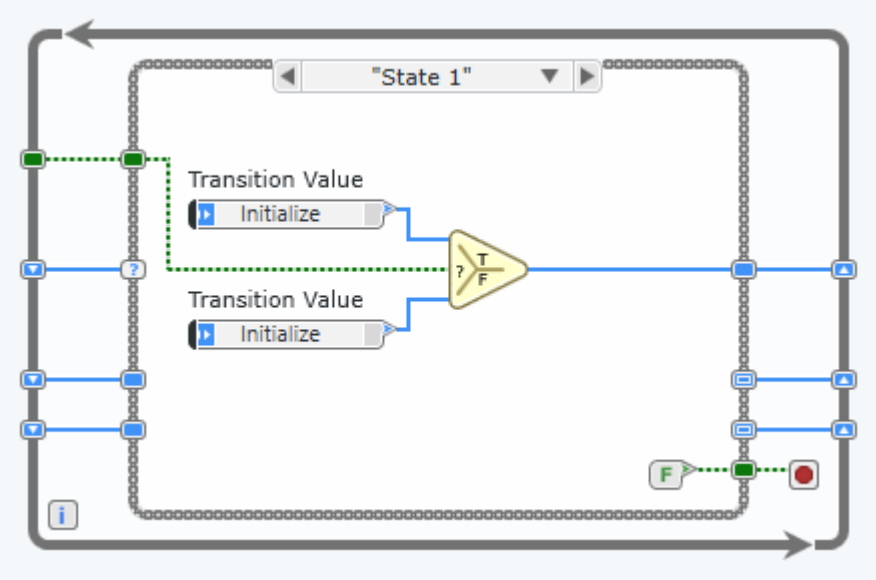
State Transition	Example Image
Use a `Select` node and a Boolean value to choose between one of two transition values.	

Table 5 : Three or More State Transition Values

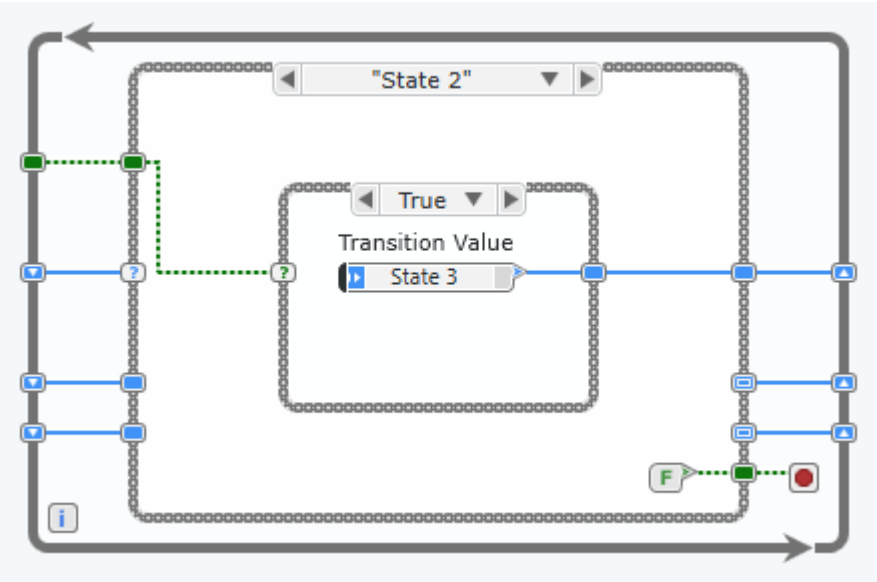
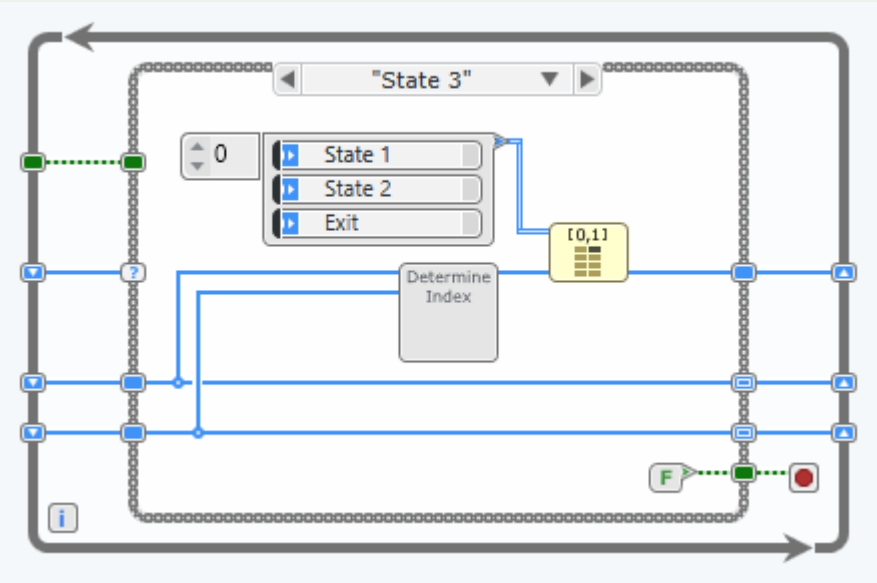
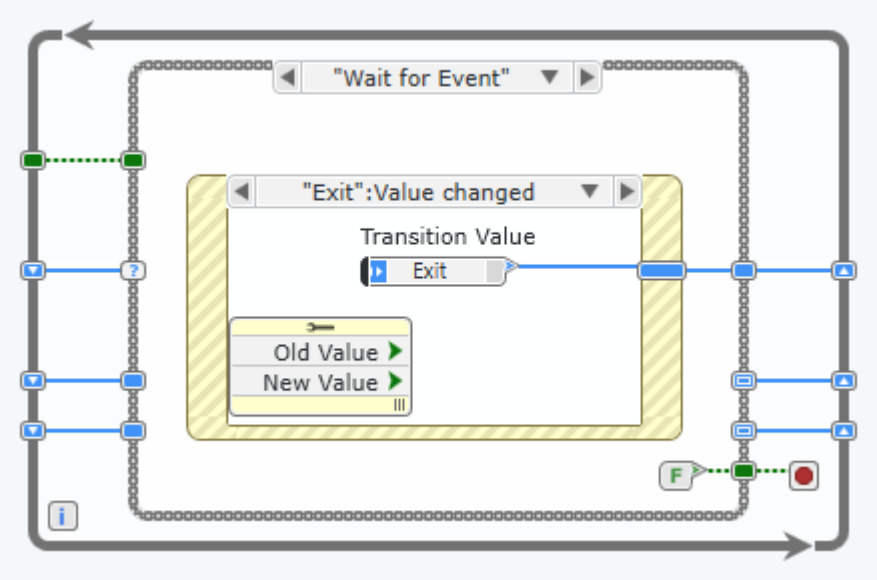
State Transition	Example Image
Use a `Case Structure` transition when you need to handle a variety of situations. Add a case for each transition you need to program and create transition code in each case to adjust the transition value when the case executes. Each case in the `Case Structure` can include a single transition value or multiple values that the program must choose between.	
Use a transition array when you need to choose between more than two transition values at one time. At run time, an `Index Array` node selects a transition value from an array constant containing each possible state transition value.	

Table 5 : Three or More State Transition Values (Continued)

State Transition	Example Image
Use an `Event Structure` when you need to change states in response to a user action in your program. The `Event Structure` monitors the program for events and executes a subdiagram of code when a specific event occurs. Each event in the `Event Structure` can include a single transition value or multiple values that the program must choose between. For example, the Wait for Event state in the simple state machine template uses an `Event Structure`. **NOTE** If you need to perform an action periodically in the Wait for Event state, such as updating a timer, you can include an application timeout event in the `Event Structure`.	

RELATED INFORMATION

Best Practices for Creating Projects in G Web Development Software

To develop projects that successfully serve your needs and the needs of your end users, refer to the following best practice guidelines:

- *File and Project Organization for Projects in G Web Development Software*
- *Icons and Connector Panes for Projects in G Web Development Software*
- *Panel Design for Projects in G Web Development Software*
- *Diagram Design for Projects in G Web Development Software*
- *Other Best Practices for Projects in G Web Development Software*

File and Project Organization in G Web Development Software

Refer to the following table for best practices for organizing files and projects in G Web Development Software.

Guideline	Required or Recommended?	Details	Example(s)
Avoid using special characters in file names.	Recommended	Using special characters in file names can cause compatibility concerns across platforms.	Avoid the following: • File separators such as colons, forward slashes, and backward slashes • Non-alphabetical and non-numerical symbols, such as the trademark symbol • Punctuation marks, such as parentheses, quotation marks, brackets, and operators • White space characters, such as tabs and new lines **NOTE** You can use spaces for most applications, but avoid using spaces in a top-level `.gviweb` to create a human-readable URL.

Guideline	Required or Recommended?	Details	Example(s)
Make sure your project organization is logical and easy to use.	Recommended	• Create a hierarchical structure with easily accessible top-level VIs. • Place support VIs in folders within the project and group them to reflect modular components, such as instrument drivers, other drivers, and configuration utilities. • Limit the number and the levels of directories you use in a project.	N/A

RELATED INFORMATION

Icons and Connector Panes for G Web Development Software Projects

Refer to the following table for best practices for creating icons and connector panes in G Web Development Software.

Guideline	Required or Recommended?	Details	Example(s)
Consider using the well-designed icons in the G Web Development Software libraries as prototypes for your icon.	Recommended	If you cannot find or create a picture to use for an icon, you can use text.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Create a uniform icon style for all related VIs.	Recommended	A uniform style helps users visually determine which subVIs are associated with a top-level VI.	N/A
Avoid using colloquialisms when making icons.	Recommended	Colloquialisms, even in pictures, are difficult to translate. Users that speak languages other than English may not understand a picture that relies on cultural background knowledge.	If you represent a data logging VI with a picture of a lumberjack, some users may not know the cultural reference to understand how a lumberjack can represent data logging.
Create a meaningful icon for every VI.	Recommended	The icon represents the VI on a palette and diagram. Well-designed icons help users gain a better understanding of the subVI without the need for excess documentation.	N/A
Consider common node sizes in LabVIEW NXG when designing an icon.	Recommended	Refer to the following common node sizes: • 30x30 for small nodes. This size supports the 3-1-1-3 connector pane pattern. • 40x40 for the default VI size. This size supports the 4-2-2-4 connector pane pattern. • 50x50 for the default size used by NI hardware driver and Advanced Analysis Library APIs. This size supports the 5-3-3-5 connector pane pattern.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Make sure to set inputs and outputs for each subVI as required, recommended, or optional in the connector pane for that subVI, and make sure the settings make sense for your project.	Recommended	Use the **Usage** settings on the **Item** tab for the connector panes of all subVIs to specify the input and output settings for each subVI. The **Usage** setting for connector pane terminals affects the appearance of the inputs and outputs in the **Context Help** and reminds users to wire subVI connections. Use the required setting for inputs that users must wire for the subVI to run properly. Use the optional setting for inputs that have default values that are appropriate for the subVI the majority of the time. Use the recommended setting for all other inputs.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Assign inputs and outputs according to the way a user will eventually wire VIs together.	Recommended	Wire inputs on the left and outputs on the right of the connector pane because standard data flow moves from the left to the right. Consistency between the location you assign inputs and outputs in the connector pane and their location in the actual data flow promotes ease of use and reuse.	• If you create a group of subVIs that you often use together, give the subVIs a consistent connector pane with common inputs in the same location to help you remember where to locate each input. • If you create a subVI that produces an output that another subVI uses as an input, such as references, task IDs, and error clusters, align the input and output connections to simplify the wiring patterns. • Assign two inputs of a VI to the left two terminals of the corresponding connector pane and two outputs of that VI to the right two terminals of the corresponding connector pane.
Reserve the bottom left and right connector pane terminals for the error input and error output.	Recommended	Session-based APIs are the only exception to this recommendation. For session-based APIs, you can place error inputs and outputs directly beneath session and reference inputs and outputs.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Avoid creating connector panes with more than 16 terminals.	Recommended	Including too many terminals can make a subVI difficult to understand. You can consider either splitting the functionality of the subVI into multiple subVIs or using clusters to create logical groupings of the inputs to the subVI, depending on which solution makes sense for your project.	N/A
If your subVI includes a pass-through input and output pair, add an *in* suffix to the control and an *out* suffix to the indicator.	Recommended	Adding these suffixes to pairs of inputs and outputs indicates a relationship between the inputs and outputs.	If you name an input reference `in`, name the related output reference `out`.
If your subVI includes a pass-through input and output pair, and you wire the control directly to the indicator on the diagram to prevent the value from changing, remove the indicator from the connector pane.	Recommended	You can exclude references from this guideline.	N/A

Panel Design for G Web Development Software Projects

Refer to the following table for best practices for designing panels in LabVIEW NXG

Guideline	Required or Recommended?	Details	Example(s)
Avoid using all capital letters in labels or panel documentation.	Recommended	Capital letters can make text seem more important than necessary.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Position the panel in the top left, spatially even with the controls palette.	Recommended	If you position the panel in the top left, you can prevent the user from opening the panel to a position that is potentially off the screen or otherwise difficult to see and read.	N/A
Display the labels of all controls and indicators on the panel.	Recommended	Make sure the labels of all controls are meaningful to increase clarity and ease of use for users.	N/A
Set reasonable default values for controls. **NOTE** Default values you set for controls automatically append to the terminal name. The **Context Help** displays the default value when you hover over the control.	Recommended	Make sure the default values you set do not generate errors when you run the top-level VI. **NOTE** When possible, avoid setting default values for indicators such as graphs, arrays, and strings. Setting default values for those types of indicators wastes disk space when you save the VI.	N/A
Use default values strategically and logically.	Recommended	Planning the use of default values can save space and simplify code.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Avoid displaying labels on the panel for buttons that display Boolean text.	Recommended	Only display the Boolean text for these buttons. **NOTE** If you click the boolean text of a checkbox, the value of that checkbox toggles. The value of that checkbox does not toggle if you click the label.	N/A
Format any text on your panel appropriately.	Recommended	• Use default fonts when possible. • When the alignment of characters is critical, use monospace fonts for string controls and indicators, and space the characters equally. • For free labels, only use carriage returns to separate paragraphs. • Resize labels to enable word wrapping. • Include extra space in free labels to allow for longer or larger strings due to font differences in localized languages.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Group and arrange controls logically and aesthetically.	Recommended	• Consider the arrangement of controls on panels, and keep panels simple to avoid confusing users. • For top-level VIs that users can see, place the most important controls in the most prominent positions. • For subVI panels, place controls and indicators of the subVI to correspond with the connector pane pattern. **NOTE** Regardless of the location of the error cluster connector pane connection, place error cluster controls and indicators at the bottom of the panel.	N/A
Use the **Align** and **Distribute** options in the **Layout** pull-down menu to create a uniform layout.	Recommended	N/A	N/A

Guideline	Required or Recommended?	Details	Example(s)
Visually group objects with related functions.	Recommended	• Use decorations from the `Drawings` palette. • Use clusters to group related data. Avoid using clusters for only aesthetic purposes.	N/A
Configure path inputs and outputs appropriately.	Recommended	• Use path controls instead of string controls to specify the location of files or directories. Path controls and indicators work similarly to strings, but the software formats paths using the standard syntax for the platform you are using. • Avoid hiding the **Browse** button on path controls. • Set the browse action correctly for the **Browse** button on path controls in the **Item** tab.	If you set a browse action in which a user needs to select a directory, select **Select Folder** from the **Action** drop-down menu in the **Item** tab.
Determine whether an enum or ring is more effective than a Boolean.	Recommended	An enum may be more efficient in cases where two states may increase to more states.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Arrange items in a cluster vertically.	Recommended	To arrange items vertically, select **Vertical** from the **Arrange** drop-down menu in the **Item** tab.	N/A
Use imported graphics to enhance the panel. **NOTE** This guideline is specific to user interfaces.	Recommended	Import graphics and text objects to use as panel backgrounds by dropping URL `Image` from the `Decorations` palette and navigating to the image on the **Item** tab.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Use color logically, sparingly, and consistently, if at all. **NOTE** This guideline is specific to user interfaces.	Recommended	• Never use color as the sole indicator of device state. • Use a minimal number of colors, emphasizing black, white, and gray. Color can be difficult to discern or distract the user from important information. • Use light gray, white, or pastel colors for backgrounds. Use bright, highlighting colors only when a term is important, such as an error notification. • Because multi-plot graphs and charts can lose meaning when displayed in black and white, use different plot styles, such as dots, and dashes, in addition to color to further differentiate multiple plots.	• A yellow, green, or bright orange background makes it difficult to see a red danger light. • People with some degree of color-blindness can struggle to detect certain color changes. You can upload an image of your panel to an online color blindness simulator to test your panel under various degrees of color blindness.

RELATED INFORMATION

Diagram Design for G Web Development Software Projects

Refer to the following table for best practices for designing diagrams in G Web Development Software.

Guideline	Required or Recommended?	Details	Example(s)
Use a type definition when you use the same unique control in more than one location or when your project includes a large data structure that passes between several subVIs.	Required	A type definition automatically propagates changes to the control or data structure throughout all relevant VIs and subVIs.	N/A
Select **Autosize list view** in the **Item** tab for any properties nodes, such as `Cluster Properties` or `Waveform Properties`.	Required	The **Autosize list view** option ensures that all element names are fully visible.	N/A
Make sure that data flows from left to right and that wires enter the diagram from the left and exit to the right.	Recommended	Although the positions of program elements do not determine execution order, avoiding right-to-left wiring helps users read and comprehend the diagram. **NOTE** Only wires and structures determine execution order.	N/A
Avoid creating any backwards wires.	Recommended	Wires that cause data to flow from right to left contradict style best practices.	N/A
For subVIs and top-level VIs that do not contain loops, place controls on the far left and indicators on the far right.	Recommended	Placing controls and indicators in these positions increases readability and usability by giving users a familiar and expected location to look for the controls and indicators.	N/A

Guideline	Required or Recommended?	Details	Example(s)
For non-user interface subVIs, make sure all controls and indicators that are on the connector pane also reside on the top-level diagram.	Recommended	N/A	N/A
When you use enumerated type constants, always create G Types of those constants.	Recommended	• Enumerated type constants are useful for making diagram code easier to read. When you wire an enumerated type constant to a `Case Structure`, the string labels appear in the selector label of the `Case Structure`. • G Types prevent you from needing to rewrite code each time you add or remove an item from an enumerated type constant.	N/A
Use a docked constant if the only purpose of the constant is to define a data type and the value is unimportant.	Recommended	Docked constants that contain meaningful values can inhibit the readability of the diagram. If a docked constant is set to the default value, it will appear hollow. If any other value is used, it will be a solid color.	N/A
If you display the list view of a node, make sure the entire name of the longest item in the node is visible.	Recommended	Also, resize the node for shorter names to reduce any extra space.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Use list view for all nodes imported with the Jenkins Shared Library Interface (JSLI) document type, and show the node label.	Recommended	List view helps users distinguish between JSLI nodes and subVI nodes and also improves readability.	N/A
If you wire an enum to a `Case Structure`, make sure none of the cases are marked as the default case.	Recommended	One exception to this guideline is if the `Case Structure` only operates on one or a small number of the overall number of items in the enum.	N/A
If you wire a ring, string, or numeric to a `Case Structure`, remove any additional values from the string in the `Case Selector Label`.	Recommended	Removing additional values increases clarity.	See the *Case Values of a Case Structure* section following this table for a visual example.
If possible, avoid using ellipsis notation for enum values.	Recommended	Enumerate all the values so that if you add a new value later, the diagram breaks and you can consciously handle the new enum value. Use range notation sparingly.	N/A
If your VI includes its own clusters or enums on the diagram, define them with type definitions.	Recommended	Type definitions allow you to manage these data types in one place and ensure that any changes you make to a data type propagate across all relevant VIs and subVIs.	N/A
Use comments to document the functionality of your code.	Recommended	N/A	N/A
Reference applicable VIs in comments instead of showing the labels on all subVIs in a diagram.	Recommended	When you display object labels, you cannot reposition them, so your diagram width must increase unnecessarily.	N/A

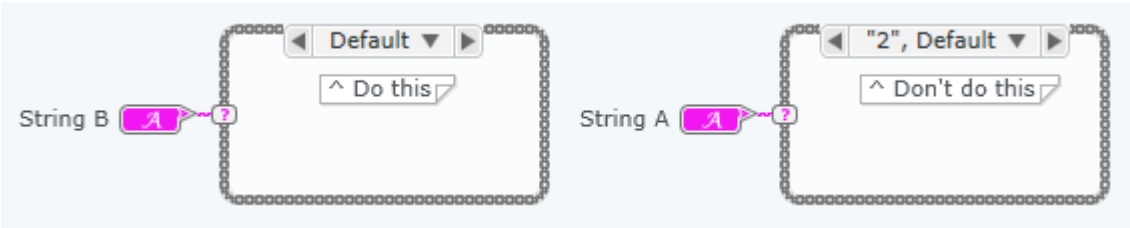
Guideline	Required or Recommended?	Details	Example(s)
Consider whether or not your VI needs an error `Case Structure` surrounding its contents.	Recommended	If the error `Case Structure` is not handling critical running code, consider simply passing the error wire through all diagram nodes without creating a case around the whole diagram. You may want to use an error case if your VI is manipulating the error cluster at all. Using an error case allows you to case out the relevant diagram so that your error manipulation code does not modify any error entering the VI.	High iteration loops and modal dialogs are examples of critical running code.
Arrange diagrams so that the primary wire travels in a straight line between nodes.	Recommended	The primary wire is typically a reference or error wire.	N/A
Keep approximately 20 pixels of white space between objects.	Recommended	An overcrowded diagram without adequate white space is difficult to read.	N/A
Expand array constants to display all array elements, plus one empty element at the end.	Recommended	Displaying all array elements with an empty space at the end helps users see the entirety of the array. If you do not have enough space for a large array constant, display the scrollbar.	N/A
Use a comment to label long wires and identify their use.	Recommended	Wire comments are useful for identifying and describing wires that come from shift registers and long wires that span the entire diagram.	N/A
Avoid bending wires when possible, and keep wires short.	Recommended	Wires with long, complicated paths are difficult to follow. **NOTE** Avoid replacing long wires with local or global variables.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Align and distribute nodes, terminals, and constants.	Recommended	When you align and distribute objects evenly, you can use straight wires to connect objects and create readable diagrams.	N/A
Avoid placing diagram objects, such as subVIs or structures, on top of wires, and avoid placing any wires under diagram objects because the software can hide certain segments of the resulting wire.	Recommended	Draw wires so that you can clearly see if a wire connects to a terminal. Avoid wiring through structures if the structure does not use the data in a wire.	N/A
Make sure control and indicator terminals on the connector pane do not reside inside structures on the diagram.	Recommended	The development environment can optimize subVI execution time when all controls and indicators on the connector pane reside on the top-level diagram of the VI.	N/A
Save the VI with the most important frame of multi-framed structures, such as a `Case Structure`, displayed.	Recommended	Saving the VI with the most important case displayed increases readability because the user does not have to switch cases immediately.	If you open a diagram and the error case that wraps the entire diagram is displayed first, you must switch to the no error case to see the actual diagram code.

Guideline	Required or Recommended?	Details	Example(s)
Maximize the performance of code on the diagram.	Recommended	When a program has large arrays or critical timing problems, you can use the following guidelines to maximize the performance of the VI: • If possible, avoid building arrays using `Build Array` within a loop because the node makes repetitive calls to the memory manager. Instead, use auto-indexing or pre-size the array and use `Replace Array Subset` to place values in the array. Also avoid using `Concatenate Strings` with strings because, in memory, the software handles strings as arrays of characters. • Choose the proper array data type to control the memory usage of the application. • Display only necessary information on the panel. Send data to indicators only if the data is different from what the indicator already displays. Frequently updating panel indicators with new data can affect the performance of the VI, especially if you display large amounts of data in graphs or charts.	Array data types can affect the memory usage of an application. For example, if your double-precision, floating-point array of 100,000 values is actually storing single-precision, floating-point values, you are using memory inefficiently. In this case, use an array of single-precision, floating-point values to match the data type stored in the array and reduce the memory usage.

Guideline	Required or Recommended?	Details	Example(s)
If your VI includes several I/O name controls, such as `Waveform` and `Digital Waveform`, that are stacked vertically, configure them to not show type.	Recommended	If you set I/O name controls to not show their type, you can maximize information density and avoid overlapping the controls.	N/A

Case Values of a Case Structure



RELATED INFORMATION

Best Practices for Designing and Developing an Application Programming Interface (API) in G Web Development Software

To develop an API to distribute to other users that is consistent with NI style recommendations for G content, refer to the following best practice guidelines:

- *File Organization and Node Naming*
- *Component Organization*
- *Icons and Connector Panes*
- *Panel Design*
- *Data Type Selection*
- *Palette Taxonomy*
- *Documentation*
- *Error Message Design*
- *API Design*

You also need to follow the ***Best Practices for Designing and Developing Projects in G Web Development Software*** if you want your API to be consistent with NI best practice recommendations for G content.

Throughout this section, the term *required* refers to a guideline that NI requires when designing an API. The term *recommended* refers to a guideline that NI suggests when designing an API.

File Organization and Node Naming for Distributed APIs

Refer to the following table for best practices for organizing files and naming nodes.

Guideline	Required or Recommended?	Details	Example(s)
Use title case.	Required	Capitalize the following in subVI names: • The first and the last word • All nouns, pronouns, adjectives, verbs, adverbs, and subordinate conjunctions (*as*, *because*, *although*)	*General Error Handler*, not *General error handler*
Capitalize the second part of a hyphenated compound.	Required	N/A	*Fixed-Point*, not *Fixed-point*
Do not capitalize articles (*a*, *an*, *the*), coordinate conjunctions (*and*, *or*, *nor*), or prepositions regardless of length, unless they are the first or last word.	Required	Boolean operators are an exception. Write Boolean operators, such as *AND* and *OR*, in all capital letters.	*Search and Replace*, not *Search And Replace*
Do not capitalize *to* in an infinitive phrase unless *to* is the first word node name.	Required	N/A	*Path to String*, not *Path To String*; *To String*, not *to String*.
Use industry-standard capitalization for an engineering or scientific term.	Required	N/A	*PolyBezier*, not *Polybezier*
Write acronyms in all capital letters.	Required	N/A	*FIFO*, not *Fifo*, *URL*, not *Url*

Guideline	Required or Recommended?	Details	Example(s)
Use spaces between words.	Required	N/A	*Feedback Node*, not *FeedbackNode*
Do not use special characters.	Required	Boolean operators are an exception. Use a question mark (?) for node names when the primary output is Boolean, such as *Equal?*, not *Is Equal*.	*Search and Replace*, not *Search & Replace*
Make sure none of the nodes or VIs in your API share the same name with a node or VI that is already in the LabVIEW NXG palettes.	Required	Unique node and VI names in an API make unique palette object names for Quick Drop users.	N/A
Do not use the forbidden word, *Example*, in the names of your API nodes or VIs.	Required	N/A	N/A

Component Organization for Distributed APIs

Refer to the following table for best practices for organizing components.

Guideline	Required or Recommended?	Details
Set the `Exported` or `Non-exported` designation of the VIs in your API appropriately.	Required	The `Exported` designation indicates to users which VIs are guaranteed to have a consistent interface.
Mark a VI as `Exported` only if it is a public member of your API.	Required	Mark a VI as `Exported`.
If a G Type is on the connector pane of an exported VI in your API, also mark that G Type `Exported`.	Required	N/A

Guideline	Required or Recommended?	Details
Follow the namespacing guideline `[Company].[Product].[Component].gcomp` for your API component.	Required	For example, `NI.GCore.DataType.gcomp` is a component NI owns, part of the G Core product (in other words, the core libraries that compose the G language), and these VIs pertain to parsing Data Types.

RELATED INFORMATION

Best Practices for Designing and Developing an Application Programming Interface (API) in G Web Development Software on page 244

Icons and Connector Panes for Distributed APIs

Refer to the following table for best practices for creating icons and connector panes.

Table 6 : Guidelines for SubVI Size

Guideline	Required or Recommended?	Details
Make all VIs or as many VIs as possible the same size.	Required	The icon size is determined by the connector pane pattern that the subVI requires. Refer to the icon and connector pane guidelines in *Icons and Connector Panes for G Web Development Software Projects* for more information.

Table 6 : Guidelines for SubVI Size (Continued)

Guideline	Required or Recommended?	Details
When resizing a VI, expand it in one direction.	Required	The direction a VI can stretch depends on `error` input and output location on the connector pane. • If the `error` input and output are beneath the instrument handles, as with driver APIs, vertically expand the VI. • If the `error` input and output are on the lower-left or lower-right of the node, horizontally expand the VI. Increase the size of the entire API to be more consistent, if necessary.

Table 7 : Guidelines for Iconography

Guideline	Required or Recommended?	Details	Example(s)
Make the node size big enough to communicate its functionality through an icon.	Required	The icon should communicate node behavior.	N/A
Make consistent iconography for all the VIs in your API.	Required	Default glyphs are in the G Web Development Software project on the **Icon** tab. You can use the default glyphs as a starting point in your VIs.	N/A

Table 8 : Guidelines for Interface Elements

Guideline	Required or Recommended?	Details	Example(s)
Include pass-through wires in the upper corners of a VI only if it is part of a reference-based API.	Required	A reference value is not modified during execution, so passing it through VIs in an API is convenient. A VI could potentially modify a data value, in other words, any wire that is not a reference or a cluster/class of references, if it is passing through. Pass data values through VIs only if the VI might modify the value of the wire.	Reference-based APIs in G Web Development Software include queue and HTTP.
Either coerce numeric inputs into a range that your code expects, or handle out-of-range values with an error condition or warning.	Required	Do not configure data limits for numeric controls.	If you specify the speed of a motor, and the motor can go only 0-100 mph, make sure you are either coercing the input value with an `In Range and Coerce` function to be within 0-100, or return an error if you specify a value outside that range.
Use lower case for input and output names.	Required	N/A	Use: *exported waveform* Do not use: *Exported Waveform*
Use input and output names that are easy to understand.	Required	N/A	Use: *remainder* Do not use: *x-y*floor(x/y)*
Name inputs and outputs consistently.	Required	N/A	For three inputs that have the same data type and source type, use: *task in, task in, task in*. Do not use: *task in, myDAQ task in, myDAQ resource in*

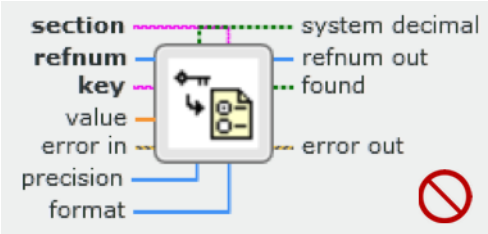
Table 8 : Guidelines for Interface Elements (Continued)

Guideline	Required or Recommended?	Details	Example(s)
Use simple words, not symbols.	Required	N/A	Use: *number of samples* Do not use: *# of samples*
Use a question mark (?) for input and output names when the data type is Boolean with an implied question.	Required	N/A	Use: *UTC?* Do not use: *is UTC*
Do not use a question mark (?) for Boolean inputs and outputs that are commands.	Required	N/A	Use: *reset* Do not use: *reset?*
Do not include default values in parentheses.	Required	Set default values in the configuration panel. G Web Development Software automatically appends the default values to the label.	Use: *max characters per row* Do not use: *max characters/row (no limit:0)*
Do not include units of measure in parentheses.	Required	Set units in the configuration panel. G Web Development Software automatically appends the units to the label.	Use: *delay time* Do not use: *delay time (s)*
Make sure the names of inputs and outputs of your VIs use complete English words, unless users worldwide know an abbreviation.	Required	N/A	N/A
If you use the top or bottom inputs and outputs on the connector pane, make sure their wires never cross each other.	Required	N/A	See the *Avoid Crossing Wires* section at the end of this table for a visual example.

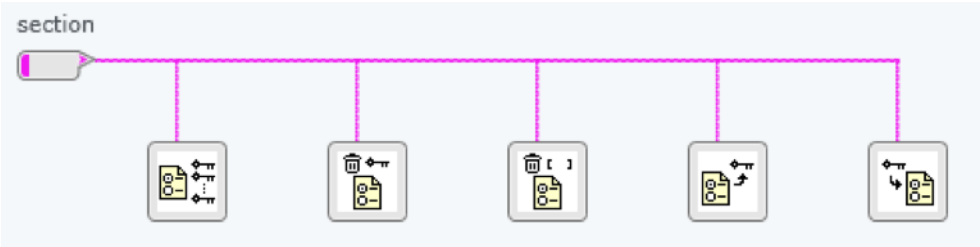
Table 8 : Guidelines for Interface Elements (Continued)

Guideline	Required or Recommended?	Details	Example(s)
Assign the input and output terminals on the connector pane in a way that simplifies the wiring diagram.	Required	Consider how users might wire the VIs together when you assign terminals. Align the output terminals with the corresponding input terminals.	N/A
If multiple VIs in your API have the same input or output, place all similar inputs and outputs in the same location on the connector pane of all VIs.	Required	N/A	See the *Locate Inputs and Outputs Consistently* section at the end of this table for a visual example.
Do not leave any terminals of your public API VIs in the unplaced items tray.	Recommended	When all controls/indicators are on the panel, the user can open the panel of your VI and see input and output values during debugging.	N/A
Set the display format of numeric controls and indicators to produce reasonable format settings for controls and indicators that users create from them.	Recommended	The display format settings of a source numeric control or indicator are passed on to any controls and indicators created from them.	N/A

Avoid Crossing Wires



Locate Inputs and Outputs Consistently



RELATED INFORMATION

Icons and Connector Panes for G Web Development Software Projects on page 227

Best Practices for Designing and Developing an Application Programming Interface (API) in G Web Development Software on page 244

Data Type Selection for Distributed APIs

Refer to the following table for best practices for selecting data types.

Guideline	Required or Recommended?	Details	Example(s)
For numeric input or output values, use double-precision for floating-point values and I32 for integer values unless there is a specific reason to use another type.	Required	N/A	N/A
Always use an error cluster for reporting error conditions. Never use an error code or error Boolean output by itself.	Required	G Web Development Software has a single Error API for error handling. If API VIs do not use the error cluster to convey error information, you cannot use the Error API for handling errors.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Use a text ring only for parameters that have a natural association to several discrete possibilities and no numeric content.	Recommended	An exception for numeric content is if the set of valid values is sufficiently small.	Examples of discrete possibilities without numeric content are days of the week, months of the year, make and model of car. An example of numeric content that would be an exception is the number of connected devices within a finite number of connections. It's better for a user to pick a value of 0, 1, 2, 3 from a ring control than to instead have a numeric control and need to make sure the specified value is in range.
If you use text rings for numeric content, do not create an item with text that represents a different numeric value.	Recommended	N/A	For example, a selection of 2 assigned to the value 0 could confuse the user.
Use the timestamp data type for time values in your API.	Recommended	N/A	N/A
Use a double-precision value to specify the number of seconds for timeout values in your API. Use -1 to indicate that the diagram object has no timeout limit.	Recommended	N/A	N/A
Use the waveform data type when your VI acquires or generates analog waveforms.	Recommended	N/A	N/A

Guideline	Required or Recommended?	Details	Example(s)
If your VI has a string input or output, consider whether another data type, such as an enum, might make your API easier to use.	Recommended	N/A	If the user needs to pick a day of the week, an enum with the seven days in it is easier to use than typing the name of the day as a string, because of possible problems with the case, spelling, and abbreviation.
If you use a cluster as an input or output of your VI, make sure the cluster logically groups parameters.	Recommended	Do not use clusters only to reduce the number of inputs and outputs on your VI, because clustering things unnecessarily to reduce the number of inputs to the VI is not a logical organization.	N/A

RELATED INFORMATION

Best Practices for Designing and Developing an Application Programming Interface (API) in G Web Development Software on page 244

Palette Taxonomy for Distributed APIs

Refer to the following table for best practices for palette taxonomy.

Guideline	Required or Recommended?	Details	Example(s)
Make the root palette easy to browse.	Recommended	Seven to 10 categories allows for full-size icons in a single-column layout.	N/A

Guideline	Required or Recommended?	Details	Example(s)
Design a structure that can expand beyond the original size of the root palette for nodes you may develop in the future.	Recommended	N/A	N/A
Make palettes for similar categories consistent.	Recommended	N/A	Across different palettes, such as numeric, string, file, and so on, that have a constants palette, similarly organize all the constants palettes.
Add synonyms or keywords to your palettes to improve the findability of your VIs.	Recommended	N/A	Mathematicians commonly refer to the functionality of the `Quotient and Remainder` node as *mod*. Make sure *mod* is a keyword for `Quotient and Remainder` so that a search for *mod* returns that node.

RELATED INFORMATION

Best Practices for Designing and Developing an Application Programming Interface (API) in G Web Development Software on page 244

Documentation for Distributed APIs

Refer to the following table for best practices for documenting an API.

Guideline	Required or Recommended?	Details
Review the VI description and parameter descriptions of all public VIs in your API for correct meaning for what the VI does, usability by third parties who are unfamiliar with your API, spelling, grammar, and naming conventions for G Web Development Software.	Required	N/A

RELATED INFORMATION

Best Practices for Designing and Developing an Application Programming Interface (API) in G Web Development Software on page 244

Error Message Design for Distributed APIs

Refer to the following table for best practices for designing error messages.

Guideline	Required or Recommended?	Details
Reserve a range of error codes for your API and use those codes for error conditions specific to your API.	Required	Pick a range for error codes between -8999 through -8000, 5000 through 9999, and 500,000 through 599,999, which are reserved for external users.
When generating or manipulating errors on the diagram of your API VIs, make sure you use the Error API to construct and manipulate error clusters.	Required	Never bundle or unbundle error cluster elements directly in G Web Development Software.
If the VI has no way to generate errors, consider excluding error inputs and outputs.	Recommended	If a VI does not generate errors, users of the VI can take advantage of parallelism in G Web Development Software.
If you exclude error inputs and outputs on a VI in your API, leave the error input and output locations on the connector pane empty.	Recommended	When unused error inputs and outputs on a VI are left empty, you can add error handling to the VI later.

RELATED INFORMATION

Best Practices for Designing and Developing an Application Programming Interface (API) in G Web Development Software on page 244

API Design for Distributed APIs

Refer to the following table for best practices for designing VIs.

Guideline	Required or Recommended?	Details	Example(s)
Include every public VI in your API in the API palette.	Required	Users may not look on disk to find advanced VIs. If a VI is not ready or intended for public consumption, make the VI private in the API component.	N/A
Do not use a VI in your API to open a dialog box unless that is the stated intent of the VI.	Required	N/A	N/A

Guideline	Required or Recommended?	Details	Example(s)
If you expect multiple calls of your API VI to run in parallel, consider making the VI reentrant.	Recommended	If the VI stores any internal state in feedback nodes or uninitialized shift registers, make the VI reentrant and *stateful*, such as a preallocated clone. If the VI doesn't store any internal state, make it *stateless*, as a shared clone.	N/A

RELATED INFORMATION

Best Practices for Designing and Developing an Application Programming Interface (API) in G Web Development Software on page 244

Testing and Debugging

When the results of your application are not what you expect, use a set of tools to determine where errors occur within your code.

Although errors are often detected automatically, sometimes your code can run successfully but not as intended. When this happens, you need to identify the source of the unintended behaviors.

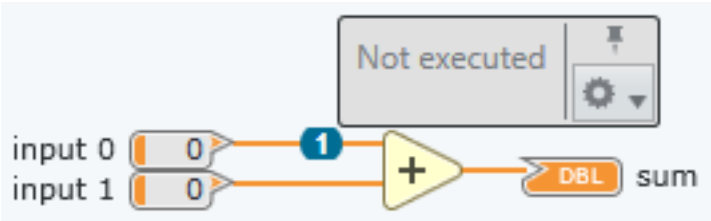
The following debugging tools can help you in this process:

- Probes
- Breakpoints

Using Probes to Check Values on a Wire

When your WebVI is running in the editor environment, you may want to check the values on the wires to determine if and where any unexpected data occurs.

To place a probe on a wire, right-click on the wire and select **Add Probe**. A probe displays the data from the wire it is on. To view a probe in a temporary overlay on the diagram, hover over the probe marker on a wire. If desired, you can then click the **Pin** button to keep this probe visible. From the **Options** drop-down you can choose the different display styles and also remove the probe, as required.



You can also find probes in the **Debugging** tab on the left side of the editor environment. The order in the list correlates with the numbers in the probe markers on the wires.

Probes display the most recent value carried by a wire. You can hover over the probes in the list and observe the tip strip that specifies when the value was last updated.

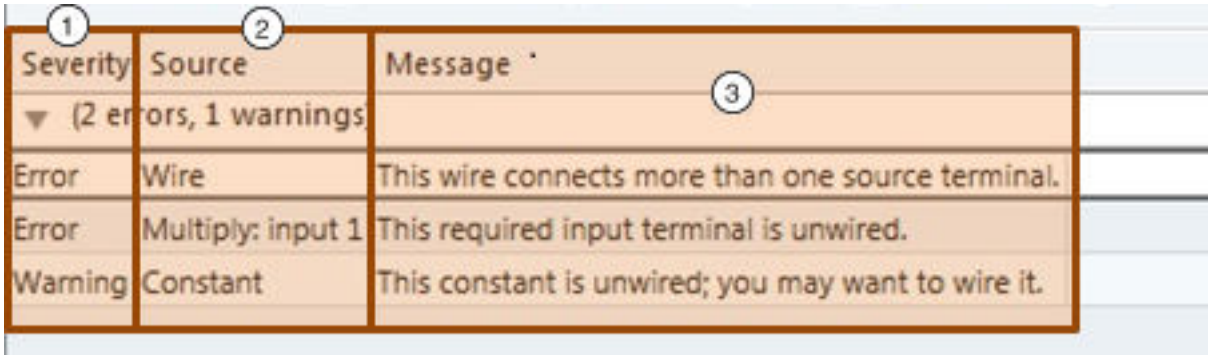
The values displayed in the probe list are limited to a line of text. However, you can hover over a probe and click the **>** button to display an expanded view of the probe and the **Options** drop-down.

Identifying Errors That Prevent You from Running Code

As you create code, a broken **Run** button communicates that the code contains errors that prevent it from running.

You can use the provided error and warning messages to help fix these problems. *Errors* break the code. You must resolve any errors before you can run the program. *Warnings* do not prevent you from running the code. They are designed to help you avoid potential problems in the program.

To identify the specific errors, click the broken **Run** button to display the **Errors and Warnings** tab. The following image highlights sections of the **Errors and Warnings** tab that define detected errors and warnings.



①	**Severity**—Denotes whether an issue is an error or a warning.
②	**Source**—Identifies the object that is causing the error or warning.
③	**Message**—Provides more detail about why the error or warning exists.

RELATED INFORMATION

Improve Applications with Execution Logs

Well-written applications use execution logs for both activity audit and monitoring. An execution log makes it easy for a developer or user to track and identify issues that occur throughout the application without excessive effort. Maintaining an execution log allows you to review all activity and make improvements to your application accordingly.

An application should record data from multiple execution points throughout an application—errors, exceptions, successful execution, and so on. An execution log can help you recreate a certain behavior and improve your application effectively. For example, you can use the log to do one or more of the following:

- Modify the code to be able to handle a situation that caused an error
- Modify the code based on the most common use case
- Update the documentation for the application so that users interact with the application in a more predictable way
- Validate successful execution

The execution log should include as much information as you need to understand the state of the product at the time the data was collected. You might include one or more of the following components in each entry of an execution log:

- The current time and date
- A pre-defined category for the activity
- A running count of the specific type of activity
- A description of the activity
- The value of data that was generated during or prior to the activity
- The location of the activity within the application
- The type and severity of the problem, if there is one

You can use the String nodes to generate, combine, and write the text for each entry of an execution log to one or more text files. A log file should be human- and computer-readable, but you can format your file in a way that fits the needs of your application.

Viewing the Hierarchy of VIs in Your Application

Open the Call Hierarchy document to see the hierarchy of VIs and subVIs and understand the calling relationships between them in your application.

For example, you can use the Call Hierarchy document to see all of the VIs that call a specific VI and the subVIs that specific VI calls. This can be helpful when you need to debug a part of your application and want to see which subVIs are connected to each other.

Procedure

1. Click **View**»**Call Hierarchy** to open a Call Hierarchy document for your project.

   NOTE
   Some callers may be hidden. Toggle the triangle icon below a node to show or hide callers.

2. (Optional) Click **Clean up diagram** to change the visual arrangement of the hierarchy.
3. (Optional) Select a subVI to highlight its calling connections.
4. (Optional) Double-click a subVI displayed in the Call Hierarchy document to open it.

RELATED INFORMATION

Standard Error Behavior

Many nodes provide **error in** and an **error out** parameters so that the node can respond to and communicate errors that occur while code is running. When you create an application, wire all error inputs and outputs of each node on the diagram so that error information moves through the application in the same way as data values. As your VI runs, each node with error-checking capabilities monitors for error information that is passed to the error input from the previous node. Since each node with error inputs and outputs does the same thing, allowing only valid data or error information to flow through the diagram.

The value of **error in** specifies whether an error occurred before the node runs. Most nodes respond to values of **error in** in a standard, predictable way:

error in does not contain an error	**error in** contains an error

If no error occurred before the node runs, the node begins execution normally. If no error occurs while the node runs, it returns no error. If an error does occur while the node runs, it returns that error information as **error out**.	If an error occurred before the node runs, the node does not execute. Instead, it returns the **error in** value as **error out**.

NOTE
When a node responds differently to these values of **error in**, the differences are documented in the Context Help for the **error in** input of that node.

If you enable error information to flow through the diagram by wiring all error inputs and outputs, you can also use `Error` nodes to process information provided by the error, access additional information about the error, and use that information to make run-time decisions based on the error conditions in the application.

Error Codes

Find and read the description for an error code you encounter in software.

Error Code	Description	Recommended Solution
2147467259	Unspecified error.	N/A
1967345152	Invalid refnum. A specified refnum was not valid.	Specify a valid refnum.
1967345151	Invalid property code. The specified property code is not valid for this refnum.	Specify a valid property for the refnum.
1967345148	Invalid argument. One of the specified arguments is invalid.	Specify a valid argument.
1967345147	Entry not found. The requested entry could not be found.	N/A
375012	A JSON numeric value is out of the range of a G Web Development Software numeric type.	N/A
375011	G Web Development Software failed to read or write a -Infinity or +Infinity floating-point value.	Enable G Web Development Software JSON extensions to allow support for a value of Infinity.
375010	G Web Development Software failed to read or write a NaN floating-point value.	Enable G Web Development Software JSON extensions to allow support for a value of NaN.
375009	The JSON string contains an invalid multi-dimensional array.	N/A

Error Code	Description	Recommended Solution
375008	The array has an invalid number of dimensions.	N/A
375007	The cluster or JSON string contains an invalid number of elements. **NOTE** When strict validation is enabled, the number of elements in the JSON string must match the number of elements in the input cluster.	N/A
375005	Type mismatch between JSON and G Web Development Software.	N/A
375004	The path cannot be found in the JSON string.	N/A
375003	The JSON string is invalid. **NOTE** JSON strings must be encoded in UTF-8 and must conform to the JSON grammar.	N/A
375002	The cluster element name is invalid. **NOTE** Each element of the cluster must have either a unique name or no name, such as an empty string.	N/A
375001	The JSON root container must be an array or cluster.	N/A
375000	The data type cannot be converted to or from JSON. Supported data types include arrays and clusters of booleans, numerics, strings, and other arrays and clusters.	N/A
4824	Clipped Floating-point data to fit the range [-1.0, 1.0].	N/A
4820	A buffer underflow has occurred because the application is not writing data quickly enough.	N/A

Error Code	Description	Recommended Solution
1	An input parameter is invalid. For example if the input is a path, the path might contain a character not allowed by the OS such as ? or @. If you receive this error while developing a large application or storing large sets of data in G Web Development Software, refer to the KnowledgeBase at ni.com for more information.	N/A
2	Memory is full. If you receive this error while developing a large application or storing large sets of data in G Web Development Software, refer to the KnowledgeBase at ni.com for more information.	N/A
4	End of file encountered.	N/A
5	File already open.	N/A
6	Generic file I/O error. A possible cause for this error is the disk or hard drive to which you are trying to save might be full. Another possible cause for this error is there might be a bad network connection. For example, the network connection is down or a network cable is unplugged.	(If applicable) Free up disk space or saving to a different disk or drive.
7	File not found. The file might be in a different location or deleted. Another possible cause for this error is there might be a bad network connection. For example, the network connection is down or a network cable is unplugged.	(If applicable) Use the command prompt or the file explorer to verify that the path is correct.

Error Code	Description	Recommended Solution
8	File permission error. You do not have the correct permissions for the file. If you receive this error while trying to build an executable in G Web Development Software 2021 or later with the Report Generation toolkit 1.1.2 or later, refer to the KnowledgeBase at ni.com for more information.	Verify that the file is not in use by another application and that file permissions are set correctly. • On Windows, navigate to the file, right-click the file, select Properties, and set the Read-only option in the dialog box. • On Linux or Mac OS X, use chmod to set file permissions. If you only need read access, you can use the Open/Create/Replace File function with the access input set to Read-only.
9	Disk full. The disk or hard drive does not have enough free space to complete the operation.	Free up disk space or saving to a different disk or drive.
10	Duplicate path. If you receive this error while building an application in G Web Development Software, refer to the KnowledgeBase at ni.com for more information. If you receive this error while using a File I/O function or VI, you may be attempting to overwrite a file or folder that already exists.	Refer to the specific function or VI reference topic in the *G Web Development Software API Manual* for information about overwriting files or folders.
11	Too many files open.	N/A
50	Message out of range.	N/A
51	Null method.	N/A
52	Unknown message.	N/A
67	Interapplication Manager initialization error.	N/A
68	Bad occurrence.	N/A
69	Handler does not know what occurrence to wait for.	N/A
70	Occurrence queue overflow.	N/A

Error Code	Description	Recommended Solution
71	File datalog type conflict.	N/A
72	Semaphore not signaled.	N/A
73	Interapplication Manager unrecognized type error.	N/A
74	Memory or data structure corrupt.	N/A
81	Format specifier type mismatch. A format specifier does not match the data type of its corresponding argument in a Format Into String or Scan From String function.	N/A
82	Unknown format specifier. A bad format specifier was found in the Format String input to a Format Into String or Scan From String function.	N/A
83	Too few format specifiers. There are not enough format specifiers to match all of the arguments of a Format Into String or Scan From String function.	N/A
84	Too many format specifiers. There are more format specifiers than the number of arguments of a Format Into String or Scan From String function.	N/A
85	Scan failed. The input string does not contain data in the expected format. The Scan From String function was unable to scan its input because the data was not in the expected format. Another possible cause for this error is if the Scan From File or Scan From String functions reach the end of the file.	Right-click the Scan From String function and select Edit Format String to configure the format string to match the input data. Refer to the KnowledgeBase at ni.com for more information about this use case.
87	Error converting to variant. An error occurred converting from G Web Development Software type to OLE variant type.	N/A
100	File contains erroneous data. This normally is a user data file.	N/A

Error Code	Description	Recommended Solution
116	Unflatten or byte stream read operation failed due to corrupt, unexpected, or truncated data.	N/A
117	Directory path supplied where a file path is required.	Provide a file path where required.
118	The supplied folder path does not exist.	N/A
122	The resource you are attempting to open was created in a more recent version of G Web Development Software and is incompatible with this version.	N/A
123	A timeout occurred.	N/A
124	A string contained an unexpected null character.	N/A
125	A stack overflow occurred.	Restart G Web Development Software and review your code for recursive data structures.
1000	The VI is not in a state compatible with this operation. This error can occur for several reasons. This error can occur if you attempt to edit a VI that is running or reserved for running. This error also can occur if you attempt to open a reference to a VI that is running or reserved for running. A VI is reserved for running when you open a reference to the VI by wiring a type specifier VI Refnum to the Open VI Reference function, or when you have a Static VI Reference linked to the VI within a running VI. This error also can occur if you try to run a VI using the run method while the target VI is running or reserved for running. This error also can occur if you attempt to obtain a VI's image while the VI is being modified programmatically.	(If applicable) To correct this error, ensure the target VI is idle or reentrant. If the VI is reentrant: • Use the Open VI Reference function with the options input set to `0x08` to prepare the VI for reentrant run. • Use the Open VI Reference function with the type specifier VI Refnum wired to a strictly typed VI reference. (If applicable) Wait until the VI is not being modified to get the image of a panel or diagram.

Error Code	Description	Recommended Solution
1001	The VI front panel is not open.	• Use the Window Appearance page to configure if the front panel is shown when the VI is run or loaded. • Use the Front Panel:Open method to open the front panel programmatically.
1002	The VI cannot run because it has a front panel control in an error state.	N/A
1003	The VI is not executable. This error may occur because the VI is either broken or contains a subVI that G Web Development Software cannot locate.	**Select File**»**Open** to open the VI and verify that you can run it. Refer to the KnowledgeBase at ni.com if you receive this error in one of the following situations: while using the VI Server to call a VI dynamically or while building an application.
1004	The VI is not in memory.	N/A
1005	VI execution has been disabled in the VI Properties dialog box.	Open the Execution page of the VI Properties dialog box to change the settings for the VI execution.
1006	FPDCO on connector pane thinks it is constant.	N/A
1008	Cannot load connector pane.	N/A
1009	Variant tag out of range.	N/A
1010	No default or operate data.	N/A
1012	Cannot load block diagram.	N/A
1013	Cannot load front panel.	Ensure you included the front panel of the VI when building a stand-alone application.
1016	Cannot load History.	N/A
1017	VI has been modified on disk since it was last loaded or saved.	N/A
1018	Unspecified error occurred.	N/A
1019	One or more untitled subVIs exist. This file cannot be saved until all dependent files have been named.	N/A
1026	VI Reference is invalid.	N/A

Error Code	Description	Recommended Solution
1027	The reference cannot be reserved as requested because it is in an improper reservation mode or the execution state must be set to running or reserved.	N/A
1028	Attribute selector is invalid.	N/A
1039	VI was aborted.	N/A
1042	Attempted recursive call.	N/A
1043	The property or method is not supported in this version of G Web Development Software. This error can occur if you use a property or method that is not supported in the G Web Development Software Run-Time Engine.	N/A
1102	The string wired to the xml string input is empty.	N/A
1103	The XML tag describing the type of data is not recognized.	N/A
1104	No end tag was found for an XML start/end tag pair.	N/A
1105	An unknown or unexpected XML tag was discovered.	N/A
1106	The XML tag describing the type of data does not match the wired type.	N/A
1107	The XML enumerated type choice(s) does not match the wired type.	N/A
1108	XML value text is illegal or out of range for type and/or format.	N/A
1109	Unsupported data type.	N/A
1110	No longer used.	N/A
1122	Refnum became invalid while node waited for it.	N/A

Error Code	Description	Recommended Solution
1124	VI is not loadable. In a built application, this error might occur because the VI being loaded was last compiled for a different OS or with CPU features, such as SSE, that this target does not support. This error also might occur if the VI is a polymorphic VI that cannot be loaded in the G Web Development Software Run-Time Engine.	(If applicable) Rebuild the application for the target OS and make sure SSE compiler settings in the build specifications match the target platform. (If applicable) Load an instance of the polymorphic VI instead of the polymorphic VI itself.
1129	You cannot assign the same numeric value to two or more strings in a ring control. You also cannot assign the same string value to two or more strings in a combo box control.	If you receive this error while trying to write to the Strings and Values property of a control, refer to the KnowledgeBase at ni.com for more information.
1135	The tree control's active item is not valid for this property.	N/A
1136	You wired an invalid item tag to a tree control property or method.	N/A
1137	Tree control's internal data is corrupt.	N/A
1146	You attempted an operation that would change a child-only item into a parent item.	N/A
1147	Cannot insert a remote VI in a subpanel control.	N/A
1148	This property is read only while the VI is in a subpanel.	N/A
1149	Cannot close or set the state of a closed front panel. The front panel must already be open before you close it or set its state.	N/A
1150	Cannot open a front panel that is already open. Set the state of the open front panel by using the Front Panel Window:State property.	N/A
1151	Invalid input for front panel state.	N/A

Error Code	Description	Recommended Solution
1156	Memory full error, possibly due to a data format not matching expected data type. This is caused by trying to allocate a buffer that is too large for memory. Although normal allocation might run the system out of memory, frequently an out of memory error is caused by interpreting the data incorrectly and treating something that is data as the size of the data.	Before assuming that the system needs more memory to complete the operation, make sure the read format you are requesting is valid for the data being read.
1157	You cannot use this property with a numeric indicator.	N/A
1186	Cannot show or hide the label on its own. Label visibility is controlled by the label owner.	N/A
1192	No data range set for digital displays.	N/A
1301	The dimension of the array passed in does not match the expected dimension for the operation.	N/A
1303	The elements of the array are not unique. There are duplicated items in the array.	N/A
1304	The array index is outside of the array bounds.	N/A
1313	You cannot use this property on a control in a radio buttons control.	N/A
1321	NI License Manager is not initialized.	Verify that NI License Manager is installed on the computer. Select **Start**»**All Programs**»**National Instruments**»**NI License Manager**.
1322	Invalid project build reference.	N/A
1359	A drag cannot start because a previous drag transaction is still pending.	N/A
1360	Cannot provide the type of data requested for this drag and drop operation.	Ensure that a drag and drop operation is in progress when you call this function and that the data type and drag data name match what is currently available during the drag and drop operation.

Error Code	Description	Recommended Solution
1362	Cannot use this property with this string display mode or if word wrapping is enabled.	Change the display mode to 0 (normal) and/or disable word wrapping to use this property.
1366	You cannot hide or show the scroll bars of a subpanel control when a VI containing multiple panels is inserted.	N/A
1370	The selected build failed to complete.	N/A
1376	A Diagram Disable structure cannot have a default frame. This error occurs when you try to use the Default Frame property with a Diagram Disable structure. You can use this property only with the Conditional Disable structure.	N/A
1377	A Diagram Disable Structure cannot have conditions. You can use the Conditions property and the Get Frame Index method only with the Conditional Disable structure.	N/A
1378	Cannot set the Active Frame property on a Conditional Disable structure. This error occurs when you try to use the Active Frame property with a Conditional Disable structure. You cannot use this property with a Conditional Disable structure because conditions determine the active frame. You can use this property only with a Diagram Disable structure.	N/A
1380	License checkout failure. Unable to checkout the requested license feature because the license is invalid or does not exist.	N/A
1397	You have not wired a required input on this subVI.	Wire the required input.
1430	The path is empty or relative. You must use an absolute path.	N/A
1432	The specified format cannot be used with floating point data. For example, hexadecimal notation is not a valid format for floating point numbers.	N/A

Error Code	Description	Recommended Solution
1434	The precision is greater than the maximum allowed value for this format.	N/A
1436	Numeric precision cannot be negative.	N/A
1449	Arrays must have at least one dimension and a maximum of 63 dimensions.	N/A
1452	This library was saved in an earlier version of G Web Development Software.	Load and save the library in the latest version of G Web Development Software.
1453	You may only set the vertical arrangement on a label, caption or free label.	N/A
1470	Specified folder is outside the library. You cannot add library items to folders that do not belong to the library.	N/A
1483	Cannot change width of the plot legend when you configure the plot legend to automatically resize, or if the legend has more than one entry and you arrange the plot legend horizontally.	Right-click the graph or chart and deselect Autosize Plot Legend in the shortcut menu to disable automatic resizing and make sure you arrange the plot legend vertically.
1546	The VI must be in a project to use this property.	N/A
1548	Queue refnum cannot be obtained with a size of zero. Size input must be a positive number or -1 for unlimited size.	N/A
1556	The reference is invalid. This error might occur because the reference has been deleted.	N/A
1557	G Web Development Software tried to access duplicate references at the same time.	N/A
1558	There is a type mismatch between the wire type and the internal data type of the reference. This problem might occur due to a type cast error.	N/A
1586	Compiled code is out of date.	N/A
1588	Compiled object cache is corrupt.	N/A

Error Code	Description	Recommended Solution
1599	An empty string is not a valid method name.	N/A
1627	The control index at position %d is invalid.	N/A
1628	G Web Development Software cannot convert the input data type at position %d to the control data type.	N/A
1629	The control must have a terminal on the block diagram.	N/A